EXOS Debug Tools Guide

Lance Richardson

Version 0.8 10/02/2014

Extreme Networks Confidential

Contents

1 Introduction **9**

2 GNU/Linux Development Utilities **11**
2.1 Overview . . . 11
2.2 Debug XMOD Installation . . . 12
2.3 addr2line . . . 12
2.4 gdb . . . 12
2.4.1 Invoking GDB on an Embedded EXOS System . . . 13
2.4.2 Using GDB to Debug Core Files . . . 13
2.4.3 Basic GDB Commands . . . 13
2.5 ldd . . . 14
2.6 nm . . . 15
2.7 objdump . . . 15
2.8 readelf . . . 16
2.9 lsof . . . 16
2.10 strace . . . 17
2.11 tcpdump . . . 17
2.12 ip . . . 18
2.13 mpls . . . 19
2.14 Further Reading . . . 19

3 Core Dumps **21**
3.1 Overview . . . 21
3.2 Quick Start: Obtaining a Core Dump . . . 21
3.3 Mechanics of Core Dumps . . . 21
3.3.1 Core Dump Causes . . . 22
3.4 Core Dump CLI Commands . . . 24
3.4.1 Enabling Core Dumps via EXOS CLI . . . 24
3.4.2 Creating a Snapshot Core Dump . . . 24
3.4.3 CLI Commands for Managing Core Dump Files . . . 24
3.4.4 Transferring Core Dump Files . . . 25
3.5 Analyzing Core Dumps with corePrep.pl . . . 25
3.6 Quick Start: Using corePrep.pl . . . 25
3.6.1 Using corePrep.pl . . . 25
3.6.2 corePrep.pl and GDB Scripts for Debug Automation . . . 26
3.6.3 corePrep.pl Menu Options . . . 27
3.6.4 Customizing corePrep.pl . . . 28
3.7 Kernel Core Dump Controls . . . 28

4 Remote Debugging using GDB **29**
4.1 Overview . . . 29
4.2 Quick Start: Remote Debugging . . . 29
4.3 How Remote Debugging Works . . . 30

5 GDB Automation **31**
5.1 Overview . . . 31
5.2 Automated Macro Loading . . . 31
5.3 GDB Macro Example . . . 31
5.4 Macros Contained in `tools/debug/gdbmacros` . . . 33
5.4.1 Data Structure Handling . . . 33
5.4.2 EMS Trace Buffer Macros . . . 34
5.4.3 Directory Server Client Debugging Macros . . . 34
5.4.4 IPML Debugging Macros . . . 34

6 NVRAM Crash Dumps **37**
6.1 Overview . . . 37
6.2 Showing and Clearing NVRAM Crash Information . . . 37
6.3 NVRAM Crash Information Transfer to CF . . . 38
6.4 Debugging Kernel Crashes from NVRAM Dumps . . . 38
6.4.1 NVRAM Oops Contents . . . 38
6.4.2 Kernel Crash Debugging Quick Start . . . 41
6.5 Debugging Process Crashes from NVRAM Dumps . . . 41
6.5.1 NVRAM Process Crash Data . . . 42
6.5.2 NVRAM Process Crash Debugging Quick Start . . . 45
6.5.3 Analyzing NVRAM Process Crashes with `crashfmt` . . . 46
6.6 NVRAM System Memory Depletion . . . 48
6.6.1 NVRAM System Memory Depletion Data . . . 48
6.6.2 NVRAM Memory Depletion Debugging Quick Start . . . 50

7 Debugging Memory Issues **51**
7.1 Overview . . . 51
7.2 Kernel vs. User-Space Memory Leaks . . . 51
7.3 Heap Fragmentation . . . 51
7.4 Using EXOS Heap Tracing . . . 52
7.4.1 Overview . . . 52
7.4.2 Enabling EXOS heap tracing . . . 52
7.4.3 Displaying Heap Tracing Results . . . 52
7.4.4 Formatting Heap Trace Data . . . 52
7.5 Electric Fence . . . 56
7.5.1 Overview . . . 56
7.5.2 Configuring Electric Fence . . . 56
7.5.3 Caveats Related to Electric Fence . . . 56
7.6 Using /proc/meminfo_better . . . 56
7.7 Using /proc/slabinfo . . . 57
7.8 Tracking Kernel kmalloc() Allocations . . . 57
7.9 Further Reading . . . 59

8 Kernel Debugging **61**
8.1 Using KGDB for Kernel Debugging . . . 61
8.1.1 Building EXOS with KGDB support . . . 61
8.1.2 Remote Debugging with KGDB using the kgdboe I/O module . . . 61
8.1.3 Remote Debugging with KGDB using the kgdboc I/O module . . . 62
8.1.4 Additional information on KGDB . . . 64
8.2 Using GDB for Kernel Debugging . . . 64
8.2.1 Finding Kernel Module Load Addresses . . . 69
8.3 Magic SysRq Facility . . . 69
8.4 Kernel Logging and printk() . . . 69
8.5 Obtaining Scheduler Traces . . . 70

8.6 /proc File System . . . 71
8.7 /debug File System . . . 71

9 Performance Analysis Tools **73**
9.1 Overview . . . 73
9.2 Eprofile . . . 73
9.2.1 Overview . . . 73
9.2.2 Eprofile Quick Start . . . 73
9.2.3 Preparing for Profiling . . . 74
9.2.4 Enabling Profiling . . . 75
9.2.5 Transferring Profile Data . . . 75
9.2.6 Cleanup . . . 76
9.2.7 Analysis of Profile Data . . . 76
9.3 Interpreting Profiling Data . . . 76
9.3.1 eprofmt Output Format . . . 76

10 PACMAN **79**
10.1 Overview . . . 79
10.2 Enabling PACMAN . . . 79
10.2.1 Enabling PACMAN from the PMON Command Prompt . . . 79
10.2.2 Enabling PACMAN from the EXOS CLI . . . 80
10.2.3 Enabling PACMAN from the Linux Shell . . . 80
10.3 PACMAN Options . . . 80
10.3.1 ! - Invoke Comand Shell . . . 80
10.3.2 enable perf hooks . . . 80
10.3.3 boot_cf_alt_fsck . . . 80
10.3.4 boot_cf_config_fsck . . . 80
10.3.5 boot_debug . . . 82
10.3.6 boot_ipaddr msm a . . . 82
10.3.7 boot_ipaddr msm b . . . 82
10.3.8 boot_netmask . . . 82
10.3.9 boot_broadcast . . . 82
10.3.10 boot_gateway . . . 82
10.3.11 var partition . . . 82
10.3.12 nfs var mount . . . 82
10.3.13 start exos at boot . . . 82
10.3.14 enable heap tracing . . . 83
10.3.15 allow /var log file . . . 83
10.3.16 node failure restart . . . 83
10.3.17 mount exos from . . . 83
10.3.18 nfs exos mount . . . 83
10.3.19 nvram core dump info . . . 83
10.3.20 debug mode . . . 83
10.3.21 environment vars . . . 84
10.3.22 pacman autostart . . . 84
10.3.23 pacman autostart delay . . . 84
10.3.24 load settings from system.cfg on hda<*N*> . . . 84
10.3.25 back to original settings . . . 84
10.3.26 save settings to system.cfg on hda<*N*> . . . 84
10.3.27 show settings for COPY . . . 84
10.3.28 read settings from PASTE . . . 84
10.3.29 reboot . . . 84
10.3.30 continue with boot process . . . 84
10.4 Further Reading . . . 85

11 PMON **87**
11.1 Overview . . . 87
11.2 PMON Command Prompt . . . 87
11.3 PMON Commands . . . 87
11.3.1 Boot Image Selection: `boot` . . . 87
11.3.2 System Reboot: `reboot` . . . 87
11.3.3 Configuration Selection: `config` . . . 88
11.3.4 PMON Network Configuration: `configip` . . . 88
11.3.5 PMON IP Ping: `ping` . . . 88
11.4 Debug/Development Mode Commands . . . 88
11.4.1 Display and Configure Boot Command Line: `cmdline` . . . 88
11.4.2 Low-level Boot Command: `pboot` . . . 89
11.4.3 Useful Kernel Initialization Parameters . . . 89
11.5 System Rescue/Recovery via TFTP . . . 89
11.6 System Rescue/Recovery via External CF . . . 90
11.7 Further Reading . . . 90

12 Miscellaneous Tips and Techniques **91**
12.1 Overview . . . 91
12.2 Modifying Individual EXOS Files . . . 91
12.2.1 Modifying Files in the /exos Partition . . . 91
12.2.2 Installing a new Kernel to the /boot Partition . . . 91
12.3 Core Dumps . . . 91
12.3.1 Forcing a Core Dump . . . 91
12.4 Watchdog Timers . . . 92
12.4.1 Forcing a Watchdog Expiration (BD8800) . . . 92
12.4.2 Forcing a Watchdog Expiration (BD10K, x450) . . . 92
12.4.3 Disabling Watchdog Timers . . . 92

13 Debugging Broadcom-Based Platforms **93**
13.1 Overview . . . 93
13.2 Quick Start: Invoking the Broadcom Shell . . . 93
13.3 Navigating the Broadcom Shell . . . 93
13.3.1 Default Unit Selection . . . 93
13.3.2 Getting Help . . . 94
13.3.3 Exiting the Shell . . . 94
13.4 Commonly Used Commands . . . 94
13.4.1 Low-Level Register Reads . . . 94
13.4.2 Low-Level Register Writes . . . 94
13.4.3 Dumping Table Entries . . . 95
13.4.4 Port Counters . . . 95

14 Code Coverage **97**
14.1 Overview . . . 97
14.2 Quick Start: Using Code Coverage . . . 97

A Platform Hardware Configurations **99**
A.1 Stackable Hardware Configurations . . . 99
A.2 Chassis Hardware Configurations . . . 99

B MIPS Programming Architecture **103**
B.1 Overview . . . 103
B.2 CPU Registers . . . 103
B.3 MIPS Coprocessors . . . 104
B.4 MIPS CP0 Registers . . . 104
B.4.1 CP0 Status Register . . . 104
B.4.2 CP0 Cause Register . . . 104
B.4.3 CP0 Status Register . . . 105
B.5 MIPS Virtual Memory Segments . . . 106
B.6 Further Reading . . . 107

C Jerry/Debug Commands **109**
C.1 aaa Debug Commands . . . 109
C.2 acl Debug Commands . . . 109
C.3 bfd Debug Commands . . . 109
C.4 bgp Debug Commands . . . 110
C.5 brm Debug Commands . . . 110
C.6 ces Debug Commands . . . 110
C.7 cli Debug Commands . . . 111
C.8 devmgr Debug Commands . . . 111
C.9 dirser Debug Commands . . . 111
C.10 dot1ag Debug Commands . . . 111
C.11 eaps Debug Commands . . . 112
C.12 elrp Debug Commands . . . 112
C.13 elsm Debug Commands . . . 112
C.14 ems Debug Commands . . . 112
C.15 epm Debug Commands . . . 113
C.16 erps Debug Commands . . . 113
C.17 esrp Debug Commands . . . 114
C.18 ethoam Debug Commands . . . 114
C.19 etmon Debug Commands . . . 114
C.20 exsshd Debug Commands . . . 115
C.21 fcoe Debug Commands . . . 115
C.22 fdb Debug Commands . . . 115
C.23 flowvsr Debug Commands . . . 116
C.24 gptp Debug Commands . . . 116
C.25 hal Debug Commands . . . 116
C.26 hclag Debug Commands . . . 120
C.27 idMgr Debug Commands . . . 121
C.28 isis Debug Commands . . . 121
C.29 lacp Debug Commands . . . 122
C.30 lldp Debug Commands . . . 122
C.31 mcmgr Debug Commands . . . 122
C.32 mpls Debug Commands . . . 123
C.33 mrp Debug Commands . . . 125
C.34 msdp Debug Commands . . . 125
C.35 netLogin Debug Commands . . . 126
C.36 netTools Debug Commands . . . 126
C.37 ntp Debug Commands . . . 127
C.38 openflow Debug Commands . . . 127
C.39 ospf Debug Commands . . . 127
C.40 ospfv3 Debug Commands . . . 128
C.41 pim Debug Commands . . . 128
C.42 polMgr Debug Commands . . . 129

C.43 ptpV2 Debug Commands . . . 129
C.44 pwmib Debug Commands . . . 129
C.45 rip Debug Commands . . . 129
C.46 rtmgr Debug Commands . . . 130
C.47 snmpMaster Debug Commands . . . 131
C.48 snmpMaster Debug Commands . . . 131
C.49 stp Debug Commands . . . 131
C.50 techSupport Debug Commands . . . 131
C.51 telnetd Debug Commands . . . 131
C.52 tftpd Debug Commands . . . 131
C.53 thttpd Debug Commands . . . 131
C.54 trill Debug Commands . . . 132
C.55 upm Debug Commands . . . 132
C.56 vlan Debug Commands . . . 133
C.57 vmt Debug Commands . . . 133
C.58 vrrp Debug Commands . . . 133
C.59 vsm Debug Commands . . . 133
C.60 xmlc Debug Commands . . . 134

D Kernel /proc Filesystem **135**
D.1 /proc/cmdline . . . 135
D.2 /proc/core_dump_info . . . 135
D.3 /proc/core_dump_info_last . . . 135
D.4 /proc/cpu_util_info . . . 135
D.5 /proc/cpuinfo . . . 136
D.6 /proc/crashtest . . . 137
D.7 /proc/devices . . . 137
D.8 /proc/diskstats . . . 137
D.9 /proc/driver . . . 137
D.10 /proc/eprofile . . . 137
D.11 /proc/execdomains . . . 137
D.12 /proc/interrupts . . . 137
D.13 /proc/kallsyms . . . 138
D.14 /proc/meminfo . . . 138
D.15 /proc/meminfo_better . . . 140
D.16 /proc/modules . . . 140
D.17 /proc/pci . . . 140
D.18 /proc/prom_envp . . . 141
D.19 /proc/slabinfo . . . 142
D.20 /proc/extr/nvram/cmdline . . . 142
D.21 /proc/net/dev . . . 143
D.22 /proc/extr/watchdog . . . 143
D.23 /proc/sys/kernel/printk . . . 144
D.24 /proc/vmallocinfo . . . 144

E Kernel /debug Filesystem **147**
E.1 /debug/mips/fpuemustats . . . 147
E.2 /debug/mips/unaligned_instructions . . . 147
E.3 /debug/mips/unaligned_action . . . 147
E.4 /debug/tracing/ . . . 147

Chapter 1

Introduction

This document serves as an introduction and reference guide for the various software and hardware debugging facilities that are available in the EXOS development and run-time environments. The primary target audience for this document is EXOS software development engineers, however much of the information presented here will also be of interest to SQA and customer support engineers.

Debugging and development tools and techniques covered in this document include:

- Using GNU/Linux utilities in the EXOS development environment:
  - gdb (debugging core dumps, remote debugging)
  - binutils (objdump, ldd, nm, etc.)
  - lsof (identify users of files, sockets, devices, etc.)
  - strace (system call tracing and timing)
  - tcpdump (network packet capture)
  - ip, mpls (kernel L3 routing and MPLS table display and modification)
- Debugging memory leaks and other dynamic memory issues:
  - Finding user-space memory leaks with the EXOS heap tracing utility.
  - Detecting buffer overruns and other errors with Electric Fence.
  - Debugging kernel memory leaks.
- Kernel debugging.
  - Using the /proc filesystem.
  - Debugging kernel crashes using NVRAM crash information.
  - Kernel debugging using gdb.
- Performance analysis tools.
  - System profiling with eprofile.
  - Using strace for performance analysis.
- PMON commands.
  - Enabling PACMAN.
  - Recovering from a damaged EXOS installation.
  - Specifying optional parameters for kernel initialization.
- Using and configuring PACMAN.

  - Reducing compile/test cycle time through NFS mounting of the /exos partition.
  - Enabling heap tracing.
- Miscellaneous tips and techniques.

Chapter 2

GNU/Linux Development Utilities

2.1 Overview

A number of standard GNU/Linux utilities useful for debugging are provided in the EXOS development environment, including cross-development tools for MIPS and i686 architectures:

mips-extr473-linux-gnu-addr2line	Convert instruction addresses to source file names and line numbers.
mips-extr473-linux-gnu-gdb	Source-level debugger.
mips-extr473-linux-gnu-ldd	Display shared library depenencies.
mips-extr473-linux-gnu-nm	Display symbol information.
mips-extr473-linux-gnu-objdump	Display object file information.
mips-extr473-linux-gnu-readelf	Display ELF object format information.
mips-extr473-linux-gnu-strings	Extract and display ASCII strings contained in a binary file.

Cross-development tools for x86:

i686-extr473-linux-gnu-addr2line	Convert instruction addresses to source file names and line numbers.
i686-extr473-linux-gnu-gdb	Source-level debugger.
i686-extr473-linux-gnu-ldd	Display shared library depenencies.
i686-extr473-linux-gnu-nm	Display symbol information.
i686-extr473-linux-gnu-objdump	Display object file information.
i686-extr473-linux-gnu-readelf	Display ELF object format information.
i686-extr473-linux-gnu-strings	Extract and display ASCII strings contained in a binary file.

Native tools available under EXOS:

gdb	Source-level debugger.
gdbserver	Remote debugging agent.
genl	Generic netlink utility.
ip	Inspect and modify L3 information in the kernel.
ldd	Display shared library depenencies.
lsof	Display all open file descriptors in a system.
mpls	Inspect and modify MPLS forwarding information in the kernel.
procutils	ps, slabtop, top, and vmstat (full-fledged versions, more capable than busybox).
strace	Trace system calls for a given process.
strings	Extract and display strings from an object file.
tcpdump	Capture and decode network packets.

Cross-development versions of these utilities are available for MIPS and x86 architectures under the EXOS tools installation directory (/mnt/software_tools/x-tools/ or /opt/bldtools/x-tools/) in the mips-extr473-linux-gnu/bin/ and i686-extr473-linux-gnu/bin/ directories. “mips-extr473-linux-gnu-".

Developers are encouraged to add:

```
export PATH=$PATH:/mnt/software_tools/x-tools/mips-extr473-linux-gnu/bin
```

and:

```
export PATH=$PATH:/mnt/software_tools/x-tools/i686-extr473-linux-gnu/bin
```

to their ~/.bashrc in order to more conveniently use these utilities.

2.2 Debug XMOD Installation

A number of debugging tools for use on the target platform are not included in the base EXOS image, instead they are packaged as an installable XMOD module. Tools contained in the debug XMOD include:

- **gdb** - Source-level debugger.
- **gdbserver** - Remote debugging agent.
- **lsof** - List information about files in use by processes.
- **strace** - System call tracing utility.
- **ip** - Provides access to kernel routing tables, devices, tunnels, etc.
- **mpls** - Provides access to MPLS information in the kernel.

The debug XMOD can be found in the <*platform*>/<*release*>/ directory (the same location where the `*.xos` and other `*.xmod` installable files are created). The name is of the form <*platform*>-<*revision-tag*>`-debug.xmod`.

The EXOS CLI command to install the debug XMOD via TFTP is:

download image [<*server-name*> | <*server-ip*>] <*filename*> { [**primary** | **secondary**] }

If the installation partition (primary or secondary) is not specified, the XMOD will be installed to the currently active partition by default.

2.3 addr2line

The **addr2line** utility converts instruction addresses to source file and line number pairs. Typical usage of the **addr2line** command is as follows:

mips-extr473-linux-gnu-addr2line -f -e <*filename*> <*address*>

The ***filename*** parameter given with the `-e` option is the file name of the unstripped executable, shared library, or kernel module that the address is associated with, and the ***address*** parameter is the address to be converted (Note: for shared libraries and kernel modules, the address must be converted to an offset by subtracting the object's load address). The `-f` option specifies that that name of the function containing the address should be displayed in addition to the source file and line number information.

The **addr2line** command is available within the build environment.

2.4 gdb

The GNU debugger, **gdb**, is available on EXOS target systems for the debugging of running applications, and is also available within the build environment as a cross-development utility for processing core files and for remote debugging of processes executing on an EXOS target system.

2.4.1 Invoking GDB on an Embedded EXOS System

GDB can be used natively under EXOS for debugging user-space processes and (with some limitations) debugging the kernel and loadable kernel modules.

To debug user-space applications effectively, gdb requires access to the debugging information contained in the unstripped versions executables and shared libraries. When the `/exos` partition is mounted via NFS (using PACMAN), all EXOS libraries and executables are available in unstripped form, however the libraries on the rootdisk (including glibc and pthread libraries) do not contain debugging information. Because of this, debugging remotely (see Chapter 4 on page 29) is generally easier and more effective.

On an EXOS system, GDB will normally not be used to start the application to be debugged, instead GDB will be attached to an already running process. The command syntax for this is:

gdb −−**pid** <*pid*>

The ***pid*** parameter given with the −−pid option is the ID of the running process to be debugged, as determined via **ps**, **pidof**, or other means.

Because the process being debugged cannot send EPM "heartbeat" messages when stopped, EPM heartbeats must be disabled before gdb is attached to the process. This can be accomplished by using the EXOS CLI command:

debug epm disable heartbeat process <*process-name*>

2.4.2 Using GDB to Debug Core Files

The process of debugging cross-platform core dumps using gdb is greatly simplified through the use of the **corePrep.pl** script. The corePrep.pl script can be found at exos/tools/debug/corePrep.pl, and is invoked as follows:

corePrep.pl {**-n**} <*core_file_name*>

See Chapter 3 on page 21 for more information regarding the use of **corePrep.pl** for debugging EXOS core dumps.

2.4.3 Basic GDB Commands

The following commands are available when running gdb on a live process or core file:

- **bt** - Show call stack (backtrace).
- **where** - Synonym for "bt".
- **help** - Online help.
- **help user** - Online help for user-defined commands (i.e. gdb scripts/macros).
- **info frame** - Display information about current stack frame.
- **info args** - Display information about argument variables (function call parameters) of current stack frame.
- **info break** - Display current breakpoints.
- **info line** *<*addr*> - Display source line number and file name given an instruction address.
- **info locals** - Display local variables.
- **info registers** - Display processor registers.
- **info sharedlib** - Display information about shared libraries.
- **info target** - Display information about address ranges.

- **info threads** - Display all threads.
- **info variables** *{regex}* - Display all variable names, optionally matching regular expression *regex*. Example: **info variables ŷlan.*** will display all global variables with names starting with "vlan".
- **list** - List source code.
- **print** <*expression*> - Display the value of an expression. Note that the expression can include variable names and typecasts.
- **print/x** <*expression*> - Display the value of an expression with numeric values in hexadecimal.
- **quit** - Exit GDB.
- **source** <filename> - Execute commands from the specified file. Useful for loading scripts/macros.
- **thread** *thread-number* - Switch debugging context to specified thread.
- **up,down** - Move up or down to next stack frame in backtrace.

The following commands are only available when running gdb on a live process:

- **break** *location* - Set a breakpoint. Location can be function name, line number, or address ('*' must be prepended to address).
- **call** <*function-name({<arg1>{,<arg2>{,<...>}}}})*> - Call an arbitrary function.
- **continue** - Continue program execution (e.g. after attaching to running program or encountering a breakpoint).
- **delete** - Delete breakpoints.
- **finish** - Continue execution until current function returns.
- **next** - Execute one source line of code (if function call, does not step into function).
- **step** - Step into next source line of code (if function call, steps into function).

The **gdb** command is available in the build and target environments.

2.5 ldd

The **ldd** utility prints the shared library dependencies of an executable. It is invoked on an EXOS platform via:

ldd <*filename*>

Figure 2.1 shows an example of running the `ldd` command on the `epm` executable.
The **ldd** command is available in the target environment.

```
# ldd /exos/bin/epm
        libpthread.so.0 => /lib/libpthread.so.0 (0x2ab40000)
        libcommon.so.0 => /exos/lib/libcommon.so.0 (0x2abc0000)
        libcli.so.0 => /exos/lib/libcli.so.0 (0x2ac40000)
        libcmbackend.so.0 => /exos/lib/libcmbackend.so.0 (0x2acc0000)
        libepm.so.0 => /exos/lib/libepm.so.0 (0x2ad40000)
        libdm.so.0 => /exos/lib/libdm.so.0 (0x2adc0000)
        libwkninfo.so.0 => /exos/lib/libwkninfo.so.0 (0x2ae40000)
        libdl.so.2 => /lib/libdl.so.2 (0x2aec0000)
        libsnmpclient.so.0 => /exos/lib/libsnmpclient.so.0 (0x2af40000)
        libems.so.0 => /exos/lib/libems.so.0 (0x2afc0000)
        libdispatch.so.0 => /exos/lib/libdispatch.so.0 (0x2b040000)
        libc.so.6 => /lib/libc.so.6 (0x2b0c0000)
        libusertrace.so.0 => /exos/lib/libusertrace.so.0 (0x2b280000)
        /lib/ld.so.1 => /lib/ld.so.1 (0x2aac0000)
```

Figure 2.1: Example output from native **ldd** command.

2.6 nm

The **nm** utility lists the symbol information contained in a given object file. It is invoked as follows:

mips-extr473-linux-gnu-nm <*options*> <*filename(s)*>

The ***filename*** parameter is the file name of an object file (executable, shared library, kernel module, vmlinux kernel). Commonly used options for **nm** include:

- **-a** Show all debug symbols.
- **-A** Print name of input file before each symbol.
- **-D** Show all dynamic symbols.
- **-n** Sort output by address.
- **-u** Display only undefined symbols.

The **nm** command is available within the EXOS build environment.

2.7 objdump

The **objdump** utility lists a variety of information from a given object file. It is invoked as follows:

mips-extr473-linux-gnu-objdump <*options*> <*filename*>

The ***filename*** parameter is the file name of an object file (executable, shared library, kernel module, vmlinux kernel). Some of the more commonly used options for objdump include:

-d	Disassemble text sections.
–help	Display full list of command-line options.
-l	Show line numbers.
-S	Interleave source lines with disassembly.
-m	Specify target architecture. Useful for disassembling architecture-dependent instructions. Use “-m mips:sb1" for 1125 and 1250 CPUs.
-T	Show dynamic symbols.
-t	Show symbols (excluding dynamic symbols)

-r	Show relocation entries
-R	Show dynamic relocation entries
−−**adjust-vma=**<**OFF**>	Add <OFF> to each address displayed

When disassembling, the options "`-drRlS -m mips:sb1`" will provide optimal results. When disassembling shared libraries or kernel module code and the load address is known, the "`−−adjust-vma=`" option can make correlating disassembled code with core dump or process/kernel crash information easier.

The **objdump** command is available within the EXOS build environment.

2.8 readelf

The **readelf** utility lists a variety of information from a given object file. It is invoked as follows:

mips-extr473-linux-gnu-readelf <*options*> <*filename(s)*>

The ***filename(s)*** parameter is one or more file names of ELF object files (executable, shared library, kernel module, vmlinux kernel, core, ...).

Some of the more commonly used options for **readelf** include:

–help	Display full list of command-line options.
-e	Display ELF file, program, and section headers.
-s	Display symbols.
-n	Display core notes (for core files).
-x <*n*>	Display hex dump of section *n*.
-w <*opts*>	Dump DWARF debug information.

The **readelf** command has a great deal of overlap with functionality provided by the **objdump** and **nm** utilities, so it is not as frequently used.

The **readelf** command is available within the EXOS build environment.

2.9 lsof

The **lsof** utility lists information about all open file descriptors in a running system. Open file descriptors include descriptors for regular files, directories, device files, libraries, sockets, and /proc files. It is invoked as follows:

lsof {<*options*>} {<*filenames*>}

Commonly used options for **lsof** include:

-c <*command*>	Only display output for identified command name.
+d <*directory*>	Only display output for the specified directory.
-p <*pid1,pid2,...*>	Only display output for identified processes.
–help	Display full list of command-line options.

The information displayed for sockets by the **lsof** command is dependent upon the currently selected network namespace (i.e., the virtual router associated with the process executing **lsof**). If, for example, sockets for the default user VR are of interest, the current VR ID should be set to 3 by executing `echo 3 > /proc/self/ns_id` before executing **lsof**.

The **lsof** command is frequently used in conjunction with the grep command to reduce the volume of output, e.g. `lsof -p 451 | grep sock` to list all sockets opened by process 451.

The **lsof** command is available in the target environment when the debug XMOD has been installed or when `/exos` is mounted via NFS.

2.10 strace

The **strace** command traces system calls and signals for a given process. It can be used to start a new process or attach to an existing process. It is invoked to start a new process as follows:

strace <*options*> <*command*> [<*args...*>]

To attach to an existing process, strace is invoked by:

strace <*options*> **-p** <*pid*>

Commonly used options for **strace** include:

-p <*pid*>	Attach to the identified process.
-c	Keep stats for system calls and report on exit.
-f	Trace child processes created by fork().
-i	Print instruction pointer of caller for each system call.
-T	Show time spent in system call.
-etrace=<*syscall*>	Trace only the specified system call (e.g. -etrace=open,close to trace open() and close() system calls.
-etrace=!<*syscall*>	Don't trace the specified system call (e.g. -etrace=!read to avoid tracing read() system calls.
-e...	See strace man page for more information about filtering strace output.
-s	Set maximum string size to print (default is 32).
-x	Print non-ASCII strings in hexadecimal.

The strace command can be used in conjunction with the grep command to reduce the volume of output.

Note that **lsof** can be used in with **strace** to map process file descriptors to file names.

The **strace** command is available in the EXOS target environment when the debug XMOD has been installed or when `/exos` is mounted via NFS.

2.11 tcpdump

The **tcpdump** command is a command-line packet capture utility available in the target environment. The basic syntax for running **tcpdump** is:

tcpdump { <misc-options> } **-i** <*interface*> { <*filter-expression*> }

The **tcpdump** utility is fully usable for management and control interfaces under EXOS (e.g. eth0 and eth1), but is generally only usable for tracing of outbound packet for VLAN interfaces.

Options for **tcpdump** include:

-c <*count*>	Exit after receiving <*count*> packets.
-e	Print link-level header for each packet.
-i <*interface*>	Listen on the specified interace (default is **eth0**.
-n	Don't convert addresses or port numbers to names.
-P <*slot*:*port*>	Filter on a port list. Portlist entered as <slot>:<port> comma or hyphen separated.
-s <*snaplen*>	Trace <*snaplen*> bytes of each packet (default 68, 0 for entire packet).
-v	Verbose output.
-vv	More verbose output.
-vvv	Maximally verbose output.
-x	Print each packet (minus link header) in hex.

-xx	Print each packet (including link header) in hex.
-X	Print each packet (minus link header) in hex and ASCII.

The network interface can be any interface listed by the `ifconfig` command, including traditional Ethernet interfaces (eth0, eth1, etc), EXOS packet devices (netTx, Broadcom, etc.), and EXOS VLAN network devices.

The `tcpdump` command is sensitive to the currently selected network namespace (i.e., the virtual router associated with the process executing `tcpdump`). Before executing `tcpdump`, the current process VR ID should be set appropriately by e.g. `echo 3 > /proc/self/ns_id` to select VR ID 3 (VR-Default).

The optional <*filter-expression*> is a very flexible mechanism for specifying which packets should be traced by tcpdump. The description of the format of the filter expression is beyond the scope of this documnet, but following are a few examples to illustrate only a tiny fraction of the possibilities:

Only trace packets to \from port 3:

tcpdump -i Broadcom -P 1:3

Only trace packets to \from port 3 and 10 and save them to a pcap file:

tcpdump -i Broadcom -P 1:3,1:10 -w /scratch/intern/packets.pcap

Only trace packets with a given source IP address:

tcpdump -i eth0 src host 10.5.2.1

Only trace packets with a given destination MAC address:

tcpdump -i eth0 dst ether ff:ff:ff:ff:ff:ff

Only trace packets with a given destination MAC address:

tcpdump -i eth0 dst ether ff:ff:ff:ff:ff:ff

Only trace ARP packets:

tcpdump -i eth0 ether proto 0x806

Trace all TELNET packets:

tcpdump -i eth0 tcp port 23

Trace IP multicast packets from a given host:

tcpdump -i eth0 'src host 10.66.4.1 and ip[16] >= 224'

Trace all transmitted and received OSPF packets (note that the filter is duplicated to include both vlan-tagged and non-tagged packets):

tcpdump -s0 -v -i netTx \(ip proto 89\) or \(vlan and ip proto 89\)

Documentation and tutorials for using `tcpdump` can be found at http://www.tcpdump.org/#documentation.

2.12 ip

The **ip** command is a command-line utility for displaying and modifying a variety of L3-related information in the kernel, including interfaces, routes, ARP/neighbour information, and tunnels.

The **ip** command is available in the EXOS target environment when the debug XMOD has been installed or when `/exos` is mounted via NFS. Documentation can be found at http://www.policyrouting.org/iproute2.doc.html

2.13 mpls

The **mpls** command is a command-line utility for displaying and modifying a variety of MPLS-related information maintained in the kernel.

The **mpls** command is available in the EXOS target environment when the debug XMOD has been installed or when `/exos` is mounted via NFS.

2.14 Further Reading

See `http://www.gnu.org/software/gdb/documentation/` and `http://sourceware.org/binutils/docs/binutils/index.html` for more information about these commands.

The book **The Linux Development Platform** by Rafeeq Rehman and Christopher Paul at `http://onlinebooks.library.upenn.edu/webbin/book/lookupid?key=olbp35712` has a good introduction to GDB with many examples.

See `http://www.tcpdump.org/tcpdump_man.html` for a full description of options, filtering, and usage of **tcpdump**.

Chapter 3

Core Dumps

3.1 Overview

Core dumps are the primary means for debugging fatal EXOS software failures in customer, test, and development environments. This chapter describes how to enable core dump capture, how to transfer core dumps off-system, and the basics of core dump analysis.

3.2 Quick Start: Obtaining a Core Dump

The process for acquiring a core dump from a switch is as follows:

1. Enable core dump capture from the EXOS CLI via **configure debug core-dumps internal-memory**. [1]
2. Wait for a core dump (if you want to force the creation of a core dump, see *Forcing a Core Dump* in section 12.3.1 on page 91 or *Creating a Snapshot CoreDump* in section 3.4.2 on page 24).
3. Find the file name of the new core dump via **ls internal-memory**.
4. Transfer the core dump to your workstation/server using tftp via e.g. **tftp put 10.x.x.x internal-memory core.xxx.xxx**
5. (Optional) Remove core file via **rm internal-memory core.xxx.xxx**.

3.3 Mechanics of Core Dumps

In general, a core file will be created by a user-space process upon receiving a fatal signal (see Section 3.3.1 on the following page for a list) if all of the following are true:

- The core file location specified by `/proc/sys/kernel/core_pattern` corresponds to a valid, writable filesystem and directory with sufficient free space to create the core file.
- The process's *RLIMIT_CORE* resource limit is greater than zero (this can be controlled from the shell via "`ulimit -c`" or by an application via the "setrlimit()" API.)
- The process has not overridden the default signal handler for the fatal signal (this is common for security-sensitive applications, and is done to prevent core files from being created that contain sensitive information such as cryptographic keys.)

The default configuration (initialized when `/exos/bin/exos` is executed during startup) has core dumps enabled system-wide, `RLIMIT_CORE` set to "unlimited", and the core file format configured to place core files in the `/scratch/intern/` (a.k.a **internal-memory**) directory.

[1] Beginning with the 11.6.1 EXOS release, core dumps are enabled by default to internal-memory.

3.3.1 Core Dump Causes

Core dumps are created when a process receives a signal for which the action is "terminate process with core dump". The list of signals for which this is the default action includes:

- SIGQUIT - Terminate process with core dump.
- SIGILL - Illegal instruction exception
- SIGTRAP - Trace/breakpoint trap
- SIGABRT - Abort
- SIGEMT - Hardware fault (MIPS only, not originated by kernel)
- SIGFPE - Arithmetic exception
- SIGBUS - Bus Error
- SIGSEGV - Segmentation Violation
- SIGXCPU - CPU limit exceeded
- SIGXFSZ - File size limit exceeded
- SIGSYS - Illegal system call (not originated by kernel)

The mapping of a signal to its numeric equivalent is platform-dependent. In general the mapping can be determined by examining the kernel header file “exos/linux/include/asm-<*arch*>/signal.h”. For convenience, the mappings for MIPS and i386 architectures are listed here:

Signal Name	MIPS	x86
SIGQUIT	3	3
SIGILL	4	4
SIGTRAP	5	5
SIGABRT	6	6
SIGEMT	7	N/A
SIGFPE	8	8
SIGBUS	10	7
SIGSEGV	11	11
SIGXCPU	30	24
SIGXFSZ	31	25
SIGSYS	12	31

These signals and their causes are described below.

SIGQUIT

A process will receive the SIGQUIT signal when Ctrl-\ is entered at terminal associated with the process.

In general, all EXOS processes using terminal I/O should have SIGQUIT masked to prevent a user from terminating a process via Ctrl-\.

SIGILL

A process will receive the SIGILL signal when it attempts to execute an invalid instruction, or when it attempts to execute a privileged instruction in an invalid context.

Often SIGILL is a sign that the process stack has become corrupted. For example, it is possible for a function call return address (normally saved on the stack when the function is entered) to be over-written before the function returns (this can happen as a result of a variety of software bugs). If the corrupted return address happens to be a valid data address for the process and is not unaligned, the process will likely cause an illegal instruction exception.

SIGTRAP

The kernel sends SIGTRAP to a process when a breakpoint or trace exception occurs.

SIGABRT

In the EXOS environment, SIGABRT is usually caused by one of the following conditions:

- A software assertion failure.
- The killing of a process by **epm** which has failed to send application heartbeat messages appropriately.
- The killing of **epm** by the application watchdog kernel module when **epm** has failed to reset the watchdog within the required time interval.

SIGSEGV

SIGSEGV results when an application referencess an address which is outside its virtual address space, or when it makes an inappropriate reference to a valid address (e.g. attempting to write to a read-only address region). When a process is using Electric Fence (see Section 7.5 on page 56), dynamic memory errors (e.g. overrun of a malloc()-ed buffer, use after free) will result in SIGSEGV.

SIGBUS

SIGBUS can have a number of potential causes, including:

- Accessing an invalid address in certain ranges (e.g. reading `0xfffff108` on some MIPS platforms will cause SIGBUS).
- Attempted page-in from an executable that has been modified or deleted from an NFS-mounted file system (this commonly happens when using PACMAN to NFS-mount an EXOS sandbox and the sandbox is rebuilt).
- A hardware error that is potentially unrelated to the application receiving the signal (e.g. uncorrectable ECC memory error, PCI error, gbus timeout). In this case, there will generally be crash information in NVRAM to assist in debugging the specific hardware cause.
- Kernel instruction emulation (e.g. floating point instruction emulation or unaligned load/store emulation) when the instruction to be emulated or memory referenced by the instruction is not accessible.

SIGFPE

SIGFPE can be caused by a variety of arithmetic exceptions, including integer divide by zero, floating point divide by zero, invalid floating point operation, and others.

SIGXCPU

SIGXCPU is sent by the kernel when a process has exceeded its CPU time user limit (normally set by the setrlimit() function call or the “ulimit -t" shell command).

SIGXFSZ

SIGXFSZ is sent by the kernel when a process has exceeded its file size user limit (normally set by the setrlimit() function call or the “ulimit -f" shell command).

SIGEMT and SIGSYS

SIGEMT and SIGSYS are not currently used within the kernel, although this may change in the future. Note that it is still possible to send these signals to a process via the `kill` command or via `kill()/raise()` library functions.

3.4 Core Dump CLI Commands

3.4.1 Enabling Core Dumps via EXOS CLI

The syntax for the EXOS CLI command to enable core dumps is:

configure debug core-dumps [**memory-card** | **internal-memory**]

Either **memory-card** or **internal-memory** can be specified as targets for this command. If **memory-card** is selected (this option is only available when hardware support is present), core dumps will be written to external storage. If **internal-memory** is selected, core dumps will be stored in the `/scratch/intern/` directory in internal compact flash.

3.4.2 Creating a Snapshot Core Dump

Beginning with EXOS version 15.3.1, a "snapshot" core dump can be created for a running process without having to terminate the process by sending it a fatal signal. Snapshot core dumps are created by suspending the process, creating an ELF core file from the current process state, and then resuming the process. Creating a snapshot core dump generally takes 2-3 seconds, so impact to the running system is minimimal.

The following CLI command syntax is used to create a snapshot core dump:

debug epm create core-snapshot process <*process-name*>>

By convention, snapshot core file names have the prefix "snap", normal core files have the prefix "core". An example invocation is shown in figure 3.1.

```
BD-8810.1 # debug epm create core-snapshot process vlan
Core file snapshot created: internal-memory snap.vlan.1362.MSM-A.gz
```

Figure 3.1: Snapshot Core File Creation Example

3.4.3 CLI Commands for Managing Core Dump Files

Core files stored in “internal-memory" or “memorycard" can be listed, copied, renamed, or deleted through the use of the EXOS ls, cp, mv, and rm CLI commands.

The syntax for the EXOS **ls** command is:

ls {[**memory-card** | **internal-memory**]}

To list core files, either **memory-card** or **internal-memory** should be specified as appropriate (by default **ls** lists configuration files in /config).

The **cp** and **mv** commands can be used to copy or move/rename core files. The syntax for the EXOS **cp** command is:

cp {[**memory-card** | **internal-memory**]} <*src-file*> { [**memory-card** | **internal-memory**] } <*dst-file*>

Note that files cannot be moved or copied to **internal-memory** unless the source file is in **internal-memory** as well.

Core files can be deleted by using the **rm** command. The syntax for the EXOS **rm** command is:

rm {[**memory-card** | **internal-memory**]} <*file-spec*>

<file-spec> can be a regular expression, e.g. “core.*".

3.4.4 Transferring Core Dump Files

While core files in external storagecan be transferred by physically transporting the media, transferring via TFTP is usually more convenient.

The syntax for the EXOS CLI **tftp** command is as follows:

tftp [**get** | **put**] <*ip-address*> {[**memory-card** | **internal-memory**]} <*src-file*> { <*dst-file*> }

where “src-file" is used to specify the local file name and the optional “dst-file" parameter is used to specify the remote file name (if not specified, destination filename defaults to the source file name).

3.5 Analyzing Core Dumps with corePrep.pl

3.6 Quick Start: Using corePrep.pl

The **corePrep.pl** script greatly simplifies the process of running gdb to analyze an EXOS core file by automatically extracting information from the core file to build an initialization script for gdb.

Beginning with EXOS 11.6.1, the `corePrep.pl` script can be found in any EXOS sandbox at:

`tools/debug/corePrep.pl`

The **corePrep.pl** script should be executed in the same directory as the core file being debugged, and that directory must be writable.

The main steps required to use the **corePrep.pl** script to analyze a core file are as follows:

1. Execute **corePrep.pl core.xxx.xxx** (compressed core files are OK, if the file extension is "*.gz" corePrep.pl will decompress it automatically.)
2. The **corePrep.pl** script will perform some simple analysis on the core file in order to determine e.g. what toolchain was used to build the executable and the location of the executable build directory, and will then present a menu similar to the one in figure 3.2 on the next page.
3. Note that all information about the image version, build directory, and executable/library locations are initialized by the script automatically. At this point, press enter to receive the prompt “Start GDB (Y/n)?", and press enter again to start gdb on the core file.

3.6.1 Using corePrep.pl

The **corePrep.pl** script is invoked as follows:

corePrep.pl { **-n** } <*core-file-name*>

Where <core-file-name> is the core file being debugged, and the ‘**-n**’ option forces non-interactive mode which bypasses the menu and all prompts. If the core file is compressed in the gzip format, it must have the extension ".gz", otherwise it will be assumed to be uncompressed.

Several restrictions must be observed when using **corePrep.pl**:

1. The core file must reside in the directory where **corePrep.pl** is invoked.
2. The directory where **corePrep.pl** is invoked must be writable.
3. The directory where the image that produced the core file was built must be accessible.
4. The `/mnt/software_tools/` directory must exist and contain the installed toolchains required for building EXOS.

When **corePrep.pl** is started, it performs the following steps:

```
corefiles$ corePrep.pl core.mcmgr.1475.s-1.gz
NOTICE: Core file core.mcmgr.1475.s-1.gz is gzipped, preparing it for processing...Done!

   Core File Name      : core.mcmgr.1475.s-1
   Platform            : cougar
   Architecture        : mips-extr473-linux-gnu
   Executable Name     : mcmgr
   Executable Version  : 4.0.0.2
   Branch Name         : fixes_v1250b10
   Branch Version      : 12.5.0.10
   Build by            : lrichardson
   Build on            : Mon Aug 16 09:15:35 EDT 2010
A) Toolchain location  : /mnt/software_tools/x-tools
   Base /lib directory : /mnt/software_tools/x-tools/mips-extr473-linux-gnu/mips-extr473-linux-gnu/lib
   GDB Path            : /mnt/software_tools/x-tools/mips-extr473-linux-gnu/bin/mips-extr473-linux-gnu-gdb
B) Base EXOS directory : /home/lrichardson/data/fixes10/cougar/cougar
C) EXOS /bin directory : /home/lrichardson/data/fixes10/cougar/cougar/exos/bin
D) EXOS /lib directory : /home/lrichardson/data/fixes10/cougar/cougar/exos/lib
E) libc /lib directory : /home/lrichardson/data/fixes10/cougar/cougar/rootdisk_unstripped/lib
F) Source Directory    : [Use Path from Compile Directory]
G) Autologging         : Enabled
H) Use DDD GUI         : false
   GDB Search Path     : /home/lrichardson/data/fixes10/cougar/cougar/exos/lib:\
                         /home/lrichardson/data/fixes10/cougar/cougar/rootdisk_unstripped/lib:\
                         /mnt/software_tools/x-tools/mips-extr473-linux-gnu/mips-extr473-linux-gnu/lib
   GDB Script          :

echo \nThreads:\n
info threads
echo \nBacktrace (top 20 frames):\n
bt 20
echo \nListing:\n
list


Would you like to change any of the defaults [A-H, <return> to proceed]:
```

Figure 3.2: Example corePrep.pl Menu

1. Decompresses the core file (if necessary).
2. Determines build information from the core file.
3. Creates a GDB script to be executed when GDB starts.
4. Executes ~/.coreprepprc (must contain valid Perl statements) if present.
5. Displays information to be used for invoking GDB, with a menu of options to allow changes (this normally isn't necessary).

Note that, after invoking **corePrep.pl** on a compressed core file, subsequent invocations should specify the uncompressed file name (i.e. without the `.gz` suffix).

After the menu is displayed with the prompt `"Would you like to change any of the defaults [A-H, <return> to proceed]:"`, pressing the "Enter" key will result in the prompt "Start GDB (Y/n)?", and pressing the "Enter" key a second time will start GDB on the core file. Once started, the GDB script created by **corePrep.pl** is executed and the "(gdb)" prompt is displayed.

3.6.2 corePrep.pl and GDB Scripts for Debug Automation

Another feature of the **corePrep.pl** script is automated loading of GDB scripts for debug automation. Scripts will be conditionally loaded from any or all of the following sources in the following order:

1. The file `tools/debug/gdbmacros` for the build associated with the core file.
2. The file `tools/debug/gdbmacros.`<procname>, where <procname> is the executable name that created the core file.
3. The file `$GDBMACRODIR/gdbmacros`, where `GDBMACRODIR` is an environment variable specifying a directory path.
4. The file `$GDBMACRODIR/gdbmacros.`<procname>, where `GDBMACRODIR` is an environment variable specifying a directory path and <procname> is the executable name that created the core file.

GDB scripts can be used to automate many tedious debugging tasks, such as walking complex data structures or automatically checking for known problems based on particular data objects within the process. See Chapter 5 on page 31 for more information regarding GDB scripting and the pre-defined scripts in exos/tools/debug/gdbmacros.

3.6.3 corePrep.pl Menu Options

While the need to modify **corePrep.pl**'s default configuration options is rare, the configuration menu does provide the ability to make such changes when needed. The options presented in the configuration menu are described in the following sections.

A) Toolchain Location

This option specifies the path to the gdb executable as well as certain shared libraries. This path defaults to /**mnt**/**software_tools**/, and falls back to /**usr**/**local**/ if the default path does not exist.

B) Base EXOS directory

This option specifies the top-level path to the build directory for the executable. The strategy used by **corePrep.pl** to determine this path is:

1. If the string "`EXOS_BUILD_DIR=`" exists in the core file, it is assumed to specify the build directory.
2. If the builder ID (retrieved from the core file from the tag string "`EXOSVER_BY=`") is "release-manager", the build directory is assumed to be of the form "`/home/release-manager/Build/`*version*/*platform*/*platform*".
3. If the path to the current directory ends with "`/exos/bin`", the top-level build directory is derived from the current directory.

C) EXOS /bin directory

This path is normally derived from option **B**) by appending "/exos/bin" to the base path.

D) EXOS /lib directory

This path is normally derived from option **B**) by appending "/exos/lib" to the base path.

E) libc /lib directory

This path is normally derived from option **B**) by appending "/rootdisk_unstripped/lib" to the base path.

F) Source Directory

This path is normally derived from option **B**).

G) Autologging

This option controls whether the console I/O for gdb is logged to a file in the current directory. This option efaults to "Enabled" if the version of the locally installed "script" command is at least version 2.12 (the minimum version to support the '-c' option). Note that commands executed from gdb history (e.g. commands retrieved via the "up arrow" key) are not logged.

H) Use DDD GUI

This option controls whether the "`ddd`" front-end for gdb is used when gdb is started. Default is "false".

3.6.4 Customizing corePrep.pl

The behavior of **corePrep.pl** can be modified through the use of a file named ".coreprepprc" in the user's home directory. Perl statements in this file are executed by **corePrep.pl**, and can be used to modify or override various variables. Examples might include:

- "`$interactive=0;`" to make the '-n' option default.
- "`$gdblog = "Disabled";`" to suppress logging.
- "`$gdbscript = join("", $gdbscript, "print foo\n");`" to print a variable automatically on startup.

3.7 Kernel Core Dump Controls

There are several controls that the kernel uses when deciding whether to create a core dump file, where to create it, and what the name of the core file should be.

Core dumps can be enabled or disabled on a per-process basis through the core file size "user limit" (see bash "ulimit -c" command and setrlimit() library function). Core file limits under EXOS currently default to "unlimited".

The name and location of core files is controlled by the `core_pattern` file in `/proc/sys/kernel/`. Current usage is as follows:

- memorycard: `/mnt/a/core.%e.%p.%h`
- internal-memory: `/scratch/intern/core.%e.%p.%h`
- off: `|/bin/bash -c cat > /dev/null` (Note the use of the "pipe" feature for piping the core file to a user-space application.)

When a core file is created, "%e" is replaced with the executable name, "%p" is replaced with the process ID, and "%h" is replaced with the host name (e.g."MSM-A" or "MSM-B").

Chapter 4

Remote Debugging using GDB

4.1 Overview

The **rgdb** script makes it possible to remotely debug EXOS applications across a network with very little effort. Benefits of remote debugging compared to other options include:

- Symbolic and line-number information is always available (PACMAN is not required.)
- Source code is available within the debugger (this is usually not feasible when using native gdb on a switch.)
- Time-consuming add-a-printf()/build/install/run iterations can be eliminated through the use of breakpoints.
- A GUI debugger interface (DDD) can be used if desired.

4.2 Quick Start: Remote Debugging

The essential steps for remotely debugging an EXOS process using **rgdb** are as follows:

1. Ensure that the workstation/server you will be executing rgdb from has access to the directory tree in which the image to be debugged (developer sandbox or release-manager build) was built. Most often this requires that`rgdb` be executed on build server that was used to build the running EXOS image.

2. Ensure that the switch to be debugged has the **gdbserver** executable present in the /exos/tools or /exos/bin directory. If not, the debug tools XMOD must be installed as described in Section 2.2 on page 12.

3. Debug-mode should be enabled on the debug target switch. If debug-mode has not been enabled on the target via PACMAN, you will be prompted to enter the debug-mode password after rgdb has connected to the target.

4. With the above points in addressed, using GDB remotely is a simple matter of starting:

   tools/debug/rgdb *{-d}* <*mgmt-ip-address*> <*process-name*>

   where the *-d* option can be used if it is desired to use the **ddd** GUI front-end for GDB.

5. After completing debugging tasks, executing the `quit` command will gdb from the running process, which will then resume normal operation.

4.3 How Remote Debugging Works

The sequence of steps that are used by the **rgdb** script are as follows:

1. A telnet session is established to the management IP address given on the command line. Login assumes defaults ("admin" is ID, no password).
2. If debug-mode is not already active, it is enabled via **enable debug-mode** and you will be prompted to enter the debug-mode password.
3. The image build directory and TCP port to be used for this remote debug session are determined by issuing the command **jerry show remote-debug process** <*process-name*>.
4. A GDB initialization script is created for this debug session.
5. EPM heartbeat checking is disabled for the process to be debugged by issuing the command **debug epm disable heartbeat process** <*process-name*>.
6. The remote debug agent **gdbserver** is started and attached to the specified process via the CLI command **jerry start remote-debug process** <*process-name*>.
7. GDB is exec-ed (optionally under **ddd**).
8. When GDB exits, heartbeats are re-enabled for the debug target process.

Chapter 5

GDB Automation

5.1 Overview

The macro capability provided by GDB has the potential for drastically reducing the effort required for repetititive debugging tasks through scripting. Some applications for GDB scripting include:

- Automated walking and display of complex data structures including trees, lists, and arrays.
- The ability to display customized summaries of important data structures (e.g. displaying selected structure fields in tabular form with meaningful labels)
- Breakpoint automation (a macro can be configured to execute each time a particular breakpoint is hit.)
- Automated sanity checks of global data structures.

5.2 Automated Macro Loading

Some useful GDB macros are loaded automatically when **corePrep.pl** starts GDB for a core file. The mechanism used by the **corePrep.pl** script is flexible and easily customized. It operates a follows:

1. If a file named `gdbmacros` exists in the `tools/debug` directory corresponding to the build directory of the application that created the core file, it is loaded by gdb for application-independent GDB macros.
2. If a file named `gdbmacros.`*process* exists in the `tools/debug` directory corresponding to the build location of the application that created the core file, it is loaded by gdb for application-specific GDB macros.
3. If the environment variable `GDBMACRODIR` exists and is a directory containing a file named `gdbmacros`, that file is loaded by gdb for user-specific GDB macros.
4. If the environment variable `GDBMACRODIR` exists and is a directory containing a file named `gdbmacros/`*process*, that file is loaded by gdb for application-specific GDB macros.

Process-specific debugging helper macros have been implemented for the EXOS `rtmgr` and `mpls` processes. These can be found in the SVN repository as `code/tools/debug/gdbmacros.rtmgr` and `code/tools/debug/gdbmacros.mpls` respectively, and contain good examples of how frequent debugging tasks can be simplified with a relatively small effort.

5.3 GDB Macro Example

Figure 5.1 on page 33 shows an example of a simple GDB macro that prints each element of the global "sfdArray" array. An explanation of each line follows:

Line 2: Macro Name

Line 2 begins the macro definition. Note that the "end" statement on line 15 terminates the definition.

Line 3: set $i=0

Here we have a variable (note: scope is global for all variables!) to be used as a loop index.

Line 4: Simple printf

Line 4 simply prints headings for the two columns using GDB's built-in "printf" commmand, which functions identically for the most part to the C language "printf()" function.

Line 5: while Loop

Line 5 is the start of a while loop to iterate through the sfdArray, up to the value contained in the variable "sfdArray-Size". Note that this loop is terminated by the "end" command on line 14.

Line 6: if Statement

Here the script is performing an "if" test using the global variable sfdArray[] from the process or core file being debugged and the iteration variable initialized on line 3; entries containing "-1" are skipped. This "if" is termminated by the "end" command on line 12.

Line 7: Assignment and Casting

Line 7 creates another variable named "scb" and sets its value to the contents of sfdArray[$i], typecast to the type (ipmlSockCb_t *). Note that variables in GDB carry both type and value.

Line 8: A More Complex printf

Here a more complex printf format is used, along with dereferencing of fields using the "scb" variable initialized on line 7.

Lines 9 and 10: Sanity Check

The condition being tested for should always be true, here we print a message if the sfdArray entry and index are not consistent.

Line 13: Update Loop Iteration Variable

Here we simply increment $i by 1 to advance to the next array index.

Lines 17-19: Documentation

Lines 17-19 define the text to be displayed by the GDB "help" command. The first line of text should contain a brief summary of the command to be displayed in the output of the "help user-defined" command. Any additional lines would be displayed with the first line in the output of "help print_ipml_sockets" command.

```

define print_ipml_sockets
    set $i=0
    printf "Index Scb        Socket Service\n"
    while $i < sfdArraySize
        if sfdArray[$i] != -1
            set $scb = (ipmlSockCb_t *)sfdArray[$i]
            printf "%-5d %p %-6d %p\n", $i, $scb, $scb->sfd, $scb->service
            if $scb->sfd != $i
                printf "---->Error - scb socket does not match array index\n"
            end
        end
        set $i = $i + 1
    end
end

document print_ipml_sockets
Print all elements of the IPML socket array.
end
```

Figure 5.1: Example GDB Macro

5.4 Macros Contained in `tools/debug/gdbmacros`

5.4.1 Data Structure Handling

The following macros provide common routines for walking and displaying data structures used in EXOS, including doubly-linked lists and trees.

get_struct_from_offset

Syntax:

`get_struct_from_offset` <*struct type*> <*field*> <*address*>

This macro is functionally equivalent to the STRUCT() macro used within EXOS. Returns the base address of a structure given the structure type, the name of a field within the structure, and the address of the named field.

dlcl_list_print

Syntax:

`dlcl_list_print` <*struct type*> <*field*> <*list node*>

This macro prints each node in a doubly-linked list of the given type gven the containing node type, the name of the list node field within the structure, and the address of a node in the list.
Example:
Print all dispatcher work items.

```
dlcl_list_print dispatchWork_t list dispatchWorkList
```

p2btree_print

Syntax:

`p2btree_print` <*struct type*> <*field*> <*tree root addr*>

This macro performs an in-order walk of a p2Btree, displaying each node. Parameters include the type of the associated data structure, the name of the p2Btree field within the structure, and the address of the tree root.
Example:
Print all VLANs by ifInstance.

```
p2btree_print searchTree_vlanIfInstance GvlanIf.searchTree_vlanIfInstance
```

5.4.2 EMS Trace Buffer Macros

These macros are used to inspect and format in-memory EMS developer trace buffers from a core file.

print_ems_trace_info

Syntax:

```
print_ems_trace_info
```

This macro prints summary information about each available EMS trace buffer in the core file, including the address of the buffer to be used with the `print_ems_buffer` command described below.

This macro is roughly equivalent to the EXOS CLI command:

debug ems show trace <*process-name*> info

print_ems_buffer

Syntax:

`print_ems_buffer` <*trace_buffer_address*>

This macro displays the contents of the specified trace buffer in the core file, and is roughly equivalent to the EXOS CLI command:

debug ems show trace <*process-name*> <*buffer-name*>

with the difference that the buffer is specified by address instead of name.

5.4.3 Directory Server Client Debugging Macros

The following macros format and display directory server client data structures.

print_dsclient_reqlist

Syntax:

```
print_dsclient_reqlist
```

This macro prints the list of pending requests from the directory server client library's data structures in the core file.

5.4.4 IPML Debugging Macros

The following macros are available for displaying IPML data structures:

print_ipml_peers

Syntax:

```
print_ipml_peers
```

Prints summary information for IPML peers. Output is similar to that of the run-time EXOS CLI command:

debug epm show process <*process-name*> ipml

print_ipml_service

Syntax:

`print_ipml_service` <*service-address*>

This macro prints the IPML socket control block, service pointer, socket file descriptor, and socket type for the specified ipmlService_t structure.

print_ipml_svc_groups

Syntax:

`print_ipml_svc_groups`

This macro prints summary information for all IPML service groups.

print_ipml_sockets

Syntax:

`print_ipml_sockets`

This macro prints summary information from the IPML socket file descriptor array.

Chapter 6

NVRAM Crash Dumps

6.1 Overview

Under certain conditions the EXOS Linux kernel will store debug information in non-volatile memory; these include:

- Kernel crashes
- Process crashes
- System memory depletion
- Kernel watchdog exceptions
- Kernel panics.

Kernel crash information stored in NVRAM is the primary source of debugging information for kernel crashes and memory depletion situations in customer, SQA, and development environments. In addition, when a full core dump is not available, user process crash information stored in NVRAM can often be used to debug EXOS process crashes.

This chapter describes how to obtain and analyze kernel and process crash information stored in NVRAM.

6.2 Showing and Clearing NVRAM Crash Information

Crash information is saved to non-volatile storage (Battery-backed NVRAM on MSM's and stackables, EEPROM for BD8800 I/O modules) when a kernel crash or kernel oops occurs, or when a process crash occurs. Because NVRAM/EEPROM space is very limited, only about 1.5KB of data can be saved per crash event. There is a total of 3KB allocated in NVRAM/EEPROM, so up to two crash events can be stored simultaneously. When full, the crash information must be cleared before new crash events can be recorded. This crash information is normally cleared quietly during boot/initialization time, this behavior can be changed via PACMAN options when enabled.

There are two /proc file system entries that provide access to NVRAM/EEPROM crash information. Doing "cat /proc/core_dump_info" from a Linux shell prompt will display the current crash information from NVRAM/EEPROM. If the crash information has been cleared (e.g. by "echo clear > /proc/core_dump_info"), "cat /proc/core_dump_info" will produce no output, but "cat /proc/core_dump_info_last" can be used to display the "cleared" crash information.

NVRAM debug information can also be displayed from the EXOS CLI. For EXOS versions prior to 12.0 (Montebello), the command syntax is:

show platform system-dump [**msm** [**a** | **b**] | **slot** <*slotno*>]

In EXOS version 12.0, the command syntax was changed to the following:

show debug system-dump [**msm** [**a** | **b**] | **slot** <*slotno*>]

In EXOS versions prior to 12.0 (Montebello), NVRAM crash information can be permanently erased by issuing the following CLI command:

clear platform system-dump [**msm** [**a** | **b**] | **slot** <*slotno*>]

In EXOS version 12.0, the syntax of the command for permanently clearing NVRAM crash information was changed to:

clear debug system-dump [**msm** [**a** | **b**] | **slot** <*slotno*>]

Note that the **msm** and **slot** parameters for these commands apply only to chassis-based platforms.

6.3 NVRAM Crash Information Transfer to CF

In EXOS 12.0, a mechanism was added by which NVRAM crash information is stored to internal-memory (a.k.a. the `/scratch/intern` directory on internal Compact FLASH) at boot time. When new crash information is found in NVRAM, it is copied to a file named core.nvram.<*sequence-number*> where *sequence-number* is a continuously incrementing integer beginning with 1.

6.4 Debugging Kernel Crashes from NVRAM Dumps

6.4.1 NVRAM Oops Contents

Figure 6.1 on the next page shows an example of a kernel oops, created by executing `echo crashtest_oops > /proc/crashtest`. The contents and interpretation of this kernel oops data are described in the following sections.

Line 1: Failure Type

The **failure** field can contain one of the following values:

- **kernel oops** - The information in this entry is from a kernel crash.
- **process crash** - The information in this entry is from a user-space process crash.
- **system memory depleted** - This entry contains memory allocation statistics for use in debugging memory leak/depletion issues.

Line 2: Failure Reason

The **reason** field contains a string describing the type of crash. Possible values include:

- **NMI**–This crash is currently unique to platforms where an NMI trap is used to signal that a hardware watchdog expiration has occurred.
- **Oops**–One of the following:
  - An unhandled page fault exception has occured within the kernel.
  - A bus error exception has occurred within the kernel.
  - A “coprocessor unusable" exception has occurred within the kernel.
  - A “reserved instruction" exception has occurred within the kernel.
  - An unaligned instruction access exception has occurred within the kernel.
  - An unhandled “unaligned access" exception has occurred within the kernel.
  - An PCI error interrupt (`PCI_SERR` or `PCI_PERR`) has occurred (Aspen I/O modules only).
- **Machine Check**–A “machine check" exception has occurred.

```
failure: kernel oops
reason: Oops
time: 1280755902
where: Oops:0
$0 : z0=00000000 at=30001f00 v0=00000029 v1=00000001
$4 : a0=00000000 a1=30001f00 a2=ffffffff a3=00002088
$8 : t0=ffffffff t1=00130013 t2=00002061 t3=fffffff8
$12: t4=2aac5354 t5=ffffffff t6=f0000000 t7=7fff7610
$16: s0=0000000f s1=87f51df0 s2=87f51f10 s3=2aba4f04
$20: s4=0000000f s5=00000030 s6=00000063 s7=0000002d
$24: t8=00000018 t9=2aba4f04
$28: gp=87f50000 sp=87f51de0 s8=0045c344 ra=801f15c4
Hi : 0000010a
Lo : 3d7ba000
epc  : 801f15c4 write_crashtest+0x2bc/0x328 Tainted: P
Status: 30001f03
Cause : 1080800c
 87f51de0: 878dae90 0000000d 00000000 00000000 848dbe00 2aba4f04 87f4b2a8 00000000
 87f51e00: 878dae90 80710000 63726173 68746573 745f6f6f 70730a00 8e4c5640 2ac36000
 87f51e20: 000000cc 00000000 00000000 2aad6000 00000e90 00000001 87f5f140 80194d94
 87f51e40: 2aba4f04 87f51f30 84a09b60 848dbe00 8e49be48 00000000 848dbe34 2aba4f04
 87f51e60: 87f51f30 80119840 00000010 00000000 00000000 00000010 00010000 00000000
 87f51e80: 00000003 0002ac39 00010000 8edbb920 8ee23320 801e50c8 00000010 00000073
 87f51ea0: 2ac39000 80195164 2ac39000 0000000f 87f51f10 00000802 00000073 8ee23320
 87f51ec0: 2ac39000 801a4058 00000003 00010000 00000000 00000000 00000802 00000030
 87f51ee0: 00000063 0000002d 8ee23320 2ac39000 2ac39000 801a41f4 4c56c8be 04c4d4f1
 87f51f00: 00000802 00000000 00000000 0000000f 00000000 00000000 0000000f 0000000f
 87f51f20: 00000030 0000000f 2ac344d8 80117eac 0047a4fc 0047a40c 0047a434 7fff7e20
 87f51f40: ffffffff 00000000 00000000 30001f00 00000fa4 00000000 00000001 2ac39000
 87f51f60: 0000000f 00000001 07ea0191 0000000d 00000002 2aad6ef8 2aac5354 ffffffff
 87f51f80: f0000000 7fff7610 0000000f 2ac344d8 2ac39000 2aba4f04 0000000f 00000030
 87f51fa0: 00000063 0000002d 2aab8e38 2aba4f04 00000001 00000000 2ac3c950 7fff7700
 87f51fc0: 0045c344 2ab46814 20001f13 00000054 0001b3bc 2aba4f04 00808020 2aba4f28
log: ... e to address zero
log: <1>CPU 0 Unable to handle kernel paging request at virtual address 00000000, epc == 801f15c4, ra == 801f15c4

Version #1 SMP Wed Jul 28 16:33:01 EDT 2 by lrichardson@Syzygy-HLR Release 2.6.28.9cougar

Call Trace:
@[<801f15c4>] write_crashtest+0x2bc/0x328
@[<801e50c8>] proc_reg_write+0x84/0xd4
@[<801a4058>] vfs_write+0xb4/0x158
@[<801a41f4>] sys_write+0x4c/0xa4
@[<80117eac>] stack_done+0x20/0x3c

 [<80194d94>] mmap_region+0x43c/0x5a4
 [<80119840>] do_page_fault+0x160/0x380
 [<80195164>] do_mmap_pgoff+0x268/0x434
Build directory: /home/lrichardson/data/fixes_clean/cougar
-------------------------------
Kernel Panic.
time: 1280755909
Reason: Fatal exception
Caller: 0x8010d6d4 (do_mt+0x0/0x100)
Version #1 SMP Wed Jul 28 16:33:01 EDT 2 by lrichardson@Syzygy-HLR Release 2.6.28.9cougar
Build directory: /home/lrichardson/data/fixes_clean/cougar
```

Figure 6.1: Example NVRAM Kernel Oops Information

Line 3: Time of Crash

The **time** field is stored in raw "Unix" time, which is an integer representing the system time as the number of seconds since 00:00:00 UTC, January 1, 1970. When displayed via the EXOS CLI “show platform system-dump" or “show tech" commands, the time is converted to a human-readable form.

Line 4: Crash Detection Location

The **where** field may contain the file name and line number where the crash was detected.

Lines 5-12: General Purpose Registers

The general-purpose registers are displayed by number and symbolic name. Note that registers $26 and $27 (also known as k0 and k1) are not displayed - these registers are reserved for use by exception handlers, and are generally not meaningful.

See Appendix B.2 on page 103 for more information about MIPS general-purpose registers.

Lines 13-14: MIPS Hi/Lo Registers

MIPS Hi and Lo registers contain the results of integer multiply and divide instructions.

Line 15: Exception Program Counter

The **epc** field contains the address of the instruction that caused the most recent exception, making it the most important single piece of information for debugging a kernel crash.

Following the hexadecimal epc value is the kernel (or kernel module) symbol corresponding to the epc address. The format is:

<*symbol-name*>+<*offset*>/<object-size> { [<*kernel-module-name*>] }

The *kernel-module-name* appears only if the symbol is within a kernel module. For kernel module symbols, the object size is not known and is always displayed as zero.

The final string is the license status of the kernel, which is intended to indicate whether a non-GPL kernel module has been loaded and has therefore “tainted" the kernel.

Line 16: CP0 Status Register

The CP0 Status register contains a variety of mode bits that affect the operation of the CPU. See Appendix B.4.3 on page 105 for a description of the fields in this register.

Line 17: CP0 Cause Register

The CP0 Cause register contains a variety of mode bits that reflect the reasons an interrupt or exception may have occurred. See Appendix B.4.2 on page 104 for a description of the fields in this register.

Lines 18-33: Kernel Stack Data

These lines contain 128 words of stack data, beginning at the current stack pointer (register 29). If the stack depth at the time of the exception was less than 128 words, the locations past the end of the stack are filled with the value 0x1bad2bad.

Lines 34-36: Log Contents

128 bytes of the most recent kernel log message are saved with the crash information.

Line 37: Kernel Version

Kernel version information, including the builder ID, build machine, time and date of build, and kernel version string is saved with each kernel oops as an aid in matching the crash information to a specific build. This information is equivalent to the information stored in `/proc/version`, although subfields may be truncated to fit NVRAM structures.

Line 39-48: Call Trace

This section of output contains the callstack at the point of the crash. One line is output for each value found in the stack data that *could possibly* be part of the actual callstack. Lines beginning with '@' represent the actual callstack, lines without '@' at the beginning are other interesting text symbols found in the stack data.

Line 49: Build Directory

This line contains the path to the EXOS build directory corresponding to the running kernel at the time of the crash.

Line 51-56: Kernel Panic Record

The processing of a kernel oops concludes with a kernel "panic" and reboot. Kernel panic information is always stored in NVRAM to enable debugging in cases where a kernel panic occurs that is not associated with a kernel oops.

6.4.2 Kernel Crash Debugging Quick Start

While there is no general formula for debugging a kernel crash from the information stored in NVRAM, the following steps outline the initial steps that are normally required to begin debugging:

1. Check the log entries in the crash information for suspicious messages. In particular, kernel crashes caused by assertions (e.g. the BUG_ON macro used generically within the kernel, or the sal_assert() function used in the BCM SDK) will generally have useful text messages in the log: section of NVRAM crash data.

2. Disassemble the kernel and any kernel modules identified in the "Call Trace" section of the crash data. This can be done by using "mips-extr473-linux-gnu-objdump -dlrS *filename* > *outfile.txt*", where *filename* is either "vmlinux" for the base kernel or the pathname for the unstripped kernel module. Note that the "vmlinux" must be the one found under exos/linux, not the one included in the image, which is actually a self-decompressing executable.

3. Correlate the epc address with function name and/or source line with the disassembly. For kernel modules the load offset can be computed from a symbol in the callstack in order to perform this correlation (see Section 8.2.1 on page 69 for an alternative method), while addresses from the base kernel can be used directly.

4. The callstack may be useful for determining how the failing function was called and what parameters it was passed.

6.5 Debugging Process Crashes from NVRAM Dumps

Ideally all process crashes will result in a core file which can be analyzed using the **corePrep.pl** script discussed in Chapter 3 on page 21. However, in some cases a core file might not be available, or additional information might be required to make sense of a core file (e.g. when a process has corrupted its stack or has crashed due to a call to an invalid function pointer). In these cases, it is often possible with some effort to debug the crash from the information stored in NVRAM.

The following sections describe the contents of the process crash information stored to NVRAM and some strategies for using it to effectively debug proces crashes.

6.5.1 NVRAM Process Crash Data

Figure 6.2 on the next page shows an example of process crash information from NRAM, the contents of which are described in the following sections.

```
failure: process crash
time: 1280754180
process vlan
pid 1426
signal 6
$0 : z0=00000000 at=2aaecb20 v0=00000004 v1=00000000
$4 : a0=0000000b a1=7fff74c0 a2=00000001 a3=00000001
$8 : t0=00000000 t1=00000410 t2=00000000 t3=00000000
$12: t4=00000000 t5=0a000000 t6=00000400 t7=2ad4f368
$16: s0=7fff7594 s1=00585618 s2=7fff74c0 s3=0000000b
$20: s4=7fff7580 s5=7fff7520 s6=0058b6b0 s7=00000000
$24: t8=00000000 t9=2ae31454
$28: gp=2aeb6950 sp=7fff7480 s8=ffffff86 ra=2aaeb4c8
Hi : 00000a00
Lo : 00000001
epc  : 2ae31478    Tainted: P
Status: 00001f13
Cause : 00808020
 7fff7480: 2ab0f738 2ab0ee54 2ad57030 2adc7a4c 2ab340c0 2ad34410 2aeb6950 2ab0ef04
 7fff74a0: 0058b5e8 00000002 2ab0b1f0 2ad50660 0058b5e8 2ad34670 00000000 2ad50660
 7fff74c0: 00000003 00000002 2ab340c0 2ab0ee54 00000002 2ab0ef04 2ad57030 7fff7594
 7fff74e0: 00585618 7fff7580 7fff752c 0058b6b0 00000000 7fff7520 ffffff8f 2aaecb84
 7fff7500: 2ad57030 2ab5f560 00000303 0001d4c0 7fff7594 7fff7520 2ab0b1f0 00000000
 7fff7520: 00013880 00000303 0001d4c0 00000303 0001d4c0 2ad4f35c 2ad4f368 00000000
 7fff7540: 00013880 00000303 2ad4f378 000f4240 00000014 2ad4f35c 00000014 2ad38864
 7fff7560: 7fff7590 2aad7c48 00000000 00000000 7fff7594 00000000 2ad57030 004cb4cc
 7fff7580: 00000000 2ad36c40 042c7002 00000303 0001d4c0 00000001 00000000 65706d48
 7fff75a0: 656c6c6f 54696d65 72000000 00000000 00000000 00000000 00000000 00000000
 7fff75c0: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 7fff75e0: 00000000 00000000 00000000 00000000 00000000 00000000 00555a30 004cb4cc
 7fff7600: 00000002 00486940 00000001 00000000 004cbf24 2aab7704 00000005 2aad7a18
 7fff7620: 00000000 2aad7ca4 00000000 00000001 00000001 00000000 2aadf010 00000000
 7fff7640: 2ad309cc 00000000 7fff76d4 00520000 0058b6b0 7fff7594 7fff758c 7fff7580
 7fff7660: 7fff76d4 00520000 00520000 004cb4cc 00000002 00486940 00000001 00000000
log: ... cd0, DynCells=0x3330,Ports=0x1d (28 24 4)
log: <6>STM: 11930:23:44 bp: not a ring active 0:0 num 1:1
log: <7>Enable watchdog
log: <7>Enable sibyte watchdog timer

Text segment map
  0x00400000-0x00540000 /exos/bin/vlan
  0x2aaa8000-0x2aac7000 /lib/ld-2.9.so
  0x2ad52000-0x2ae96000 /lib/libc-2.9.so
  0x2aad8000-0x2aaf3000 /exos/lib/libipml.so
  0x2ab04000-0x2ab1c000 /exos/lib/libcommon.so
  0x2ab2e000-0x2ab33000 /exos/lib/libcli.so
  0x2ab44000-0x2ab4f000 /exos/lib/libepm.so
  0x2ab60000-0x2ab83000 /exos/lib/libexpat.so
  0x2ab96000-0x2abd8000 /exos/lib/libvlan.so
  0x2abee000-0x2abf7000 /exos/lib/libds.so
  0x2ac08000-0x2ac1c000 /exos/lib/libcmbackend.so
  0x2ac2e000-0x2ac39000 /exos/lib/libems.so
  0x2ac4a000-0x2ac95000 /exos/lib/libdm.so
  0x2acaa000-0x2acae000 /exos/lib/libhal.so
  0x2acc0000-0x2acc3000 /exos/lib/libsnmpclient.so
  0x2ace8000-0x2acf9000 /exos/lib/libwkninfo.so
  0x2ad30000-0x2ad3f000 /exos/lib/libdispatch.so
Build directory: /home/lrichardson/data/fixes_clean/cougar
```

Figure 6.2: Example NVRAM Process Crash Information

Line 1: Failure Type

The **failure** field can contain either of the following values:

- **kernel oops** - The information in this entry is from a kernel crash.
- **process crash** - The information in this entry is from a user-space process crash.
- **system memory depleted** - This entry contains memory allocation statistics for use in debugging memory leak/depletion issues.

The format of the crash information varies depending upon the failure type.

Line 2: Time of Crash

The **time** field is stored in raw "unix" time, which is an integer representing the system time as the number of seconds since 00:00:00 UTC, January 1, 1970. When displayed via the EXOS CLI “show platform system-dump" or “show tech" commands, the time is converted to a human-readable form.

Line 3: Process Name

This line contains the name of the crashed process. Normally this is identical to the executable name.

Line 4: Process ID

This line contains the process ID of the crashed process.

Line 5: Signal ID

This line contains the numeric ID of the signal associated with the process crash. See Section 3.3.1 on page 22 for more information about signals that can cause process crashes.

Lines 6-13: General Purpose Registers

The general-purpose registers are displayed by number and symbolic name. Note that registers $26 and $27 (also known as k0 and k1) are not displayed - these registers are reserved for use by exception handlers, and are generally not meaningful.

See Appendix B.2 on page 103 for more information about MIPS general-purpose registers.

Lines 14-15: MIPS Hi/Lo Registers

MIPS Hi and Lo registers contain the results of integer multiply and divide instructions.

Line 16: Exception Program Counter

The **epc** field contains the address of the instruction that caused the most recent exception, making it the most important single piece of information for debugging a user process crash. Note that the “Text segment map” information is generally required in order to map the exception program counter to an object file (executable or shared library.)

The final string is the license status of the kernel, which is intended to indicate whether a non-GPL kernel module has been loaded and has therefore “tainted" the kernel.

Line 17: CP0 Status Register

The CP0 Status register contains a variety of mode bits that affect the operation of the CPU. See Appendix B.4.3 on page 105 for a description of the fields in this register.

Line 18: CP0 Cause Register

The CP0 Cause register contains a variety of mode bits that reflect the reasons an interrupt or exception may have occurred. See Appendix B.4.2 on page 104 for a description of the fields in this register.

Lines 19-34: Process Stack Data

These lines contain 128 words of stack data, beginning at the current stack pointer (register 29). If the stack depth at the time of the exception was less than 128 words, the locations past the end of the stack are filled with the value 0x1bad2bad.

Lines 35-38: Log Contents

256 bytes of the most recent kernel log message are saved with the crash information.

Lines 40-57: Text Segment Map

These lines contain the base address of the text section for the executable and each shared library for the process. This information is required to map addresses from the epc, general-purpose registers, and stack contents to offsets within object files.

Line 58: Build Directory

This line contains the path to the EXOS build directory corresponding to the image that was running at the time of the crash.

6.5.2 NVRAM Process Crash Debugging Quick Start

As with a kernel oops, there is no general formula for debugging a process crash from the information stored in NVRAM, but the initial steps are generally as follows:

1. Note what specific signal caused the process crash. Signal 6 (SIGABRT) generally means the process was either killed by epm because it failed to send heartbeat messages frequently enough or it crashed due to an assertion failure. Signal 11 (SIGSEGV) is also common, and usually indicates that an access to memory was made using an invalid pointer. A full list of signals can be found in Section 3.3.1 on page 22.
2. Use the epc address and "Text segment map" to determine the shared library the epc is associated with. For example, in Figure 6.2 on page 43 the epc (0x2ae31478) is in `/lib/libc-2.9.so` because 0x2ae31478 falls within the range 0x2ad52000-0x2ae96000. Knowing this, we can compute the offset of the epc within the shared library by subtracting the load address from the epc. In this case the offset is 0x2ae31478 - 0x2ad52000 = 0xdf478.
3. From the shared library and offset, the source file name, line number, and function name can often be determined by running e.g. "`mips-extr473-linux-gnu-addr2line -e cougar/rootdisk_unstripped/lib/libc-2.9.so -f 0xdf478`".
4. Additionally, the failing code should be disassembled. Continuing this example, running "`mips-extr473-linux-gnu-objdum -dlSr -adjust-vma=0x2ad52000 cougar/rootdisk_unstripped/lib/libc-2.9.so > libc.tmp`" will produce a disassembly of the shared library in which the original epc can be located directly.
5. If a callstack is required, it can often be determined by examining the disassembly at the current epc to determine the size of the current stack frame (look for a constant being added to the sp register) and location the return address (ra register) is saved on the stack (look for register ra being stored at some offset from the sp register). With these two pieces of information, the saved return address can be found in the saved stack contents and the next stack frame's address can be computed by adding the stack frame size to the current stack pointer. This process can be repeated, using the "Text segment map" and objdump at each step as above, to determine the actuall callstack (at least until the saved stack contents are exhausted.)

6.5.3 Analyzing NVRAM Process Crashes with `crashfmt`

The **crashfmt** script (located at `tools/debug/crashfmt` can, in many cases, help to automate the task of analyzing a process crash from data store in NVRAM, and in some cases (e.g. for crashes due to invalid function pointers) can produces a correct callstack when `gdb` cannot.

The syntax for invoking the **crashfmt** script is as follows:

tools/debug/crashfmt <*nvram-file*> <*build-directory*>

Where <*nvram-file*> is the name of the file containing the NVRAM process crash data, and <*build-directory*> is the path to the platform build directory.

Figure 6.3 on the facing page shows an example of using the **crashfmt** script to format NVRAM process crash information, with the process and output described in the following sections:

Line 1: Invoking the crashfmt Script

The **crashfmt** script is invoked for the NVRAM process crash information contained in `core.nvram.1` that was produced by the EXOS image built under `/home/lrichardson/data/fixes10/cougar/cougar`. Note that the path must include the platform directory within the build shadow directory.

Lines 3-24: Computed Callstack

These lines contain for each component of the computed callstack the function name, offset within function, computed stack frame size, offset within object file, and source file/line number.

Line 26: End of Stack

The message on this line indicates that the remainder of the callstack could not be determined because the needed portion of the stack data was outside the 512 bytes that were stored to NVRAM.

Lines 27-42: Disassembly of Callstack Functions

These lines (with large portions deleted for purposes of illustration) contain the disassembly of each function found in the callstack. This is included as a convenience for deeper manual analysis of the crash such as the determination of function call parameters from register and stack state.

```
~/test$ tools/debug/crashfmt core.nvram.1 /home/lrichardson/data/fixes10/cougar/cougar

================================================================================
Computed Callstack
--------------------------------------------------------------------------------
 1: 0x2ae35478 (epoll_wait+0x24)
    Stack frame: 0 bytes at offset 0x0
    Offset 0xdf478 in /lib/libc-2.9.so
    Source: ??:0

 2: 0x2aaeb4c8 (ipmlRecvEpoll+0xa4)
    Stack frame: 128 bytes at offset 0x0
    Offset 0x134c8 in /exos/lib/libipml.so
    Source: /home/lrichardson/data/fixes10/cougar/code/system/ipml/src/ipml.c:1446

 3: 0x2aaecb84 (ipmlRecvOnServiceGroupInternal+0xe0)
    Stack frame: 96 bytes at offset 0x80
    Offset 0x14b84 in /exos/lib/libipml.so
    Source: /home/lrichardson/data/fixes10/cougar/code/system/ipml/src/ipmlServiceGroup.c:431

 4: 0x2ad3c864 (dispatchHandler+0x3e8)
    Stack frame: 296 bytes at offset 0xe0
    Offset 0x8864 in /exos/lib/libdispatch.so
    Source: /home/lrichardson/data/fixes10/cougar/code/system/common/src/dispatch.c:768

stack data at offset 0x204 not available, end of stack trace
================================================================================
Disassembly of functions in callstack
--------------------------------------------------------------------------------
000df454 <epoll_wait>:
epoll_wait():
   df454:     3c1c0008     lui    gp,0x8
   df458:     279c54fc     addiu  gp,gp,21756
   df45c:     0399e021     addu   gp,gp,t9
   ...
00013424 <ipmlRecvEpoll>:
ipmlRecvEpoll():
/home/lrichardson/data/fixes10/cougar/code/system/ipml/src/ipml.c:1428
   13424:     3c1c0002     lui    gp,0x2
   13428:     279cfdcc     addiu  gp,gp,-564
   1342c:     0399e021     addu   gp,gp,t9
   ...
```

Figure 6.3: Usage Example for crashfmt Script

6.6 NVRAM System Memory Depletion

6.6.1 NVRAM System Memory Depletion Data

Memory depletion in EXOS generally results in the system becoming unresponsive at some point, followed shortly by a reboot due to hardware watchdog expiration or application watchdog expiration. As an aid to determining the root cause of the memory depletion, some high-level statistics are collected and stored in NVRAM when the kernel detects that the system is under memory pressure.

Figure 6.4 on the next page shows an example of an NVRAM system memory depletion snapshot, the contents of which are described in the following sections.

Line 3: System memory depleted

This field identifies this NVRAM entry as containing memory depletion debugging information.

Line 4: Time OOM Information was Recorded

The **time** field is stored in raw "unix" time, which is an integer representing the system time as the number of seconds since 00:00:00 UTC, January 1, 1970. When displayed via the EXOS CLI “show platform system-dump" or “show tech" commands, the time is formatted in a human-readable format.

Line 5: Global Memory Stats

This section contains high-level statistics about memory usage. Key values to note include **MemFree, Cached, and LowFree**. The interpretation of these values is described below.

Line 6: MemTotal

This value reflects the total physical DRAM size minus the memory required to initially load the kernel and any memory reserved for use outside the kernel.

Line 7: MemFree

This value is the sum of **HighFree** and textbfLowFree, described below.

Line 8: Buffers

The amount of memory currently in use for block device buffer heads.

Line 9: Cached

The amount of memory currently used by the page cache.

Line 10: Active(anon)

The amount of anonymously-mapped memory currently marked as "Active" (recently used).

Line 11: Inactive(anon)

The amount of anonymously-mapped memory currently marked as "Inactive" (not recently used).

Line 12: Active(file)

The amount of file-mapped memory currently marked as "Active" (recently used).

```

-------------------------------
System memory depleted.
time: 1281974467
Global Memory Stats:
                   MemTotal:       254680 kB
                    MemFree:         2012 kB
                    Buffers:           20 kB
                     Cached:         5860 kB
               Active(anon):        18464 kB
             Inactive(anon):        50484 kB
               Active(file):         3208 kB
             Inactive(file):         2652 kB
                  HighTotal:            0 kB
                   HighFree:            0 kB
                   LowTotal        254680 kB
                    LowFree:         2012 kB
               Committed_AS:       838296 kB
Top Processes by RSS:
                  cliMaster:        27460 kB
                        hal:         3680 kB
                      etmon:         3336 kB
                       xmld:         2232 kB
                        epm:         1108 kB
                  emsServer:         1620 kB
               snmpSubagent:         1060 kB
                       vlan:         1152 kB
                     idMgr:         1116 kB
Top Slab Caches by Memory Consumption:
                  size-2048:       107852 kB
                 size-65536:         8512 kB
               size-2097152:         4096 kB
                  size-8192:         2128 kB
             jffs2_refblock:         1692 kB
                size-524288:         1024 kB
                size-262144:         1280 kB
                size-131072:         1280 kB
                    size-64:         1912 kB
                   size-512:          652 kB
                       UNIX:          600 kB
           sock_inode_cache:          544 kB
                     dentry:          576 kB
                       filp:          600 kB
            jffs2_node_frag:          404 kB
             vm_area_struct:          464 kB

```

Figure 6.4: Example NVRAM Memory Depletion Information

Line 13: Inactive(file)

The amount of file-mapped memory currently marked as "Inactive" (not recently used).

Line 14: HighTotal

The total amount of "high" memory. This memory is not directly addressable by the kernel without mapping via page table entries.

Line 15: HighFree

The total amount of "high" memory that is currently not allocated for any purpose. Note that low values of free memory don't necessarily indicate a problem unless the value of **Cached** is also low.

Line 16: LowTotal

The total amount of "low" memory. This memory is directly addressable in the kernel without page table mappings. Note that when low memory is depleted, the kernel is out of memory regardless of how much memory is available otherwise.

Line 17: LowFree

The total amount of "low" memory that is not allocated for any use. This memory is directly addressable in the kernel without page table mappings. Note that when low memory is depleted, the kernel is out of memory regardless of how much memory is available otherwise. Also note that low values of **LowFree** don't necessarily indicate a problem unless the value of **Cached** is also low.

Line 18: Committed_AS

This value reflects the amount of memory that would be required to satisfy all current allocations (note that the Linux kernel over-commits memory, this value compared with **MemTotal** gives an indication of the current degree to which memory is overcommitted.

Line 19-28: Top Processes by RSS

This section lists the top user-space processes consuming memory.

Line 29-46: Top Slab Caches by Memory Consumption

This section lists the top kernel-space slab caches consuming memory. In some cases the name of the slab cache (e.g. "sock_inode_cache" or "vm_area_struct") is a strong clue to the cause of the leak. In this example, we see that the size-2048 slab cache is consuming about 107 MB, and is most likely the cause of the out-of-memory condition.

6.6.2 NVRAM Memory Depletion Debugging Quick Start

TBD...

Chapter 7

Debugging Memory Issues

7.1 Overview

This chapter addresses the debugging of issues related to dynamic memory allocation in EXOS, both in user-space and in the kernel.

Several facilities are available within EXOS for the debugging of memory leaks, buffer overruns, and other issues related to dynamic memory allocation. These include:

- EXOS heap trace facility
- **Electric Fence**, a buffer overrun debugger

The facilities provided under EXOS for kernel leak debugging are more limited, and tools for kernel buffer overrun debugging are unfortunately non-existent. The facilities that do exist are accessed through /proc filesystem entries:

- Summary memory allocation statistics: `/proc/meminfo_better`
- Slab cache statistics: /proc/slabinfo
- Platform-specific kernel allocation reporting.

7.2 Kernel vs. User-Space Memory Leaks

The tools and techniques for debugging kernel memory leaks versus user-space memory leaks are quite different, so it is important to determine which type of memory leak is occurring up front. Often it is easiest to attempt to find a user-space processes that is responsible for the leak; if none can be found, the kernel is the likely suspect by elimination.

The EXOS commands “show memory" and “top" can be used to identify user-space processes with increasing memory consumption over time. The “top" command is particularly useful since the process list can be sorted by memory usage by pressing ’M’ (Shift-M) while top is running. Top also displays RSS (Resident Set Size, or the number of physical DRAM pages actually used) for each process.

The “show memory <process>" form of the EXOS “show memory" command can also be used to monitor memory usage for a particular process. Note, however, that prior to the 11.6 release of EXOS the memory utilization metric used by "show memory" is virtual memory size, which is generally much larger than the resident set size.

7.3 Heap Fragmentation

Occasionally an EXOS process will be found to have a very large allocation of heap memory (perhaps after a period of high activity), which leads to the suspicion of a memory leak in that process. In such cases it is possible that the

process allocated a large amount of memory during a period of high activity and subsequently freed it properly, but the process still owns this this memory and cannot return it to the operating system.

One scenario where this situation can occur is when the **sbrk** system call is used by malloc() the heap, then at some later time most of the allocated memory that caused the heap to grow was freed, except for perhaps a few allocations near the top of the heap. In this case the memory which has been freed by the process cannot be returned to the operating system since **sbrk** can only reduce the heap size as memory is freed when the objects at the end of the heap are freed.

7.4 Using EXOS Heap Tracing

7.4.1 Overview

A facility for debugging dynamic memory leaks for allocations made via **malloc()**, **calloc()**, and **realloc()** was first introduced in the 11.6 (Mt. Fuji) EXOS release.

Heap tracing can be enabled through any of the following methods:

- Globally for all EXOS processes via PACMAN.
- On a per-process basis in epmrc.
- On a per-process basis from the EXOS CLI.

Note that when heap tracing is enabled via PACMAN or epmrc, allocations are traced from the beginning of execution. If heap tracing is enabled after a process has started, information about earlier allocations is not available.

7.4.2 Enabling EXOS heap tracing

For best results, Heap tracing should be enabled either through the use of the PACMAN "enable heap tracing" option (see Section 10.3.14 on page 83), or through the CLI command:

debug epm enable heaptrace process <*process-name*>

Note that the `debug epm enable heaptrace` command will not take effect until the next time the switch is rebooted.

Heap tracing can also be enabled dynamically for a running process by using the EXOS CLI command:

debug epm configure process <*process-name*> **memory trace 1**

The disadvantage to enabling heap tracing dynamically is that any allocations that were made previously cannot be correctly matched to an allocating function.

7.4.3 Displaying Heap Tracing Results

Regardless of how heap tracing was enabled, the current results can be displayed by issuing the CLI command:

debug epm show process <*process-name*> **memory**

Sample output from this command is shown in Figure 7.1 on the next page

7.4.4 Formatting Heap Trace Data

The addresses and executable/library filenames listed in Figure 7.1 on the facing page can trivially be mapped to corresponding source file and line numbers using the **mips-extr473-linux-gnu-addr2line** utility, but this can be tedious given the large number of entries. Fortunately there are two helper scripts to make this task easier.

The first of these helper scripts is a Perl script called **heaptrcfmt**, which takes as its input a text file containing the output of the **debug epm show process** <*process*> **memory** command. The syntax for using this script is:

exos/tools/perftools/exmmon/heaptrcfmt –sbox <*build-directory*> <*file-name*>

```

(debug) X450a-48t.1 # debug epm show process dirser memory
Global heap statistics
    Total allocated space:        173648
    Number of free blocks:        3
    Number of mmapped regions:    0
    Total mmapped bytes:          0
    Total sbrk bytes:             173648
    Total in-use bytes (non-mmap):169624
    Total free bytes (non-mmap):  4024
    Trimmable space:              3968
EXMMON Statistics:
Build=/home/lrichardson/data/footprint3/cougar/cougar
    Total malloc calls:           1551
    Total free calls:             999
    Total realloc calls:          0
    Alien free/realloc:           0
    Bad index in free/realloc:    0
Detailed heap statistics per caller:
0x2aec5c9c         /exos/lib/libdm.so.0+0x005c9c:    20336 bytes in use 31 mallocs 0 frees 0 reallocs
0x2add0cfc   /exos/lib/libdispatch.so.0+0x010cfc:       28 bytes in use 1 mallocs 0 frees 0 reallocs
0x2aec6aa4         /exos/lib/libdm.so.0+0x006aa4:       80 bytes in use 1 mallocs 0 frees 0 reallocs
0x2ade8e70   /exos/lib/libdispatch.so.0+0x028e70:       72 bytes in use 1 mallocs 0 frees 0 reallocs
0x2ad428dc        /exos/lib/libems.so.0+0x0028dc:      180 bytes in use 6 mallocs 0 frees 0 reallocs
0x2ade2ec8   /exos/lib/libdispatch.so.0+0x022ec8:      184 bytes in use 1 mallocs 0 frees 0 reallocs
0x2ae4815c  /exos/lib/libcmbackend.so.0+0x00815c:     4864 bytes in use 5 mallocs 0 frees 0 reallocs
0x2ac443b8        /exos/lib/libepm.so.0+0x0043b8:        0 bytes in use 104 mallocs 104 frees 0 reallocs
0x2ade429c   /exos/lib/libdispatch.so.0+0x02429c:     1704 bytes in use 71 mallocs 0 frees 0 reallocs
0x00403c50                         ./ds+0x403c50:     6536 bytes in use 86 mallocs 0 frees 0 reallocs
0x2ad427ec        /exos/lib/libems.so.0+0x0027ec:      288 bytes in use 6 mallocs 0 frees 0 reallocs
0x00405e5c                         ./ds+0x405e5c:        0 bytes in use 585 mallocs 585 frees 0 reallocs
0x2ae43db0  /exos/lib/libcmbackend.so.0+0x003db0:     8100 bytes in use 1 mallocs 0 frees 0 reallocs
0x2adea110   /exos/lib/libdispatch.so.0+0x02a110:        0 bytes in use 9 mallocs 9 frees 0 reallocs
0x2ac43890        /exos/lib/libepm.so.0+0x003890:      132 bytes in use 1 mallocs 0 frees 0 reallocs
0x2ac471d4        /exos/lib/libepm.so.0+0x0071d4:       24 bytes in use 1 mallocs 0 frees 0 reallocs
0x2acc14a0        /exos/lib/libcli.so.0+0x0014a0:     8100 bytes in use 1 mallocs 0 frees 0 reallocs
0x2abcb07c     /exos/lib/libcommon.so.0+0x00b07c:     2048 bytes in use 1 mallocs 0 frees 0 reallocs
0x2ac46ddc        /exos/lib/libepm.so.0+0x006ddc:       72 bytes in use 1 mallocs 0 frees 0 reallocs
0x2ab4f938         /lib/libpthread.so.0+0x00f938:       12 bytes in use 1 mallocs 0 frees 0 reallocs
0x2add42dc   /exos/lib/libdispatch.so.0+0x0142dc:    34080 bytes in use 71 mallocs 0 frees 0 reallocs
0x2aec7bd0         /exos/lib/libdm.so.0+0x007bd0:        0 bytes in use 12 mallocs 12 frees 0 reallocs
0x2ade991c   /exos/lib/libdispatch.so.0+0x02991c:        0 bytes in use 1 mallocs 1 frees 0 reallocs
0x2ae440f0  /exos/lib/libcmbackend.so.0+0x0040f0:        0 bytes in use 1 mallocs 1 frees 0 reallocs
0x2ae48130  /exos/lib/libcmbackend.so.0+0x008130:       60 bytes in use 5 mallocs 0 frees 0 reallocs
0x2ae43c70  /exos/lib/libcmbackend.so.0+0x003c70:      212 bytes in use 1 mallocs 0 frees 0 reallocs
0x2ad48bf0        /exos/lib/libems.so.0+0x008bf0:        0 bytes in use 1 mallocs 1 frees 0 reallocs
0x2adc88f4   /exos/lib/libdispatch.so.0+0x0088f4:       44 bytes in use 1 mallocs 0 frees 0 reallocs
0x2ade5128   /exos/lib/libdispatch.so.0+0x025128:        0 bytes in use 1 mallocs 1 frees 0 reallocs
0x2adc90e8   /exos/lib/libdispatch.so.0+0x0090e8:     2088 bytes in use 16 mallocs 7 frees 0 reallocs
0x00404788                         ./ds+0x404788:       24 bytes in use 1 mallocs 0 frees 0 reallocs
0x2adea7dc   /exos/lib/libdispatch.so.0+0x02a7dc:        0 bytes in use 1 mallocs 1 frees 0 reallocs
0x004031e0                         ./ds+0x4031e0:       40 bytes in use 1 mallocs 0 frees 0 reallocs
0x2add197c   /exos/lib/libdispatch.so.0+0x01197c:     2640 bytes in use 10 mallocs 0 frees 0 reallocs
0x2abcbab8     /exos/lib/libcommon.so.0+0x00bab8:       80 bytes in use 15 mallocs 13 frees 0 reallocs
0x2ad496f0        /exos/lib/libems.so.0+0x0096f0:        0 bytes in use 6 mallocs 6 frees 0 reallocs
0x2aec6e34         /exos/lib/libdm.so.0+0x006e34:      356 bytes in use 1 mallocs 0 frees 0 reallocs
0x2aec92c8         /exos/lib/libdm.so.0+0x0092c8:        0 bytes in use 1 mallocs 1 frees 0 reallocs
0x2ad42948        /exos/lib/libems.so.0+0x002948:        8 bytes in use 6 mallocs 0 frees 0 reallocs
0x2abceadc     /exos/lib/libcommon.so.0+0x00eadc:    36624 bytes in use 6 mallocs 0 frees 0 reallocs
0x2adcc490   /exos/lib/libdispatch.so.0+0x00c490:      144 bytes in use 16 mallocs 15 frees 0 reallocs
0x2acc1454        /exos/lib/libcli.so.0+0x001454:        0 bytes in use 1 mallocs 1 frees 0 reallocs
0x2adce0bc   /exos/lib/libdispatch.so.0+0x00e0bc:      528 bytes in use 6 mallocs 0 frees 0 reallocs
0x2aed3a50         /exos/lib/libdm.so.0+0x013a50:       20 bytes in use 1 mallocs 0 frees 0 reallocs
0x2acc3870        /exos/lib/libcli.so.0+0x003870:       24 bytes in use 2 mallocs 0 frees 0 reallocs
0x2adc95d0   /exos/lib/libdispatch.so.0+0x0095d0:      780 bytes in use 65 mallocs 0 frees 0 reallocs
0x2adcb3fc   /exos/lib/libdispatch.so.0+0x00b3fc:        0 bytes in use 5 mallocs 5 frees 0 reallocs
0x2abceac0     /exos/lib/libcommon.so.0+0x00eac0:      432 bytes in use 6 mallocs 0 frees 0 reallocs
0x2acc1434        /exos/lib/libcli.so.0+0x001434:       72 bytes in use 1 mallocs 0 frees 0 reallocs
0x2ae47670  /exos/lib/libcmbackend.so.0+0x007670:       32 bytes in use 2 mallocs 0 frees 0 reallocs
0x2ad494bc        /exos/lib/libems.so.0+0x0094bc:        0 bytes in use 45 mallocs 45 frees 0 reallocs
0x2adc849c   /exos/lib/libdispatch.so.0+0x00849c:     8324 bytes in use 1 mallocs 0 frees 0 reallocs
0x2add9088   /exos/lib/libdispatch.so.0+0x019088:        0 bytes in use 6 mallocs 6 frees 0 reallocs
0x2ac43b24        /exos/lib/libepm.so.0+0x003b24:        0 bytes in use 1 mallocs 1 frees 0 reallocs
0x2add0704   /exos/lib/libdispatch.so.0+0x010704:     2156 bytes in use 77 mallocs 0 frees 0 reallocs
0x2aecd6e8         /exos/lib/libdm.so.0+0x00d6e8:     4964 bytes in use 45 mallocs 0 frees 0 reallocs
0x2ad4292c        /exos/lib/libems.so.0+0x00292c:        8 bytes in use 6 mallocs 0 frees 0 reallocs
0x004068d4                         ./ds+0x4068d4:        0 bytes in use 86 mallocs 86 frees 0 reallocs
0x2add979c   /exos/lib/libdispatch.so.0+0x01979c:        0 bytes in use 65 mallocs 65 frees 0 reallocs
0x2ae47444  /exos/lib/libcmbackend.so.0+0x007444:      576 bytes in use 2 mallocs 0 frees 0 reallocs
0x2abcb0d8     /exos/lib/libcommon.so.0+0x00b0d8:     2048 bytes in use 4 mallocs 0 frees 0 reallocs
```

Figure 7.1: Example Heap Trace Data

The second script is much simpler to use, and its syntax is:

exos/tools/perftools/exmmon/heaptrc <*management-ip*> <*process-name*>

Sample output for the **heaptrc** script is shown in Figure 7.2 on the next page. Note that function name, source file name and line number are provided for each entry and the output is sorted by the number of bytes currently in use by the entry.

```

Bytes     Function                      Mallocs  Frees    Reallocs Source Location
 -------- ----------------------------- -------- -------- -------- ---------------
        0 cliSubagentInit                      1        1        0 code/cfgmgmt/cli/src/cli_subagent.c:168
        0 cmBackendInit                        1        1        0 code/cfgmgmt/cmBackend/src/cm_backend.c:217
        0 dispatchAddClient                    4        4        0 code/system/common/src/dispatch.c:1936
        0 dispatchAddClient                    5        5        0 code/system/common/src/dispatch.c:1924
        0 dmConnect                            1        1        0 code/system/dm/src/dmlib.c:1510
        0 dmCreateIpmlMsg                     12       12        0 code/system/dm/src/dmlib.c:992
        0 dmObjAlloc                           1        1        0 code/system/dm/src/dmdb.c:176
        0 dsBlockGetEntryByAttr                1        1        0 code/system/ds/src/dsClient.c:1899
        0 dsMsgLookupEntries                 588      588        0 code/system/ds/src/ds.c:1154
        0 dsNBGetEntryByAttr                   9        9        0 code/system/ds/src/dsClient.c:1761
        0 dsNBRegister                         1        1        0 code/system/ds/src/dsClient.c:1593
        0 dsRequestAlloc                      11       11        0 code/common/inc/lock.h:115
        0 dsRequestAlloc                      11       11        0 code/system/ds/src/dsClient.c:1212
        0 dsSendReply                         86       86        0 code/system/ds/src/ds.c:1367
        0 dsSetServerAddr                      1        1        0 code/system/ds/src/dsClient.c:212
        0 emsSendClientID                      1        1        0 code/system/ems/src/ems_library.c:3217
        0 emsTxRegData                        45       45        0 code/system/ems/src/ems_library.c:3633
        0 emsTxRegFin                          6        6        0 code/system/ems/src/ems_library.c:3677
        0 emsTxRegReq                          6        6        0 code/system/ems/src/ems_library.c:3579
        0 epmClientRegisterProcess             1        1        0 code/system/epm/src/epmClient.c:445
        0 epmClientThreadKeepAlive          1322     1322        0 code/system/epm/src/epmClient.c:605
        0 epmSendStats                         1        1        0 code/system/epm/src/epmClient.c:891
        0 ipmlFlushMsgQueue                    1        1        0 code/system/ipml/src/ipmlPeer.c:830
        0 ipmlSendVerificationAck             67       67        0 code/system/ipml/src/ipmlTcp.c:283
        0 ipmlVerifyConnection                 6        6        0 code/system/ipml/src/ipmlTcp.c:196
        7 dmSetProcessDependency               1        0        0 code/system/dm/src/dmproc.c:261
        8 emsRegisterComponent                 6        0        0 code/system/ems/src/ems_library.c:488
        8 emsRegisterComponent                 6        0        0 code/system/ems/src/ems_library.c:496
       12 rwlock_add_to_list                   1        0        0 glibc-2.2.5/linuxthreads/rwlock.c:95
       20 dmSetProcessDependency               1        0        0 code/system/dm/src/dmproc.c:255
       24 dsInit                               1        0        0 code/common/inc/lock.h:32
       24 epmClientSetPid                      1        0        0 code/common/inc/lock.h:32
       24 stackAlloc                           2        0        0 code/cfgmgmt/cli/src/stack.c:36
       28 ipmlInitMsgQueueFd                   1        0        0 code/system/ipml/src/ipmlSocket.c:401
       32 cmObject_t_alloc                     2        0        0 code/cfgmgmt/cmBackend/src/cm_backend.c:1698
       40 dsInitPortMgr                        1        0        0 code/system/ds/src/ds.c:273
       44 dispatchInit                         1        0        0 code/system/common/src/dispatch.c:962
       60 bufferAlloc                          5        0        0 code/cfgmgmt/cmBackend/src/buffer.c:39
       72 cliSubagentInit                      1        0        0 code/cfgmgmt/cli/src/cli_subagent.c:161
       72 dsAllocRegister                      1        0        0 code/system/ds/src/dsClient.c:1350
       72 epmClientRxMsg                       1        0        0 code/system/epm/src/epmClient.c:1273
       80 dmHandleNodeState                    1        0        0 code/system/dm/src/dmlib.c:525
       80 exTimerCreate                       15       13        0 code/library/sharedlib/src/exTimer.c:312
      132 epmClientRegisterProcess             1        0        0 code/system/epm/src/epmClient.c:400
      144 dispatchExecuteWorkItem             16       15        0 code/system/common/src/dispatch.c:2322
      180 emsRegisterComponent                 6        0        0 code/system/ems/src/ems_library.c:472
      184 ipmlCreateServiceGroup               1        0        0 code/system/ipml/src/ipmlServiceGroup.c:96
      212 cmBackendInit                        1        0        0 code/cfgmgmt/cmBackend/src/cm_backend.c:151
      256 stackAlloc                           2        0        0 code/cfgmgmt/cli/src/stack.c:40
      288 emsRegisterComponent                 6        0        0 code/system/ems/src/ems_library.c:420
      356 dmUpdateDevInfo                      1        0        0 code/system/dm/src/dmlib.c:708
      432 emsInitTraceBuffer                   6        0        0 code/library/sharedlib/src/ems_trace.c:160
      528 lmAllocLcb                           6        0        0 code/system/common/src/lm.c:40
      576 cmBackendContextAlloc                2        0        0 code/cfgmgmt/cmBackend/src/cm_backend.c:1639
      792 dispatchServerAddPeer               67        1        0 code/system/common/src/dispatch.c:1213
     1728 ipmlQueueInit                       73        1        0 code/common/inc/lock.h:32
     2048 exTimerInit                          1        0        0 code/library/sharedlib/src/exTimer.c:59
     2048 exTimerInit                          4        0        0 code/library/sharedlib/src/exTimer.c:70
     2088 dispatchEntryAlloc                  16        7        0 code/system/common/src/dispatch.c:1098
     2184 ipmlInitScb                         79        1        0 code/system/ipml/src/ipmlSocket.c:300
     2640 ipmlServiceCreate                   10        0        0 code/system/ipml/src/ipmlService.c:215
     4864 bufferAlloc                          5        0        0 code/cfgmgmt/cmBackend/src/buffer.c:43
     4964 dmObjAdd                            45        0        0 code/system/dm/src/dmdb.c:104
     6536 dsAddEntry                          86        0        0 code/system/ds/src/ds.c:485
     8100 cliSubagentInit                      1        0        0 code/cfgmgmt/cli/src/cli_subagent.c:183
     8100 cmBackendInit                        1        0        0 code/cfgmgmt/cmBackend/src/cm_backend.c:173
     8324 dispatchHandler                      1        0        0 code/system/common/src/dispatch.c:705
    20336 dmInit                              31        0        0 code/system/dm/src/dmlib.c:356
    34560 ipmlAllocPeer                       73        1        0 code/system/ipml/src/ipmlPeer.c:692
    36624 emsInitTraceBuffer                   6        0        0 code/library/sharedlib/src/ems_trace.c:167


```

Figure 7.2: Example Formatted Heap Trace Data

7.5 Electric Fence

7.5.1 Overview

Electric Fence is an open-source dynamic memory allocation debugging library originally created by Bruce Perens. This library acts as a replacement for **malloc()**, **calloc()**, **realloc()**, **memalign()**, **valloc()**, and **free()** functions that are normally provided by glibc. By replacing these functions, Electric Fence can detect a number of dynamic memory usage problems, including:

- Buffer overruns or underruns.
- Freeing invalid pointer.
- Double-free.
- Access to a buffer after it has been freed.

Violations detected by Electric Fence cause an immedate SIGSEGV and core dump, pinpointing the precise instruction which is at fault.

7.5.2 Configuring Electric Fence

The CLI command syntax for enabling or disabling Electric Fence are as follows:

debug epm enable heapdbg process <*process-name*>

debug epm disable heapdbg [**process** <*process-name*> | **all**]

Note that the switch must be rebooted before these commands will take effect.

7.5.3 Caveats Related to Electric Fence

1. The mechanism used by Electric Fence to detect buffer overruns can increase memory utilization significantly (note that allocations use an integral multiple of the page size, 4 KB, and the minimum amount of memory used for an allocation is 8 KB). As a result, Electric Fence cannot be used in cases where a large number of memory allocations must be made (e.g. current cliMaster or in scaling testing.)
2. The configuration mechanism for enabling Electric Fence modifies the epmrc file installed on the switch. Once enabled, Electric Fence will remain enabled until explicitly disabled or a new image is installed.
3. Because epmrc is used for configuration, only processes started by EPM can have Electric Fence enabled. Significantly, the processes epm and exsh cannot currently have Electric Fence enabled.
4. In some cases it has been observed that our compiler generates unsafe optimizations, e.g. rounding up the size of a string literal to a multiple of 4 bytes to optimize copying of constant strings. This violation will be detected and reported by Electric Fence.

7.6 Using /proc/meminfo_better

See section D.15 on page 140 for a description of the information contained in `/proc/meminfo_better`. Of particular interest are the entries `LowMemFreeAct`, which reports the amount of free and free-able memory in the region used primarily by the kernel, and `MemFreeAct`, which reports the total amount of free and free-able memory in the system.

7.7 Using /proc/slabinfo

A useful technique for tracking down kernel memory leaks is to monitor the /proc/slabinfo file for slab allocations that increase over time. Since many of the slab caches that are used for dedicated purposes (e.g. socket structures and skb structures) have meaningful names, this can sometimes lead fairly quickly to a specific area of kernel or loadable kernel module code. If the slab cache is one of the generic caches used by kmalloc() (e.g. "size-32", "size-64", "size-2048", etc.), it may be necessary to instrument suspected calls to kmalloc().

The following table describes the format of the /proc/slabinfo file.

Field Number	Description	Usage
1	`name`	Slab cache name.
2	`active_objs`	Number of active (in-use) objects in the slab cache.
3	`num_objs`	Total number (free and in-use) of objects in the slab cache.
4	`objsize`	Object size in bytes.
5	`objsperslab`	Number of objects per slab.
6	`pagesperslab`	Number of 4 KB pages per slab.
7	`: tunables`	Separator (SMP systems only).
8	`limit`	Maximum number of objects that will be cached for each CPU.
9	`batchcount`	Maximum number of free objects that will be transferred to a per-CPU cache when it becomes empty.
10	`sharedfactor`	If non-zero, sharing of objects between CPUs is allowed.
11	`: slabdata`	Separator (SMP systems only).
12	`active_slabs`	Total number of active slabs.
13	`num_slabs`	Total number of slabs (active and inactive).
14	`sharedavail`	Number of available shared objects.

Table 7.1: /proc/slabinfo Field Descriptions

7.8 Tracking Kernel kmalloc() Allocations

A useful tool for debugging kernel memory leaks in the generic slab caches that are used for kmalloc() allocations is the `EXTREME_SLAB_DEBUG` feature. Enabling this feature requires that the macro `CONFIG_EXTREME_SLAB_DEBUG` be enabled in `linux26/include/linux/slab.h` and rebuilding the kernel. Refer to comments in this header file for additional information.

Once enabled, kernel memory allocations made via `kmalloc()` will be tracked on a per-caller basis, similar to what is provided by the EXOS Heap Tracing facility as described in Section 7.4 on page 52. The current status of the top 40 allocating function (sorted by memory usage) can be displayed by executing:

cat /proc/slabdebug | sort -nr | head -n 40

An example of the output from this command is shown in Figure 7.3 on the following page.

In addition to tracking kmalloc() allocations, the `EXTREME_SLAB_DEBUG` feature also enables some basic checks for buffer overruns by maintaining sentinal values at the end of each allocated buffer and verifying the presence of the sentinal when the buffer is freed.

Enabling the `EXTREME_SLAB_DEBUG` feature does impact performance slightly, and may increase memory utilization due to the additional space for tracking that is needed for each buffer.

```
70779200 0x8043d2fc: 16561490 alloc 16418790 free 0 overruns : skb_copy+0x40/0xc0
20356448 0xc26335e0:    12288 alloc     2722 free 0 overruns : s2c_dma_cfg_init+0x22c/0x500 [spath]
19429648 0x803e663c:   201929 alloc   199948 free 0 overruns : mac_frin_replenish+0xac/0x4c8
 5488112 0xc2632fe4:  2116146 alloc  2113567 free 0 overruns : s2c_process_rx_buffers+0x450/0x820 [spath]
 1779400 0x8024410c:     1876 alloc     1659 free 0 overruns : copy_process+0xa4/0x1710
  589920 0x803cfe98:       12 alloc        0 free 0 overruns : msgring_init+0x98/0x170
  392604 0x802bccf8:      741 alloc       96 free 0 overruns : alloc_arraycache+0x28/0x54
  341152 0x80367294:        4 alloc        0 free 0 overruns : jffs2_do_mount_fs+0x364/0x720
  327712 0x8036d0e8:        4 alloc        0 free 0 overruns : jffs2_sum_init+0x58/0xd0
  222540 0x803131f0:     1847 alloc        0 free 0 overruns : __proc_create+0xb0/0x114
  211680 0x803dad24:      630 alloc        0 free 0 overruns : device_create_vargs+0x88/0xf8
  188288 0x8044c14c:      137 alloc        1 free 0 overruns : alloc_netdev_mq+0x58/0x1ec
  147552 0xc26334a0:       12 alloc        0 free 0 overruns : s2c_dma_cfg_init+0xec/0x500 [spath]
  112640 0x80284d84:     2816 alloc        0 free 0 overruns : ring_buffer_alloc+0x1c4/0x3c4
  107888 0x803e6a00:       34 alloc       23 free 0 overruns : mac_frin_replenish+0x470/0x4c8
   98400 0xc26334d8:       12 alloc        0 free 0 overruns : s2c_dma_cfg_init+0x124/0x500 [spath]
   98400 0xc26334b8:       12 alloc        0 free 0 overruns : s2c_dma_cfg_init+0x104/0x500 [spath]
   94688 0xc281ccd4:     1197 alloc      121 free 0 overruns : configVlanAddPort+0x128/0x78c [exvlan]
   79200 0xc262faf4:    18156 alloc    18006 free 0 overruns : prepend_flood_headers+0x298/0x3c4 [spath]
   63000 0x802ff7fc:      948 alloc       73 free 0 overruns : sys_epoll_create1+0xa4/0x184
   48504 0x802dbd9c:      135 alloc       88 free 0 overruns : alloc_fdtable+0x150/0x1c0
   46192 0xc260d640:      645 alloc      595 free 0 overruns : cleanup_module+0x1cc/0x2248 [exosq]
   44925 0x8031d2a8:     3019 alloc       19 free 0 overruns : sysfs_new_dirent+0x4c/0x180
   43056 0x80438158:   619512 alloc   619425 free 0 overruns : sock_alloc_send_skb+0x43c/0x600
   39520 0xc27955c4:      389 alloc        9 free 0 overruns : expkt_packet_sock_destruct+0x1ee4/0x2a04 [nettx]
   38688 0x8027690c:   298517 alloc   297711 free 0 overruns : smp_call_function_mask+0xdc/0x2b4
   36808 0x8035e03c:     3987 alloc     3170 free 0 overruns : jffs2_alloc_full_dirent+0x14/0x20
   31936 0x804b7798:       60 alloc        0 free 0 overruns : snmp_mib_init+0x24/0x44
   31108 0xc260f2c0:       11 alloc        0 free 0 overruns : cleanup_module+0x1e4c/0x2248 [exosq]
   30016 0x802caac4:      141 alloc       74 free 0 overruns : alloc_pipe_info+0x20/0x5c
   24000 0x8043b05c:       85 alloc        1 free 0 overruns : reqsk_queue_alloc+0x16c/0x17c
   21544 0x80250b38:       73 alloc        1 free 0 overruns : __register_sysctl_paths+0x80/0x320
   18704 0x804b3ef4:       15 alloc        1 free 0 overruns : __devinet_sysctl_register+0x8c/0x15c
   16796 0x804eaeb8:       13 alloc        0 free 0 overruns : __addrconf_sysctl_register+0x94/0x174
   16424 0x803e8a2c:        1 alloc        0 free 0 overruns : rmi_phnx_config_spill_area+0x700/0xe3c
   16424 0x803e83a0:        1 alloc        0 free 0 overruns : rmi_phnx_config_spill_area+0x74/0xe3c
   16392 0x8062a1b4:        1 alloc        0 free 0 overruns : ip_rt_init+0xc8/0x1dc
   15400 0x803a52f0:      385 alloc        0 free 0 overruns : percpu_counter_init+0x30/0x50
   15180 0x8039b978:     1017 alloc        9 free 0 overruns : kvasprintf+0x48/0x8c
   14720 0xc281b290:       92 alloc        0 free 0 overruns : addPort+0x58/0x27c [exvlan]

```

Figure 7.3: Example kmalloc() Trace Data

7.9 Further Reading

An overview of memory leak debugging tools for embedded Linux systems is provided at http://www.linuxjournal.com/article/6059. A brief description of the Linux 2.4 kernel slab allocator appears in *Understanding the Linux Kernel* []Bove2003.

Chapter 8

Kernel Debugging

8.1 Using KGDB for Kernel Debugging

KGDB is a source level debugger for the linux kernel. It is used along with GDB to debug a linux kernel. The expectation is that GDB can be used to "break in" to the kernel to inspect memory, variables and look through a call stack information similar to what an application developer would use GDB for. It is possible to place breakpoints in kernel code and perform some limited execution stepping.

Two machines are required for using KGDB. One of these machines is a development machine and the other is the target EXOS switch. The kernel to be debugged runs on the EXOS switch. The development machine runs an instance of GDB against the vmlinux file which contains the symbols (not boot image such as bzImage, zImage, uImage...). In GDB the developer specifies the connection parameters and connects to KGDB.

Before initiating a remote KGDB session it is often useful to turn on printk logging as detailed in section 8.4 on page 69. If KGDB is invoked using the **rgdb** tool this is not necessary because **rgdb** automaticaly turns on printk logging.

8.1.1 Building EXOS with KGDB support

In order to use KGDB the EXOS kernel must be built with KGDB support along with one or more KGDB I/O modules. The available I/O modules support both debugging across the RS232 console connection (kgdboc) or debugging across the network (kgdboe).

- Configure the kernel using Linux menuconfig

  ```
  make -C code linux_pass_menuconfig
  ```

  To enable KGDB select **Kernel hacking**, type `y` for the item **KGDB: kernel debugging with remote gdb** and select this item. The item **KGDB: use kgdb over the serial console** should already be marked as selected with a *, if it is not enable it by typing `y`. Enable the item **KGDB: On ethernet** by typing `y`. Exit up the hiercharchy and then save your changes.

- Overwrite the old configuration file with the newly generated one

  ```
  cp linux26/.config linux26/build/configs/defconfig-<platform>
  ```

8.1.2 Remote Debugging with KGDB using the kgdboe I/O module

The essential steps for remotely debugging an EXOS kernel using **rgdb** across a network are as follows:

1. Ensure that the workstation/server you will be executing **rgdb** from has access to the directory tree in which the kernel to be debugged (developer sandbox or release-manager build) was built. Most often this requires that **rgdb** be executed on the build server that was used to build the running EXOS image.

2. Debug-mode should be enabled on the debug target switch. If debug-mode has not been enabled on the target via PACMAN, you will be prompted to enter the debug-mode password after **rgdb** has connected to the target.
3. With the above points in addressed, using KGDB remotely is a simple matter of starting:

   `tools/debug/rgdb -k {-d} <mgmt-ip-address>`

   where the *-d* option can be used if it is desired to use the **ddd** GUI front-end for GDB.
4. After completing debugging tasks, executing the `quit` command will detach GDB from the target switch, which will then resume normal operation.

How Remote Debugging with KGDB using the kgdboe I/O module works

The sequence of steps that are used by the **rgdb** script are as follows:

1. A telnet session is established to the management IP address given on the command line. Login assumes defaults ("admin" is ID, no password).
2. If debug-mode is not already active, it is enabled via:

   `enable debug-mode`

   and you will be prompted to enter the debug-mode password.
3. The image build directory for this remote debug session are determined by issuing the command:

   `jerry show remote-debug process epm`
4. A GDB initialization script is created for this debug session.
5. The kernel watchdog and any application watchdogs are disabled by issuing the command

   `!echo disableWdg > /proc/extr/watchdog`
6. printk logging at level 8 is enabled by issuing the command:

   `!echo 8 > /proc/sys/kernel/printk`
7. KGDB over Ethernet is enabled by issuing the command:

   `!echo "@/,@' + my_ip + '/" > /sys/module/kgdboe/parameters/kgdboe`
8. GDB is exec-ed (optionally under **ddd**).
9. When GDB exits, KGDB is disabled and the kernel and application watchdogs are re-enabled.

8.1.3 Remote Debugging with KGDB using the kgdboc I/O module

If it is not possible to use the network to debug using KGDB, most likely because you want to debug the kernel bringup and BCM SDK initialization or maybe because you want to insert breakpoints in the the ethernet driver itself you can debug using the serial console I/O module (kgdboc). Before using kgdboc for the first time configure the following in PACMAN:

1. Set pacman option `4 (boot_debug)` in pacman to `true`. amongst other things this will prevent the system from resetting the printk debug level during startup.
2. Set pacman option `7a (start exos at boot)` to `false`. This will cause the system to only initialize as far as the initial shell when it is booted.
3. Type `c` to continue and `y` to save the changed options.

The essential steps for remotely debugging an EXOS kernel using **rgdb** to the console are as follows:

1. Ensure that the workstation/server you will be executing **rgdb** from has access to the directory tree in which the kernel to be debugged (developer sandbox or release-manager build) was built. Most often this requires that **rgdb** be executed on the build server that was used to build the running EXOS image.

2. Reboot the target system. It will stop in the initial shell.

3. Disconnect your terminal server console session so that the debugger can access the console.

4. Start your KGDB session using the command:

   ```
   tools/debug/rgdb -k {-d} <terminal-server-ip-address:terminal-server-port>
   ```

   where the *-d* option can be used if it is desired to use the **ddd** GUI front-end for GDB. rgdb must be invoked from the shadow build area of the image that is currently running in the target switch.

5. The debugger will hit a pre-planted breakpoint in the initialization of the **linux_uk_extr** kernel module which is prior to the initialization of the BCM SDK.

6. Use the `ldmod` command to load symbols for any kernel modules that you wish to debug. Planting a breakpoint at **bcm_init** is probably a good idea. When you have planted your breakpoints continue using the `c` command.

7. Once you have continued in the debugger, unlike with kgdboe, it is impossible to get back to the gdb prompt using `Ctrl/C`. In order to get back to the debugger prompt you must abort the debugger by typing `Ctrl/Z` followed by `kill %1`. Connect back to the console. In order to force the system into the debugger issue the **Sysrq** sequence i.e. type `Ctrl/V Ctrl/X` and then type `g` in order to force the system into the debugger. At this point the console will become unusable and the system will freeze. Disconnect the console and manually invoke the debugger using the command:

   ```
   /opt/bldtools/x-tools/mips-extr473-linux-gnu/bin/mips-extr473-linux-gnu-gdb linux26/vmlinux
   target remote <terminal-server-ip-address>:<terminal-server-port>
   ```

   This should give you a `gdb>` prompt.

8. After completing debugging tasks, executing the `quit` command will detach GDB from the target switch, which will then resume normal operation.

How Remote Debugging with KGDB using the kgdboc I/O module works

The sequence of steps that are used by the **rgdb** script are as follows:

1. A telnet session is established to the terminal server in order to access the console port.

2. The kernel watchdog and any application watchdogs are disabled by issuing the command

   ```
   echo disableWdg > /proc/extr/watchdog
   ```

3. printk logging at level 8 is enabled by issuing the command:

   ```
   echo 8 > /proc/sys/kernel/printk
   ```

4. printk console output to the debugging session is enabled using the command:

   ```
   echo 1 > /sys/module/kgdb/parameters/kgdb_use_con
   ```

5. KGDB using the console is enabled by issuing the command:

   ```
   echo ttyS0 > /sys/module/kgdboc/parameters/kgdboc
   ```

6. The system is forced to continue using the commands:

   ```
   rm -f /tmp/exos.disable
   exit
   ```

7. The telnet session used to control the console is disconnected.

8. GDB is exec-ed (optionally under **ddd**) using the command:

   ```
   /opt/bldtools/x-tools/mips-extr473-linux-gnu/bin/mips-extr473-linux-gnu-gdb linux26/vmlinux
   target remote <terminal-server-ip-address>:<terminal-server-port>
   ```

   where `<terminal-server-ip-address>` and `<terminal-server-port>` are the IP address of the terminal server and the port serving the console for the target system respectively.

9. When GDB exits, KGDB is disabled and the kernel and application watchdogs are re-enabled.

8.1.4 Additional information on KGDB

- There appears to be a bug in GDB concerning symbols for data items in kernel modules. The **ldmod** command that is defined by the **rgdb** script enables the loading of symbols for kernel modules. It does this correctly for routine names, i.e. code that exists in the .text section. But it appears to report incorrect addresses for data symbols that are in the the .data or .bss sections. These symbols are incorrectly reported relative to the .text section. In order to calculate the correct location obtain the reported address of the symbol using the command:

  ```
  print &symbol-name
  ```

  For a data symbol add the base of the .data section and subtract the base of the .text section (as reported by ldmod). For a .bss symbol use the .bss section address instead.

- Further, but somewhat outdated, information on KGDB may be found at:

  https://www.kernel.org/pub/linux/kernel/people/jwessel/kgdb/

8.2 Using GDB for Kernel Debugging

If live kernel debugging is not possible using KGDB (maybe because KGDB is not built into the kernel) GDB may be used for some kernel debugging tasks such as inspection of kernel variables and data structures. The `/proc/kcore` pseudo-file provides a user-space representation of kernel memory that behaves as if it is an ELF core file.

The steps for using GDB on a kernel are:

1. Transfer the unstripped vmlinux file to the `/scratch/intern` directory on the switch. The unstripped vmlinux file can be found at `linux26/vmlinux`.

2. Start GDB via:

   ```
   gdb /scratch/intern/vmlinux /proc/kcore
   ```

   At this point the well-known GDB commands can be used to display kernel data structures of interest.

There are a few caveats associated with using GDB for kernel debugging. The main restriction is that no modifications of kernel data are allowed (remember that, as far as GDB is concerned, it is being used to debug a core file.) Also, note that GDB caches chunks of data as they are read from a core file. This means that multiple successive reads of the same variable will continue to display the same value(s), even if they have changed in the kernel. The solution to the issue is to issue the gdb command "`core-file /proc/kcore`" whenever cached data needs to be refetched from the kernel.

It is also possible to examine data within kernel modules using /proc/kcore using the following steps:

1. Transfer an unstripped version of the kernel module to the `/scratch/intern` directory (or access via NFS mount).

2. Determine the load addresses for text and data sections of the kernel module via:

   `grep "" /sys/module/<`*module-name*`>/sections/.*`

   For example, this would be done for the `exvlan.ko` module via:

```
grep "" /sys/module/exvlan/sections/.*
```

For each section of interest (typically only `rodata`, `bss`, `text`, and `data`), record the section's load address.

3. With GDB started as above (e.g. `gdb /scratch/intern/vmlinux /proc/kcore`), load the symbol information for the kernel module by issuing the command "`add-symbol-file` <*module-filename*> <*text-section-address*> `-s .data` <*data-section-address*> `-s .bss` <*bss-section-address*> `-s .rodata` <*rodata-section-address*>

Figure 8.1 on the following page shows a sample kernel debugging session using GDB, with a line-by-line explanation as follows:

- **Line 1:** Here we transfer the unstripped `vmlinux` file to the `/scratch/intern` directory using TFTP. Note the use of the `-b` "blocksize" option for quicker download and to avoid issues with some TFTP servers for file sizes greater than 32 MB.
- **Line 2:** GDB is started using `vmlinux` as the executable and `/proc/kcore` as the core file.
- **Lines 3-12:** GDB displays its startup banner and initial output.
- **Lines 13-14:** The kernel global variable `jiffies` is displayed.
- **Lines 15-16:** The kernel global variable `jiffies` is displayed again, with the same result due to the previous value being cached.
- **Lines 17-20:** Cached values (including `jiffies`) are cleared by forcing GDB to reload `/proc/kcore`.
- **Lines 21-22:** The kernel global variable `jiffies` is displayed once again, this time with a more recent value due to the clearing of cached data.
- **Lines 23-40:** Various other kernel variables are displayed.
- **Line 41:** Finally, we exit the GDB debugger.

```
/scratch/intern # tftp -b 1400 -g -r vmlinux 10.5.2.158
/scratch/intern # gdb vmlinux /proc/kcore
GNU gdb 6.8
Copyright (C) 2008 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copying"
and "show warranty" for details.
This GDB was configured as "mips-extr473-linux-gnu"...
Core was generated by ‘root=/dev/ram0 ro init=/linuxrc quiet dhash_max=5 ihash_max=5 mhash_max=4 bhash_’.
[New process 0]
#0  0x00000000 in ?? ()
(gdb) p jiffies
$1 = 703882
(gdb) p jiffies
$2 = 703882
(gdb) core-file /proc/kcore
Core was generated by ‘root=/dev/ram0 ro init=/linuxrc quiet dhash_max=5 ihash_max=5 mhash_max=4 bhash_’.
[New process 0]
#0  0x00000000 in ?? ()
(gdb) p jiffies
$3 = 707126
(gdb) p envp_init
$4 = {0x804485fc "HOME=/", 0x80448604 "TERM=linux", 0x81200ce6 "dhash_max=5",
  0x81200cf2 "ihash_max=5", 0x81200cfe "mhash_max=4",
  0x81200d0a "bhash_max=7", 0x81200d16 "pacman=1",
  0x81200d1f "kgdboc=duart0,9600", 0x0 <repeats 26 times>}
(gdb) p script_format
$5 = {lh = {next = 0x8049aef0, prev = 0x8049b360}, module = 0x0,
  load_binary = 0x801e0050 <load_script>, load_shlib = 0, core_dump = 0,
  min_coredump = 0, hasvdso = 0}
(gdb) x/4x hardirq_stack
0x806f4000 <hardirq_stack>:     0x00000000      0x00000000      0x00000000      0x00000000
(gdb) p printk
$6 = {int (const char *, ...)} 0x803fbdd8 <printk>
(gdb) info line *0x803fbdd8
Line 571 of "kernel/printk.c" starts at address 0x803fbdd8 <printk>
   and ends at 0x803fbddc <printk+4>.
(gdb) p expkt_rcv_pkt_hook
$7 = (struct sk_buff *(*)(struct sk_buff *)) 0xc35372b0
(gdb) quit
/scratch/intern #
```

Figure 8.1: Example Kernel Debugging Session with GDB

Figure 8.2 on the next page shows a sample kernel module debugging session using GDB, with a line-by-line explanation as follows:

- **Line 1:** Here we transfer the unstripped `vmlinux` file to the `/scratch/intern` directory using TFTP. Note the use of the `-b` "blocksize" option for quicker download and to avoid issues with some TFTP servers for file sizes greater than 32 MB.
- **Line 2:** Here we transfer the unstripped `exvlan.ko` file to the `/scratch/intern` directory using TFTP.
- **Lines 3-16:** Module load addresses for the `exvlan` kernel module are obtained from information in the `/sys` filesystem.
- **Line 17:** GDB is started using `vmlinux` as the executable and `/proc/kcore` as the core file.
- **Lines 18-28:** GDB displays its startup banner and initial output.
- **Lines 28-36:** Debugging information for the `exvlan.ko` module is loaded and addresses are initialized. Note that addresses for `.text`, `.bss`, `.data`, and `.rodata` were obtained before starting GDB in lines 3-16).
- **Lines 37-54:** Various kernel module variables are displayed.
- **Line 55:** Finally, we exit the GDB debugger.

```
/scratch/intern # tftp -b 1400 -g -r vmlinux 10.5.2.158
/scratch/intern # tftp -g -r exvlan.ko 10.5.2.158
/scratch/intern # grep "" /sys/module/exvlan/sections/.*
/sys/module/exvlan/sections/.bss:0xc3601d00
/sys/module/exvlan/sections/.data:0xc3601780
/sys/module/exvlan/sections/.exit.text:0xc35f1070
/sys/module/exvlan/sections/.gnu.linkonce.this_module:0xc3601b80
/sys/module/exvlan/sections/.init.text:0xc3625000
/sys/module/exvlan/sections/.note.gnu.build-id:0xc35f176c
/sys/module/exvlan/sections/.reginfo:0xc35f1754
/sys/module/exvlan/sections/.rodata:0xc35f1790
/sys/module/exvlan/sections/.rodata.cst4:0xc35fa47c
/sys/module/exvlan/sections/.rodata.str1.4:0xc35f2950
/sys/module/exvlan/sections/.strtab:0xc35fee74
/sys/module/exvlan/sections/.symtab:0xc35fa484
/sys/module/exvlan/sections/.text:0xc35c8000
/scratch/intern # gdb /scratch/intern/vmlinux /proc/kcore
GNU gdb 6.8
Copyright (C) 2008 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copying"
and "show warranty" for details.
This GDB was configured as "mips-extr473-linux-gnu"...
Core was generated by ‘root=/dev/ram0 ro init=/linuxrc quiet dhash_max=5 ihash_max=5 mhash_max=4 bhash_’.
[New process 0]
#0  0x00000000 in ?? ()

add-symbol-file exvlan.ko 0xc35c8000 -s .bss 0xc3601d00 -s .data 0xc3601780 -s .rodata 0xc35f1790
add symbol table from file "exvlan.ko" at
        .text_addr = 0xc35c8000
        .bss_addr = 0xc3601d00
        .data_addr = 0xc3601780
        .rodata_addr = 0xc35f1790
(y or n) y
Reading symbols from /scratch/intern/exvlan.ko...done.
(gdb) p Fdb
$1 = {hdr = {{next = 0xc36021b4, prev = 0xc36021b4}, {next = 0xc36021bc,
      prev = 0xc36021bc}, {next = 0xc36021c4, prev = 0xc36021c4}, {
      next = 0xc36021cc, prev = 0xc36021cc}, {next = 0xc36021d4,
      ...
      prev = 0xc36027d4}, {next = 0xc36027dc, prev = 0xc36027dc}, {
      next = 0xc36027e4, prev = 0xc36027e4}, {next = 0xc36027ec,
      prev = 0xc36027ec}...}, entCnt = 0, maxEntCnt = 524288,
  entStaticCnt = 0, lock = {raw_lock = {lock = 0}}, kmem_cachep = 0x8ea55920}
(gdb) p GvlanIf
$2 = {vlan_name_tree = 0x87f45400, vlan_instance_tree = 0x8eba9c10,
  vlanInstance = 0, vlanIfTotalCnt = 3, lock = {raw_lock = {lock = 0}}}
(gdb) p/x Gpif
$3 = {pif_tree = 0x8c86c5c0, pifTotalCnt = 0x34, pifInstance = 0x34,
  hwType = 0x0, dot1q = 0x8100, vManEthertype = 0x88a8,
  vManBackCompatible = 0x1, dot1ahEthertype = 0x88b5, pSize = 0x2400, gMir = {
    gMirEnable = 0x0, gMirToSlot = 0x0, gMirToPort = 0x0,
    gMirSflowEnable = 0x0, gMirMode = 0x1}, vManEthertypeSec = 0x0}
(gdb) quit
```

Figure 8.2: Example Kernel Module Debugging Session with GDB

8.2.1 Finding Kernel Module Load Addresses

For example, assume that you have an instruction address (e.g. “0xC0BEC12C") from a particular kernel module (e.g. “bcmpkt.ko"). In order to use the addr2line utility to match this instruction address to a source file and line number, the address at which the module’s text segment was loaded must be determined. Conveniently, the kernel maintains section load addresses in the `/sys` pseudo-filesystem.

Again, using bcmpkt.ko as an example, we can obtain the load address for the text section of this module via “`cat /sys/module/bcmpkt/sections/.text`”. If the result is e.g. 0xC0BE7060, we can compute the offset 0x50CC = 0xC0BEC12C - 0xC0BE7060 and execute:

mips-extr473-linux-gnu-addr2line -e bcmpkt.ko 0x50CC

to find that the original address corresponds to:

/home/auser/exos/code/kernel_modules/pib/bcmpkt/src/aspen_rxtx.c:789

8.3 Magic SysRq Facility

The “Magic SysRq" facility is useful in cases where an EXOS system has, for some reason, become unresponsive to console input but is still able to process interrupts from the console UART.

The Magic SysRq facility is enabled by default if PACMAN is enabled. It can also be enabled at any time via:

echo 1 > /proc/sys/kernel/sysrq

The Magic SysRq facility can only be invoked via the system serial console (not via telnet sessions), and is invoked via the key sequence Ctrl-V Ctrl-X <key>, where “key" is one of the following:

0-9	Set console log level (control printk() output to console).
b	Reboot
e	Terminate all tasks except init by sending SIGTERM
f	Invoke out-of-memory handling (including saving OOM info to NVRAM)
i	Kill all processes by sending SIGKILL
l	Show current process registers for all CPUs
m	Show memory allocation stats
n	Nice all real-time processes
p	Show current process registers
q	Show all kernel timers
s	Synchronize filesystems to disk
t	Show all tasks
u	Unmount all filesystems
w	Show all blocked tasks

8.4 Kernel Logging and printk()

Kernel logged messages, which are written into the kernel log buffer using the **printk()** function, can be quite useful when debugging kernel issues. The **printk()** function mostly behaves the same as the standard C **printf()** function, although there are some important differences. Important things to know abot **printk()** include:

- **printk()** can be called from almost any context in the kernel, including interrupt handler context and process contexts.
- **printk()** can be called while holding spinlocks (although this should be avoided.)
- **printk()** output is atomic, so it is safe to have **printk()** calls executing on multiple processors in an SMP system simultaneously.
- **printk()** output is always written to the kernel log buffer, and may be written to the serial console if the log level of the message is below the configured console loglevel threshold.
- The kernel log buffer is read by the user-space EMS server process which is responsible for logging the message via EMS (note that whether a message appears in the EMS log will depend on the EMS configuration and the loglevel of the message.)

Each kernel log message has an associated loglevel, which is either specified explicity for each **printk()** call or is equal to the global default loglevel configured by `/proc/sys/kernel/printk`. Defined loglevels and their meanings are as follows:

Macro	Value	Explanation
KERN_EMERG	<0>	Emergency condition
KERN_ALERT	<1>	Alert: immediate attention required
KERN_CRIT	<2>	Critical condition
KERN_ERR	<3>	Serious/unexpected error condition
KERN_WARNING	<4>	Warning
KERN_NOTICE	<5>	A significant but normal condition has occurred
KERN_INFO	<6>	Informational
KERN_DEBUG	<7>	Developer debug message

The default console loglevel thrshold for EXOS is "2", which causes KERN_EMERG, KERN_ALRT, and KERN_CRIT messages to be written to the console. Raising the console loglevel threshold is often needed for debugging, but not that if the volume of log messages to the console is high, system instability can result (e.g. watchdog or EPM heartbeat failures may occur.) There are a number of ways in which the console loglevel threshold can be changed to a new value <*N*>:

- Via Ctrl-V Ctrl-X *N*, if Magic SysRq is enabled (e.g. when PACMAN is enabled.)
- By "**echo** *N* > **/proc/sys/kernel/printk**".
- By "**jerry printk enable**" to set the threshold to "7".

See section D.23 on page 144 for a description of the `/proc/sys/kernel/printk` special file.

8.5 Obtaining Scheduler Traces

In some circumstances (e.g. determining the cause of soft watchdog expiration messages, determining why real-time timing requirements are not being met), a scheduler trace can be very helpful in determining a root cause.

Obtaining a scheduler trace is straightforward, as illustrated by the following steps:

1. `echo sched_switch > /debug/tracing/current_tracer`
2. `echo print-parent > /debug/tracing/iter_ctrl`
3. `echo 1 > /debug/tracing/tracing_enabled`
4. (wait a few seconds...)
5. `echo 0 > /debug/tracing/tracing_enabled`

6. `cat /debug/tracing/trace > /scratch/intern/trace.txt`

At the end of these steps, the output file `trace.txt` will be visible via the EXOS `ls internal-memory` command, and can be uploaded via TFTP for analysis. A sample of scheduler trace output is shown in Figure 8.3.

```
# tracer: sched_switch
#
#           TASK-PID    CPU#    TIMESTAMP  FUNCTION
#              | |       |          |         |
            echo-1436  [000]    73.296695:   1436:120:R   + [000]  1435:120:S
            echo-1436  [000]    73.296700:   1436:120:X ==> [000]  1435:120:R
          <idle>-0     [001]    73.296747:      0:140:R ==> [001]   778:120:R
             epm-778   [001]    73.296765:    778:120:R   + [000]  1128:120:S
             epm-778   [001]    73.296787:    778:120:S ==> [001]     0:140:R
          <idle>-0     [001]    73.296789:      0:140:R   + [001]     7:115:S
          <idle>-0     [001]    73.296790:      0:140:R ==> [001]     7:115:R
     ksoftirqd/1-7     [001]    73.296791:      7:115:S ==> [001]     0:140:R
              sh-1435  [000]    73.296939:   1435:120:S   + [000]  1435:120:S
              sh-1435  [000]    73.296943:   1435:120:S ==> [000]  1128:120:R
            msdp-1128  [000]    73.296945:   1128:120:S ==> [000]     0:140:R
          <idle>-0     [000]    73.296947:      0:140:R   + [000]     4:115:S
          <idle>-0     [000]    73.296948:      0:140:R ==> [000]     4:115:R
     ksoftirqd/0-4     [000]    73.296949:      4:115:S ==> [000]     0:140:R
          <idle>-0     [000]    73.296949:      0:140:R   + [000]     4:115:S
          <idle>-0     [000]    73.296950:      0:140:R ==> [000]     4:115:R
     ksoftirqd/0-4     [000]    73.296952:      4:115:S ==> [000]     0:140:R
          <idle>-0     [000]    73.296952:      0:140:R   + [000]     4:115:S
          <idle>-0     [000]    73.296953:      0:140:R ==> [000]     4:115:R
     ksoftirqd/0-4     [000]    73.296953:      4:115:S ==> [000]     0:140:R
          <idle>-0     [001]    73.297751:      0:140:R   + [001]     7:115:S
          <idle>-0     [001]    73.297754:      0:140:R ==> [001]     7:115:R
     ksoftirqd/1-7     [001]    73.297756:      7:115:S ==> [001]     0:140:R
          <idle>-0     [001]    73.301418:      0:140:R   + [001]  1175:120:S
          <idle>-0     [001]    73.301439:      0:140:R ==> [001]  1175:120:R
             fdb-1175  [001]    73.301449:   1175:120:S ==> [001]     0:140:R
```

Figure 8.3: Example Scheduler Trace

8.6 /proc File System

The /proc file system contains a number of files useful for debugging kernel issues. A subset of these is described in Appendix D on page 135.

8.7 /debug File System

The /debug file system contains a number of files useful for debugging kernel issues. A subset of these is described in Appendix E on page 147.

Chapter 9

Performance Analysis Tools

9.1 Overview

9.2 Eprofile

9.2.1 Overview

Eprofile is a statistical profiling tool that is capable of profiling the execution of a number of types of EXOS software including user applications, shared libraries, the Linux kernel, as well as Linux kernel modules. As of this writing, `eprofile` has been used to improve performance in the following areas:

- Boot time with 4K VLAN configuration
- IPML performance
- exosq performance
- Aspen EAPS performance

Eprofile is supported for all curent MIPS-and x86-based EXOS platforms, including I/O modules.

9.2.2 Eprofile Quick Start

The following "recipes" outline the essential steps for profiling software executing on an MSM or I/O module. These steps are described in more detail in subsequent sections.

Quick Start for MSM Profiling

The basic process of using `eprofile` to profile code executing on an MSM is as follows:

1. Start performance test case.
2. From the CLI, execute " `debug performance start profiling buffer-size 1024` "
3. (Wait for test case to complete...)
4. From the CLI, execute "`debug performance stop profiling`"
5. From the CLI, execute "`debug performance upload profile <ip-address>`"
6. On build server, execute "`exos/tools/perftools/eprofile/eprofmt -profile /tftpboot/eprofile > report.txt`".
7. Inspect "report.txt" for performance bottlenecks, update code, rinse/lather/repeat...

Quick Start for I/O Module Profiling

When using profiling on an I/O module, the steps are only slightly different:

1. Start performance test case.
2. From I/O module shell, execute "`echo buffer=512 > /proc/eprofile`"
3. Again from the I/O module shell, execute "`echo start > /proc/eprofile`"
4. (Wait ~8 minutes = 512KB/((12 bytes per entry)*(100 entries/second)))
5. From I/O module shell, execute "`echo stop > /proc/eprofile`"
6. From MSM linux shell, execute "`mount -oremount,rw /exos` "to make /exos/tftpboot writable. (Note: this is not required when /exos is mounted via NFS.)
7. From MSM linux shell, execute "`touch /exos/tftpboot/mio-eprofile /exos/tftpboot/mio-eprofile.procs` "
   (This is necessary because the MSM tftp server is not configured to create new files.)
8. From MSM linux shell, execute "`cp /exos/bin/rdeprofile /exos/tftpboot`"
9. From EXOS CLI on MSM, execute "`jerry config tftp read_write`" to enable tftp puts to the EXOS tftp server.
10. From shell on I/O module, execute "`tftp -g -rrdeprofile -l/tmp/rdeprofile $MASTER_MSM_IP`"
11. From shell on I/O module, execute "`chmod +x /tmp/rdeprofile`"
12. From shell on I/O module, execute "`/tmp/rdeprofile`"
13. From MSM shell, execute "`tftp -p -r eprofile -l/exos/tftpboot/eprofile` $[tftp\text{-}ip]$" and "`tftp -p -r eprofile.procs -l/exos/tftpboot/eprofile.procs` $[tftp\text{-}ip]$" .
14. Execute "`exos/tools/perftools/eprofile/eprofmt -profile /tftpboot/eprofile > report.txt`".
15. Inspect "report.txt" for performance bottlenecks, update code, rinse/lather/repeat...

9.2.3 Preparing for Profiling

Profiling Test Cases

For profiling to be effective, a minimum of several minutes of profiling samples should be collected[1], so any tests designed to drive a particular code path for profiling should run for at least this duration (10-15 minutes would be even better). Some examples of good test drivers for profiling include:

- Using an Ixia to generate slow-path IP traffic for an arbitrary duration.
- Using a configuration with a large number of VLANs to profile boot-time configuration processing (boot time with large configurations ranges from 4 to 8 minutes).
- Using a "test harness" to drive a specific API (e.g. IPML, EMS, table management, etc.).

If possible, the test cases used for profiling should also provide metrics that can be used to quantify any performance improvements.

[1] This isn't entirely true, as little as 10-15 seconds of profiling data can be useful in some cases.

Sandbox

No special options or other changes are needed when building an image for profiling. Because symbolic and line number information are required in order to analyze the collected profile data, the sandbox in which the profiled code was built must be unchanged when `eprofmt` is used to generate the profiling report.

The profile analysis tool `eprofmt` can be built by executing "`make`" in the directory:

`exos/tools/perftools/eprofile`.

9.2.4 Enabling Profiling

The thee EXOS CLI command to enable profiling is:

debug performance start profiling { **buffer-size** <*size*> }

Where the optional **buffer-size** parameter is in units of kilobytes. The default buffer size is 1024 (1 MB)

Once started, profiling will stop when the buffer is full or when manually stopped using the CLI command:

debug performance stop profiling

When stopped, profiling can be restarted (clearing existing profiling data and optionally resizing the profiling buffer) using the CLI command:

debug performance restart profiling { **buffer-size** <*size*> }

There are two other ways in which profiling can be enabled. The first method, which is useful when profiling EXOS initialization (including configuration loading) is to simply execute "`echo buffer=`*buffer_size* `> /proc/eprofile`" at a linux shell prompt, where *buffer_size* specifies the desired profile buffer size in KB, and then "`echo start > /proc/eprofile`".

The second method by which profiling can be enabled is to use PMON to specify the kernel boot parameter "`eprofile=`*buffer_size*". This is useful when profiling of kernel initialization is needed.

Once profiling is enabled, profiling data is collected by the kernel within the context of the system timer interrupt handler that runs every 10 milliseconds. On each timer interrupt, the following data is collected and stored in the kernel profiling buffer:

- Program Counter
- Process ID
- CPU ID

The current status of profiling buffer, including the buffer size, current utilization, and profiling state (running/stopped) can be displayed via the CLI command:

debug performance show profiling

9.2.5 Transferring Profile Data

Profiling data consists of two files. The first, named "eprofile" by default, is a binary file with the profiling buffer contents. The second, with the default name "eprofile.procs", is a text file containing address map information for each process and symbol information for kernel modules.

Once profiling is complete, the two profile data files can be transferred to a TFTP server by invoking the CLI:

debug performance upload profile <*ipaddress*> { <*prefix*> }

Where <*ipaddress*> identifies a TFTP server reachable via the management VR and the optional <*prefix*> is prepended to the default file names "eprofile" and "eprofile.procs".

Profiling data can also be transferred using the `rdeprofile` script located in /exos/bin. The first parameter of the `rdeprofile` script is the IP address of the destination TFTP server. The second parameter is optional, and is a string to be used as a prefix for the profile data file name (default: "eprofile") and the system information file name (default: "eprofile.procs".

The `rdeprofile` script is located in /extr/bin for I/O modules, and no parameters are required (the $MASTER_MSM_IP environment variable is used as the tftp server IP address).

9.2.6 Cleanup

The profiling buffer can be freed (allowing the memory to be used for other purposes) by invoking the CLI:

debug performance terminate profling

9.2.7 Analysis of Profile Data

Once profile data has been transferred to a workstation, a report can be generated by using the `eprofmt` tool in the EXOS tools/perftools/eprofile directory.

The basic invocation syntax for `eprofmt` is:

`eprofmt -profile` *eprofile* > *output.txt*

where the parameters are used as follows:

- `-profile` is used to specify the profiling information file.

When `eprofmt` completes execution, the resulting report contains the following information:

- A list of processes sorted by CPU utilization
- A list of functions sorted by CPU utilization
- For each function, the profile count, source file name and line number for each sampled address within the function.

Other options that can be used with `eprofmt` include:

- `-nolines` Don't output source file name/line number information (makes eprofmt much faster).
- `-pid` *pid* Only count samples for the specified process ID.
- `-verbose` Output all profile sample data and some internal tables in addition to the normal output.
- `-range` <start>:<end> Only process profiling samples in the specified range.

9.3 Interpreting Profiling Data

9.3.1 eprofmt Output Format

The output of the `eprofmt` utility, as shown in Figure 9.1 on the facing page, consists of three main sections:

1. A synopsis of CPU utilization for each CPU at 10-second intervals.
2. A sorted list of all processes (user-space and kernel) with total CPU utilization as well as user-space and kernel-space CPU utilizations.
3. A sorted list of functions executed while profiling with total CPU utilization for each function as well as sample counts for instruction offsets within the function. If available, source file name and line number are provided for each instruction offset listed.

Lines 2-3: Build directory information

These lines identify the EXOS build directory and rootdisk manifest file that were used to obtain symbolic information for formatting the profile data.

```

EXOS directory: </data/lrichardson/fixes/cougar/cougar/exos/>
Manifest: </data/lrichardson/fixes/cougar/cougar/manifest>
01000->00:00-00:09: Cpu util:   7.2%
02000->00:10-00:19: Cpu util:  53.3%
03000->00:20-00:29: Cpu util: 100.0%
04000->00:30-00:39: Cpu util: 100.0%
05000->00:40-00:49: Cpu util: 100.0%

Per-process execution statistics
Rank  PID   Process Name         %Total %User  %Kernel
----  ----- -------------------- ------ ------ -------
   0)  1462 vlan                 19.63% 13.63%  6.00%
   1)  1668 cliMaster            14.65% 13.34%  1.32%
   2)  1441 hal                   5.00%  3.84%  1.17%
   3)  1468 fdb                   3.69%  2.88%  0.81%
   4)  1445 cliMaster             3.61%  2.41%  1.20%
   5)  1452 snmpSubagent          2.51%  2.06%  0.46%
   6)  1506 mcmgr                 2.24%  2.04%  0.20%

Global execution statistics
Rank  Function Name                  Object File               Count  Global %        User/Kernel %
----  ---------------------------- ---------------------- --------  --------------- ---------------
   0) rcu_pending                    vmlinux                    1204  5.51% (total)  14.17% (kernel)
        rcu_pending+0x18:                     187
        rcu_pending+0x20:                     180
        rcu_pending+0xb8:                     163
   1) __GI___strcasecmp              libc-2.9.so                 961  4.40% (total)   7.20% (user)
        __GI___strcasecmp+0x10:               491 glibc-2.9/string/strcasecmp.c:56
        __GI___strcasecmp+0x60:                29 glibc-2.9/string/strcasecmp.c:66
        __GI___strcasecmp+0x68:                22 glibc-2.9/string/strcasecmp.c:70
   2) finish_task_switch             vmlinux                     923  4.23% (total)  10.87% (kernel)
        finish_task_switch+0x34:               10
        finish_task_switch+0x38:              905
        finish_task_switch+0x3c:                1
   3) __copy_user                    vmlinux                     541  2.48% (total)   6.37% (kernel)
        __copy_user+0x20:                       4
        __copy_user+0x2c:                     147
        __copy_user+0x60:                      48
        __copy_user+0x6c:                     170
   4) exosq_msg_add                  exosq.ko                    531  2.43% (total)   6.25% (kernel)
        exosq_msg_add+0x0:                      1 cougar/code/kernel_modules/exosq/src/qbuf.c:40
        exosq_msg_add+0x17c:                  454 cougar/code/kernel_modules/exosq/src/qbuf.c:71
        exosq_msg_add+0x1d0:                    3 cougar/code/kernel_modules/exosq/src/qbuf.c:32
   5) pthread_mutex_lock             libpthread-2.9.so           405  1.85% (total)   3.03% (user)
        pthread_mutex_lock+0x0:                22 glibc-2.9/nptl/pthread_mutex_lock.c:43
        pthread_mutex_lock+0x8:                 4 glibc-2.9/nptl/pthread_mutex_lock.c:43
        pthread_mutex_lock+0x10:               10 glibc-2.9/nptl/pthread_mutex_lock.c:43
        pthread_mutex_lock+0x18:                2 glibc-2.9/nptl/pthread_mutex_lock.c:43
        pthread_mutex_lock+0x44:              166 glibc-2.9/nptl/pthread_mutex_lock.c:50
   6) sym????                        file????                    367  1.68% (total)   2.75% (user)
        sym????+0x0:                          367

```

Figure 9.1: Example Profiling Output from eprofmt

Lines 4-8: Per-CPU utilization at 10-second intervals

A line is displayed showing the starting profile sample number (e.g. for use with the `-range` option), the elapsed time, and overall CPU utilization for each CPU (note that this example if for a uniprocessor system).

Lines 10-19: Per-process execution statistics

A line is shown for each process (or individual thread, if multi-threaded) with overall percentage CPU utilization and a breakdown of the overall utilization into user-space and kernel-space components.

Lines 22-50: Per-Function Statistics

These lines show each function (by name) that was sampled, along with the following information:

- **Object File** - The object file (executable, shared library, kernel module, or kernel) associated with the function.
- **Count** - The number of profiling samples taken while this function was executing.
- **Global Utilization** - The overall CPU utilization accounted for by this function.
- **User/Kernel Utilization** - The percentage CPU utilization for this function within its execution domain (user-space or kernel-space)

In addition, a line is displayed for each sampled instruction location within the function showing the number of profiling samples for that location and (if available) the source file and line number corresponding to the instruction location.

Lines 51-52: Unresolved Profiling Samples

The function name "`sym????`" is used for all profiling samples that were not able to be correlated to an executable name. In general this will be true for a process that exits before the profiling data is collected by the `rdeprofile` script, which might include shell commands issued at the CLI or spawned by calls to the `system()` function.

Chapter 10

PACMAN

10.1 Overview

PACMAN is a utility that provides access to a variety of options to facilitate the development and debugging of an EXOS system, including:

- File system integrity checking.
- EXOS boot/initialization debugging.
- Developer management Ethernet IP configuration.
- Source location of /var mount.
- Whether syslog output is written to /var.
- Whether EXOS is started at boot time.
- Source location of /exos mount.
- Whether EXOS is restarted after failure.
- How NVRAM kernel/core information is dealt with at boot time.
- Developer debug-mode.
- Access to Linux command shell prior to EXOS startup via command.

10.2 Enabling PACMAN

As described in the following sections, there are several ways to enable or disable PACMAN.

10.2.1 Enabling PACMAN from the PMON Command Prompt

Assuming PMON debug-mode is enabled, PACMAN can be enabled via:

cmdline “pacman=1"

or disabled via:

cmdline “ "

from the PMON command prompt.

10.2.2 Enabling PACMAN from the EXOS CLI

Assuming EXOS debug-mode is enabled, PACMAN can be enabled via:

jerry enable pacman

or disabled via:

jerry disable pacman

from the EXOS CLI prompt.

10.2.3 Enabling PACMAN from the Linux Shell

PACMAN can be enabled via:

echo "pacman=1" > /proc/extr/nvram/cmdline

or disabled via:

echo " " > /proc/extr/nvram/cmdline

from a shell prompt.

Once enabled, the PACMAN configuration menu will be presented at boot time as show in figure 10.1 on the next page.

10.3 PACMAN Options

PACMAN options are configured via a text menu that is presented at boot time (after base linux/filesystem initialization, but before EXOS is started). The configuration is stored in the files /boot/system.cfg and /alt/boot/system.cfg for primary and secondary image partitions, with a separate PACMAN configuration maintained for each of the primary and secondary images.

10.3.1 ! - Invoke Comand Shell

This option is not presented by the PACMAN command menu, but is very useful nonetheless. Typing "!" at the PACMAN command menu prompt causes a new shell to be started. When finished, the shell can be exited via "exit" or "Ctrl-D" (as usual).

Note that the "!" must appear alone - the commonly used "!<shell-command>" syntax is not supported.

10.3.2 enable perf hooks

Valid options are **true** and **false**. This option currently performs no function, and is maintained for compatibility with earlier releases.

10.3.3 boot_cf_alt_fsck

Valid options are **true** and **false**. If **true**, the /alt/boot filesystem is checked via fsck before EXOS is started.

10.3.4 boot_cf_config_fsck

Valid options are **true** and **false**. If **true**, the /config filesystem is checked via fsck before EXOS is started. Note that, as of branch-shasta10_v1120b7, this setting is a no-op due to an apparent typo in the linuxrc script.

```
==========================
===== developer menu =====
==========================
1) enable perf hooks..........."false"
2) boot_cf_alt_fsck............"true"
3) boot_cf_config_fsck........."true"
4) boot_debug................."false"

management port settings
 5a) boot_ipaddr msm a........"10.66.4.10"
 5b) boot_ipaddr msm b........"10.66.4.29"
 5c) boot_netmask............."255.255.255.0"
 5d) boot_broadcast..........."10.66.4.255"
 5e) boot_gateway............."10.66.4.1"

var partition settings
 6a) var partition............"tmp" [tmp, nfs or extcf]
 6b) nfs var mount............""

exos settings
 7a) start exos at boot......."true"
 7b) enable heap tracing......"false"
 7c) allow /var log file......"false"
 7d) node failure restart....."true"
 7e) mount exos from.........."local" [local, nfs]
 7f) nfs exos mount..........""
 7g) nvram core dump info....."show_clear"
 7h) debug mode..............."false"
 7i) environment vars ........""

pacman settings
 8a) pacman autostart........."false"
 8b) pacman autostart delay...5

l) load settings from system.cfg on hda2
o) back to original settings
s) save settings to system.cfg on hda2

copy)  show settings for COPY
paste) read settings from PASTE

r) reboot

c) continue with boot process
~>
```

Figure 10.1: Example PACMAN Menu

10.3.5 boot_debug

Valid options are **true** and **false**. If **true**, “set -x" is executed to trace progress of the /linuxrc and /exos/exos initialization scripts.

10.3.6 boot_ipaddr msm a

This option configures the management IP address for MSM A. A valid, unique IP address must be configured in order to mount /exos or /var via NFS. If IP addresses are configured for MSM A and MSM B, both addresses must reside on the same subnet.

10.3.7 boot_ipaddr msm b

This option configures the management IP address for MSM B. A valid, unique IP address must be configured in order to mount /exos or /var via NFS. If IP addresses are configured for MSM A and MSM B, both addresses must reside on the same subnet.

10.3.8 boot_netmask

This option configures the netmask to be used with the MSM A and MSM B management IP addresses.

10.3.9 boot_broadcast

This option configures the management IP subnet broadcast address to be used for MSM A and MSM B. The default address is computed from the IP address and netmask, but can be overridden if needed.

10.3.10 boot_gateway

This option configures the default management gateway to be used for MSM A and MSM B. The default address is computed from the IP address and netmask, but can be overridden if needed.

10.3.11 var partition

This option configures the mount point for the /var file system. Valid options are **tmp** (default), **nfs**, and **extcf**.

10.3.12 nfs var mount

When **nfs** is configured for the var partition, this option configures the remote NFS file system to be mounted as /var. The format of the NFS file system specifier is:

<*ip-address*>:/<*nfs-exported-path*>

Note that <*nfs-exported-path*>/<*slot-name*> must exist, where <*slot-name*>:

- Is either **msm_a** or **msm_b** for chassis-based switches.
- Is always **msm_a** for Summit switches that are do not have stacking enabled.
- Is e.g. **slot_1**, **slot_2**, etc. for Summit switches with stacking enabled.

10.3.13 start exos at boot

This option configures whether exos is started automatically at boot time. Valid options are **true** (default) and **false**.

10.3.14 enable heap tracing

This option can be used to enable EXOS heap tracing (exmmon). Valid options are **true** and **false** (default).

10.3.15 allow /var log file

This option configures whether log files are to be stored in /var. Valid options are **true** and **false** (default).

10.3.16 node failure restart

This option configures whether EXOS should be restarted after an unrecoverable error (this is useful for debugging these errors). Valid options are **true** (default) and **false**.

10.3.17 mount exos from

This option configures the file system to be mounted for /exos. Valid options are **local** (default) and **nfs**. Mounting /exos via NFS is a tremendous time saver for development since it allows the time-consuming download/install process to be skipped (Note: see section 12.2.2 on page 91 for information on upgrading the kernel without having to download/install a new image).

10.3.18 nfs exos mount

This option configures the location of the NFS file system to be mounted as /exos when EXOS is configured to be mounted via NFS. The format of the NFS file system specifier is:

<ip-address>:/<nfs-exported-path>

For example:

10.5.2.224:/sw-pool/auser/v1220b28/cougar/cougar/exos

10.3.19 nvram core dump info

This option configures the action to be taken at initialization time on kernel oops and process crash information in NVRAM. Valid options include:

- **show_clear** - Show NVRAM crash information at boot time, then clear it.
- **show_keep** - Show NVRAM crash information at boot time without clearing.
- **quiet_keep** - Do not show or clear NVRAM crash information.
- **quiet_clear** - Clear NVRAM crash information at boot time without showing.
- **prompt** - Prompt user for desired action.

10.3.20 debug mode

This option causes EXOS “debug-mode" to be entered automatically after login via the serial console (setting this to “true" is highly recommended for development as it saves time spent finding/remembering/typing the EXOS debug-mode password).

10.3.21 environment vars

This option allow arbitrary environment variables to be exported from the linuxrc script. For example, setting this option to:

MY_VAR_1="FOO" ANOTHER_VAR="BAR" YET_ANOTHER=28

Will export the three name/value pairs as environment variables from the shell executing the /linuxrc initialization script.

10.3.22 pacman autostart

This option causes PACMAN to start EXOS automatically after a configurable delay instead of waiting for the user to issue the "**c**" command (see next item.)

10.3.23 pacman autostart delay

If PACMAN autostart is enabled, this specifies the delay (in seconds) PACMAN will wait before starting EXOS automatically.

10.3.24 load settings from system.cfg on hda<*N*>

This command causes settings to be reloaded from the PACMAN configuration file /boot/system.cfg which is located on either the **hda1** or the **hda2** partition depending upon whether the primary or secondary partition was booted.

10.3.25 back to original settings

This command causes all PACMAN configuration settings to be reset to their default values.

10.3.26 save settings to system.cfg on hda<*N*>

This command causes PACMAN configuration settings to be saved to /boot/system.cfg, which is located either the **hda1** or the **hda2** partition depending upon whether the primary or secondary partition was booted.

10.3.27 show settings for COPY

This command displays current PACMAN settings in a form acceptable for the **read settings from PASTE** command (see next).

10.3.28 read settings from PASTE

This command causes PACMAN settings to be read from the console, which allows all PACMAN settings to be configured by pasting them to a terminal emulator window.

10.3.29 reboot

This command, unsurprisingly, causes the switch to reboot.

10.3.30 continue with boot process

This command causes the PACMAN menu to exit and EXOS initialization to proceed, unless configured otherwise.

10.4 Further Reading

The only other documentation for PACMAN appears to be the source itself. PACMAN is implemented in the exos /linuxrc shell script, which can be found in the EXOS source tree as:

`exos/fs/rootdisk/files_to_copy/linuxrc`

Chapter 11

PMON

11.1 Overview

PMON is the bootloader firmware that is currently used across all EXOS platforms, and was originally based on the open-source PMON2000 project. PMON provides a variety of services including hardware initialization, POST, software image selection, software image loading via CF and Ethernet, and other system management and initialization functions.

11.2 PMON Command Prompt

PMON provides a command line interface via the serial console. The PMON command line interface can be accessed by holding down the space bar while rebooting for BD8K and BD10K platforms. Summit and BD12K platforms require the space bar to be pressed at the second spacebar prompt.

11.3 PMON Commands

This section lists some of the more frequently used PMON commands that are available in the default (non-debug mode) command environment. Sources for more detailed command documentation are listed at the end of this chapter.

11.3.1 Boot Image Selection: `boot`

The syntax of the `boot` command is as follows:

boot { [**1** | **2** | **3** | **4**] }

The `boot` command with no parameters causes the currently selected image to be loaded and booted; this is useful for continuing the normal boot sequence after working with the PMON command prompt.

The `boot` command can also be used to boot an executable image different than the one that is currently configured. For example, the command **boot 1** causes the primary image to be loaded and executed while **boot 2** causes the secondary image to be loaded and executed.

The BD8K platform also includes the **boot 3** and **boot 4** options. These are used to boot the primary and secondary manufacturing diagnostics images.

11.3.2 System Reboot: `reboot`

The `reboot` command causes the system to reboot via system hard reset.

11.3.3 Configuration Selection: `config`

The syntax for the PMON `config` command is:

config [**default** | **alt** | **none** | **file** <*filename*>]

The `config` command can be used to override the currently selected configuration. The valid forms of the `config` command are as follows:

config default	Use the currently selected configuration.
config alt	Use the alternate configuration (i.e. use primary if secondary is currently selected and vice-versa).
config none	Use factory default configuration.
config file <*filename*>	Use the specified configuration file.

11.3.4 PMON Network Configuration: `configip`

The `configip` command is used to configure the management Ethernet interface for use by PMON. This command is most often used for network rescue/recovery (see section 11.5 on the next page). The syntax of the PMON `configip` command is as follows:

configip ipaddress <*ip-address*>[/<*net-mask*>] **gateway** <*gateway-address*>

11.3.5 PMON IP Ping: `ping`

When the management Ethernet interface has been configured via the **configip** command, the PMON **ping** command can be used to verify network connectivity. The syntax for the **ping** command is:

ping { **-fqrv** } { **-c** <*count*> } {**-s** <*size*> } <*ip-address*>

FIXME: PROVIDE DESCRIPTIONS FOR PMON PING OPTIONS.

11.4 Debug/Development Mode Commands

Debug mode is enabled in PMON through the `enable debug-mode` command. Upon entering this command, the user is prompted for the debug-mode password and, assuming the correct password is provided, a number of additional PMON commands are made available. The following sections describe some of the more generally useful PMON debug mode commands.

11.4.1 Display and Configure Boot Command Line: `cmdline`

The `cmdline` command is typically used to configure the Linux kernel boot "pacman" parameter, e.g. `cmdline pacman=1`. The current cmdline setting can be displayed by issuing the `cmdline` command with no argument.

Note that while one might expect this command to allow a command line with arbitrary kernel parameters to be configured, this is not the case - although an arbitrary command line can be specified, all kernel parameters other than "pacman=1" appear to be ignored when Linux is started. For information about specifying an arbitrary set of kernel parameters via the `pboot` command, see Section 11.4.2 on the facing page.

11.4.2 Low-level Boot Command: `pboot`

The `pboot` command is a low-level relative of the `boot` command, and is useful in cases where full control of loaded image and the command line parameters is required.

The syntax for the pboot command is:

pboot <*image-source*> { <*kernparm1*> <*kernparm2*> ... }

The *image-source* parameter has several different forms. For example:

```
pboot /dev/fat/wd0a/vmlinux init=linuxrc
```

boots the vmlinux image in the primary boot partition. The secondary image can be booted via:

```
pboot /dev/fat/wd0b/vmlinux init=linuxrc
```

Additionally, if a PMON IP address has been configured, the boot image can be downloaded via tftp:

```
pboot tftp://10.5.2.224/vmlinux init=linuxrc
```

11.4.3 Useful Kernel Initialization Parameters

Kernel initialization parameters that can be specified via the **pboot** command include:

eprofile=<**buffer_size**>	Initialize global profiling with specified profiling buffer size.
init=<**command**>	Specify initial process.
pacman=1	Enable PACMAN.
quiet	Suppress output of kernel initialization messages.

11.5 System Rescue/Recovery via TFTP

In the event that a system's EXOS file system becomes corrupted in some way, PMON provides a mechanism for system recovery. System recovery includes re-partitioning of the compact flash drive and installation of a new EXOS image. Note that all previous file system contents (including configuration files) are destroyed in this process.

The TFTP rescue process is straighforward, assuming a TFTP server is available with an appropriate EXOS image for the system being recovered:

1. Reboot the system and hold space bar to interrupt boot process.

2. At the PMON command prompt, do:

   configip ipaddress <*ip-address*>[/<*net-mask*>] **gateway** <*gateway-address*>

   to configure the management Ethernet interface.

3. At the PMON command prompt, do:

   download image <*tftp-server-address*> <*filename.xos*>

After PMON has downloaded and booted the image, the compact flash device will be repartitioned and the image will be installed automatically.

11.6 System Rescue/Recovery via External CF

In some cases (e.g. when a TFTP server is not available) it can be more convenient to perform recovery from external compact flash. Assuming that an appropriate image exists on a compact flash module in the external CF socket, the process for rescue/recovery is as follows:

1. Reboot the system and hold space bar to interrupt boot process.
2. At the PMON command prompt, do:

boot file <*filename.xos*>

PMON will then load and boot the image, the compact flash device will be repartitioned and the image will be installed. (Again, all files including configuration files from the original installation will be destroyed by this process).

11.7 Further Reading

The Livelink folder *Enterprise→Workspace→Corporate Library→Technical Library→Boot Code (PMON) Specs* contains a number of documents related to PMON.

The PMON2000 website http://www.pmon2000.com provides an online manual, knowledge base, and CVS access to the original source code base. Note that the Extreme version of PMON has diverged significantly from the original code base.

The file **linux/Documentation/kernel-parameters.txt** documents the list of standard kernel initialization parameters.

Chapter 12

Miscellaneous Tips and Techniques

12.1 Overview

12.2 Modifying Individual EXOS Files

12.2.1 Modifying Files in the /exos Partition

1. mount -o remount,rw /exos
2. Update file via e.g. tftp, vi, cp
3. sync
4. mount -o remount,ro /exos

12.2.2 Installing a new Kernel to the /boot Partition

This tip is useful for avoiding long download times to update the kernel when using PACMAN to mount /exos via NFS.

1. tftp -g -r vmlinux -l /tmp/vmlinux *tftp-server-address*
2. (verify that vmlinux was downloaded successfully)
3. mount -o remount,rw /boot
4. cp /tmp/vmlinux /boot
5. sync
6. mount -o remount,ro /boot

12.3 Core Dumps

12.3.1 Forcing a Core Dump

A process can be killed and a core dump created as follows:

1. pidof *process-name*
2. (select a process ID from the output of the *pidof* command.)
3. kill -6 *process-ID*

12.4 Watchdog Timers

12.4.1 Forcing a Watchdog Expiration (BD8800)

For testing, a watchdog expiration can be forced as follows:

1. echo 'testuCWdg 1' > /proc/sys/watchdog

12.4.2 Forcing a Watchdog Expiration (BD10K, x450)

For testing, a watchdog expiration can be forced as follows:

1. echo 'testSibyteWdg 3' > /proc/sys/watchdog

12.4.3 Disabling Watchdog Timers

For debugging, watchdog timers can be disabled as follows:

1. echo disableuCWdg > /proc/sys/watchdog
2. echo disableSibyteWdg > /proc/sys/watchdog
3. echo 'cleanUpTimer epm' > /proc/sys/watchdog

Chapter 13

Debugging Broadcom-Based Platforms

13.1 Overview

This chapter describes platform-specific debugging facilities for Broadcom-based platforms, including the BD8K and Summit families of EXOS-based switches.

13.2 Quick Start: Invoking the Broadcom Shell

1. First, enable debug-mode if it is not already enabled.
2. For BD8K I/O modules and for remote members of a SummitStack, telnet to the device to be debugged using the device's slot-based IP address, For example, the following command would be used to telnet to an I/O module in slot 5 or slot 5 of a Summit stack:

   !telnet -v 1 10.0.5.2

3. For local use (e.g. BD8K MSM or local Summit), start a Linux shell by executing **!sh** at the EXOS CLI prompt.
4. Start the Broadcom shell by executing either **/exos/bin/bcm.shell** for BD8K MSMs and Summit switches or **/extr/bin/bcm.shell** for BD8K I/O modules.
5. Press the `Enter` key once to display the BCM shell prompt `BCM.0>` .
6. When done, enter the `"quit"` command to exit the shell, then `"exit"` to leave the telnet/shell session.

13.3 Navigating the Broadcom Shell

13.3.1 Default Unit Selection

The Broadcom shell maintains a default unit number (in Broadcom terminology a "unit" refers to a specific chip) which is used for chip-specific commands. The current default unit number is displayed in the shell command prompt, e.g. the prompt `"BCM.5> "` shows that the current default unit number is 5.

Selecting a different default unit number can be accomplished by issuing:

<*unit-number*>**:**

Where <*unit-number*> is the desired new default unit number. For example, Entering `"1:"` will change the default unit number to “1”, assuming that is a valid unit number.

13.3.2 Getting Help

Online help is available for the Broadcom shell. The syntax of the **help** command is:

help { <*command*> }

If the optional <*command*> parameter is not specified, a list of all commands with a brief description will be displayed. If the <*command*> parameter is specified, detailed help for that command (if available) will be displayed.

Note that listed command names will often include a mix of upper- and lower-case letters. If this is the case, the command may be abbreviated by omitting the lower-case letters. For example, the **EditReg** command can be executed by entering either **editreg** or **er**.

13.3.3 Exiting the Shell

Exiting the Broadcom shell can be accomplished by executing the **quit** command.

13.4 Commonly Used Commands

13.4.1 Low-Level Register Reads

The **getreg** command provides the ability to perform low-level register reads. The syntax for this command is:

Getreg { [**hex**|**raw**|**chg**] } { <*regtype*> } <*regname*>

By default, the output is formatted to display all fields of the specified register. Optionally, the output mode can be specified to be **hex** (single hexadecimal value), **raw** (without field decoding), or **chg** (only non-default values are displayed).

The default register type <*regtype*> is **soc**. Other register types may optionally be specified, including **pcic** (PCI config space), **pcim** (PCI CMIC space), or **phy** (PHY register space).

The <*regname*> parameter can be a register name or a numeric offset. A list of register names and their corresponding addresses can be obtained by executing the `dump` <*regtype*> command.

13.4.2 Low-Level Register Writes

The **setreg** command provides the ability to perform low-level register writes to entire registers or fields within the register. The syntax for the first form of this command is:

Setreg { <*regtype*> } <*regname*> <*value*>

Where <*regtype*> and <*regname*> are as described for the **getreg** command in Section 13.4.1, and <*value*> is the value to be written to the entire register.

The syntax for the second form of the **setreg** command is:

Setreg { <*regtype*> } <*regname*> <*fieldname1*>**=**<*value1*> { **,**... }

Where <*regtype*> and <*regname*> are as described for the **getreg** command in Section 13.4.1, and <*value1*> is the value to be written to <*fieldname1*>. Multiple fields can be written in a single command by specifiying additional fieldname/value pairs.

The **modreg** command is similar in effect to the second form of the **setreg** command, with <*regtype*> fixed to be **soc**. The syntax of the **modreg** command is as follows:

Modreg <*regname*> <*fieldname1*>**=**<*value1*> { **,**... }

13.4.3 Dumping Table Entries

The **dump** command can be used to display multiple registers and table entries.

When dumping table memory entries, the syntax for the **dump** command is:

Dump { [**raw** | **hex** | **all** | **chg**] } <*table-name*> { <*start-index*> } { <*count*> }

The optional output format specifiers include:

- **raw** - Don't format individual fields, display numeric values only.
- **hex** - Display only numeric hexadecimal values for each entry.
- **all** - Display all entries (even those that are invalid).
- **chg** - Display only entries/fields with non-default values.

The <*table-name*> specifies which table to dump entries from. A list of valid table names can be obtained by executing the **listmem** command.

The optional <*start-index*> and <*count*> parameters specify the starting index (default: 0) and the number of entries to display (default: all).

Other forms of the **dump** command can be used to display information for other entities (PCI configuration space, CMIC, PHY registers, etc.). Refer to online help for details.

13.4.4 Port Counters

The **show counters** command is used to display physical port counters for the current SoC device. The syntax for this command is:

show Counters { **Changed** } { **Same** } { **Z** } { **NZ** } { **All** } { **ErDisc** } { **Hex** } { **Raw** } { <**reg**> } { **.** } { <**pbmp**> }

- **Changed** - Only show counters that have changed since the last display (default).
- **Same** - Only show counters that have not changed since the last display.
- **Z** - Only show counters that have zero value.
- **NZ** - Only show counters that have non-zero value (default).
- **Hex** - Display counters as 64-bit hex values.
- **Raw** - Do not display register names.
- **ErDisc** - Only show counters marked with Error/Discard.

Chapter 14

Code Coverage

14.1 Overview

The EXOS development environment includes tools and infrastructure for code execution coverage measurement. This allows EXOS applications and shared libraries to be instrumented at build time for coverage measurement that can then be executed in standard test environments, producing an easily interpreted report showing which portions of code were executed (and which weren't) that can be used to determine whether additional test cases need to be implemented to improve test coverage.

14.2 Quick Start: Using Code Coverage

The process for enabling and using code coverage is as follows:

1. In a sandbox, modify the Makefile(s) for the desired applications and libraries by adding the following line, and rebuild:

   ```
   WITH_CODE_COVERAGE := 1
   ```

2. Load the resulting image and execute desired test cases.
3. When testing is complete, reboot the switch to ensure that all coverage data has been written to the file system.
4. Transfer the coverage data files (which can be found via "**ls internal-memory ccov***" to the build server.
5. Create the HTML coverage report by executing "**tools/debug/cover ccov.*.tgz**".
6. View the report using your favorite web browser.

Appendix A

Platform Hardware Configurations

A.1 Stackable Hardware Configurations

Table A.1: Summit Hardware Configurations.

Module	Codename	CPU	Dataplane Chipset
X450-24T	Cougar	BCM1125	BCM5675 (Hercules1.5) BCM5695 (Draco1.5) with BCM5464 PHYs BCM5674 (Lynx) with Internal PHYs
X450-24X	Leopard	BCM1125	BCM5675 (Hercules1.5) BCM5695 (Draco1.5) with Internal PHYs BCM5674 (Lynx) with Internal PHYs
X150	Lynx	BCM1122	BCM56112 (Felix+) with BCM5248 PHYs
X250e	Panther	BCM1122	BCM56112 (Felix+) with BCM5248 PHYs
X350	LynxG	BCM1122	BCM56304 (Helix) with BCM5464 PHYs
X450e	Jaguar	BCM1122	BCM56304 (Helix) with BCM5464 and BCM8704 PHYs
X450a	Jaguar	BCM1122	BCM56504 (Firebolt) with BCM5464 and BCM8704 PHYs
X650	Tiger	BCM1240	BCM56720 (Humv+) BCM56821 (Scorpion) with TN1010 PHYs
X480	Sylvester	XLS208@1GHz 1 GB DRAM	BCM56626 (Triumph) with BCM5466SR and BCM8725 PHYs
X460	Ocelot	XLS204@600 MHz 512 MB DRAM	BCM56526 (Apollo) with BCM54680 and BCM5482SA2K PHYs
X670-48X	Wildcat	XLS408@1GHz 1 GB DRAM	BCM56843 (Trident)
X670V-48t	Wildcat	XLS408@1GHz 1 GB DRAM	BCM56846 (Trident+) with BCM84834 PHYs
X440	Cobra	XLS104@600MHz 512MB DRAM	BCM56142 (Hurricane) with BCM54682/BCM54685 PHYs

A.2 Chassis Hardware Configurations

Table A.2: Aspen V1 Module Configurations.

Module	Codename	CPU	Dataplane Chipset
MSM-G8X	Harlan	BCM1125	BCM5675 (Hercules1.5) BCM5695 (Draco1.5) with Internal PHYs
G48T	Arrowood	BCM5836	BCM5675 (Hercules1.5) BCM5695 (Draco1.5) with BCM5464 PHYs
G48P	Caymus	BCM5836	BCM5675 (Hercules1.5) BCM5695 (Draco1.5) with BCM5464 PHYs
10G4X	Opus	BCM5836	BCM5675 (Hercules1.5) BCM5674 (Lynx) with Internal PHYs
G24X	Kistler	BCM5836	BCM5675 (Hercules1.5) BCM5695 (Draco1.5) with Internal PHYs

Table A.3: Aspen V2 (Napa/Sonoma) Modules.

Module	Codename	CPU	Dataplane Chipset
MSM-48	Turley	BCM1125	BCM5675 (Hercules1.5)
G48Te	Lockwood	BCM5836	BCM56304 (Helix) with BCM5464 PHYs
G48Pe	Renwood	BCM5836	BCM56304 (Helix) with BCM5464 PHYs
G48Ta	Blackstone	BCM5836	BCM56504 (Firebolt) with BCM5464 PHYs
G48Xa	Beringer	BCM5836	BCM56504 (Firebolt) with BCM5464 PHYs
10G4Xa	Pahlmeyer	BCM5836	BCM5675 (Hercules1.5) BCM56501 (Firebolt) with BCM8705 PHYs
10G4Ca	Pahlmeyer	BCM5836	BCM5675 (Hercules1.5) BCM56501 (Firebolt) with Internal PHYs

Table A.4: Aspen V3 (Melange) Modules.

Module	Codename	CPU	Dataplane Chipset
MSM-48c	Tarragon	BCM1240	BCM5675 (Hercules1.5)
S-G8Xc	Ginger	BCM5836	BCM56512 (Firebolt2) with Internal PHYs
S-10G1Xc	Fennel	BCM5836	BCM56512 (Firebolt2) with BCM8705 PHYs
G48Te2	Coriander	BCM5836	BCM56314 (Helix+) with BCM5488 PHYs
G24Xc	Nutmeg-24	BCM5836	BCM56514 (Firebolt2) with BCM5488 PHYs
G48Xc	Nutmeg-48	BCM5836	BCM56514 (Firebolt2) with Internal PHYs
G48Tc	Cilantro	BCM5836	BCM56514 (Firebolt2) with BCM5488 PHYs
10G4Xc	Saffron-4	BCM5836	BCM56701 (Humv) BCM56511 (Firebolt2) with BCM8705 PHYs
10G8Xc	Saffron-8	BCM5836	BCM56701 (Humv) BCM56511 (Firebolt2) with BCM8705 PHYs

Table A.5: Aspen V4 (Manhattan) Modules.

Module	Codename	CPU	Dataplane Chipset
MSM-80	None	BCM1240	BCM56725 (Conqueror)
G48Tm	None	XLS204	BCM56725 (Conqueror) BCM56624 (Triumph) with BCM54980 PHYs
G48Xm	None	XLS204	BCM56725 (Conqueror) BCM56624 (Triumph) with Internal PHYs
10G8Xm	None	XLS204	BCM56725 (Conqueror) BCM56628 (Triumph) with BCM8725 PHYs

G96Tc	None	XLS204	BCM56725 (Conqueror) BCM56628 (Triumph) with BCM54980 PHYs
10G24Xc	None	XLS204	BCM56822 (Scorpion) with BCM8725 PHYs

Appendix B

MIPS Programming Architecture

B.1 Overview

All current EXOS platforms, with the exceptions of the i386 VM platform and the Everest (BDX8) MM use CPUs implementing some variant of the MIPS architecture. architectures.

This chapter provides a very cursory overview of the MIPS architecture as it applies to EXOS systems. Several excellent references are listed at the end of the chapter for more detailed information.

B.2 CPU Registers

The MIPS CPU register set includes 32 general-purpose registers (r0-r31), 32 general-purpose floating-point registers (f0-f31), 3 special-purpose registers (HI and LO, which hold the result of integer multiply and divide operations, and PC, the program counter register).

Register Number	Register Name	Usage
0	`zero`	Read-only, always contains 0.
1	`at`	Assembler temporary; used to implement macro pseudo-instructions.
2-3	`v0-v1`	Function return value.
4-7	`a0-a3`	Function arguments (first four 32-bit integer arguments).
8-15	`t0-t7`	Function temporaries, not preserved by callee.
24-25	`t8-t9`	Function temporaries, not preserved by callee.
16-23	`s0-s7`	Subroutine register variables, preserved by callee.
26-27	`k0-k1`	Reserved for kernel interrupt/exception handler use.
28	`gp`	Global pointer (depends upon ABI).
29	`sp`	Stack pointer.
30	`s8/fp`	Ninth subroutine register, or frame pointer (depends on compiler options).
31	`ra`	Function return address register.

Table B.1: MIPS General-Purpose CPU Register Usage (o32 ABI)

B.3 MIPS Coprocessors

In every MIPS implementation, there exists at least one coprocessor, which is coprocessor 0 (CP0). CP0 implements a set of registers to manage virtual to physical address translation, exceptions, user vs. kernel privilege states, instruction and data caches, and error reporting. Coprocessor 1 (CP1) is also present if the CPU implements a floating-point unit. floating-point unit.

B.4 MIPS CP0 Registers

B.4.1 CP0 Status Register

The CP0 Status register format varies somewhat between different MIPS architecture implementations. This is the format of the MIPS64 architecture Status register, which is used in the Broadcom SB1 core:

B.4.2 CP0 Cause Register

The CP0 Cause register format varies somewhat between different MIPS architecture implementations. This is the format of the MIPS64 architecture Status register, which is used in the Broadcom SB1 and NetLogic XLR/XLS cores:

Field Name	Bits	Description
BD	31	Set to 1 if last exception occurred in a branch-delay slot.
CE	29:28	Identifies coprocessor unit when coprocessor unusable exception is taken.
IV	23	Indicates whether an interrupt uses the general exception vector (0x180) or special interrupt vector (0x200).
WP	22	Indicates whether a watchpoint exception was deferred.
IP[7:2]	15:10	External interrupt pending status.
IP[1:0]	9:8	Software interrupt pending status.
ExcCode	6:2	Exception code - identifies reason for exception (see table B.3 on the next page).

Table B.2: MIPS CP0 Cause Register Fields

Name	Value (Hex)	Description
Int	0	Interrupt.
Mod	1	TLB modification exception.
TLBL	2	TLB load (or instruction fetch) exception.
TLBS	3	TLB store exception.
AdEL	4	Address error exception (load for instruction fetch).
AdES	5	Address error exception (store).
IBE	6	Bus error exception (instruction fetch).
DBE	7	Bus error exception (data load/store).
Sys	8	System call exception.
Bp	9	Breakpoint exception.
RI	0xA	Reserved instruction exception.
CpU	0xB	Coprocessor unusable exception.
Ov	0xC	Arithmetic overflow exception.

Tr	0xD	Trap exception.
FPE	0xF	Floating point exception.
C2E	0x12	Coprocessor 2 exception.
WATCH	0x17	WatchHi/WatchLo exception.
MCheck	0x18	Machine check exception.
CacheErr	0x1e	Cache error exception.

Table B.3: MIPS CP0 Cause Register Exception Code Field

B.4.3 CP0 Status Register

The CP0 Status register format varies somewhat between different MIPS architecture implementations. This is the format of the MIPS64 architecture Status register, which is used in the Broadcom SB1 and NetLogic XLR/XLS cores:

Field Name	Bits	Description
CU3	31	Coprocessor 3 access enable. Reserved in both SB1 and XLR/XLS.
CU2	30	Coprocessor 2 access enable. Reserved in SB1.
CU1	29	Coprocessor 1 (floating point) access enable. Reserved in XLR/XLS.
CU0	28	Coprocessor 0 access enable.
RP	27	Reduced power operation enable. No effect on XLR, XLS, or SB1.
FR	26	Floating-point register encoding. Reserved for XLR and XLS.
RE	25	Reverse-endian user mode enable.
MX	24	MDMX instruction access enable. Reserved for XLR and XLS.
PX	23	Enables access to 64-bit operations without enabling 64-bit addressing.
BEV	22	Exception vector location (0=Normal, 1=Boot ROM).
TS	21	Indicates that the TLB has detected a match on multiple entries (fatal).
SR	20	Indicates whether the last reset was due to a soft reset (0 for hard reset or NMI, 1 for soft reset.
NMI	19	Indicates whether the last reset was due to an NMI (0 for hard or soft reset, 1 for NMI.
Rsvd	17-18	Reserved.
SBX	16	Enables instruction set extensions for SB1. Reserved for XLR/XLS.
IM0-7	8-15	Interrupt mask 0-7.
KX	7	Enable 64-bit addressing for kernel mode.
SX	6	Enable 64-bit addressing for supervisor mode.
UX	5	Enable 64-bit addressing for user mode.
KSU	3-4	Identifies current operating mode (0=user, 1=supervisor, 2=kernel).
ERL	2	Error level. Set when reset, soft reset, NMI, or cache error exception is taken.

EXL	1	Exception level. Set when exception other than reset, NMI, or cache error is taken.
IE	0	Interrupt enable (0=disabled, 1=enabled).

Table B.4: MIPS CP0 Status Register Fields

B.5 MIPS Virtual Memory Segments

The following virtual memory segments are defined by the MIPS32 architecture (and MIPS64 in 32-bit compatibility mode):

kseg0 0x80000000-0x9FFFFFFF
Cached, accessible only in kernel mode, bypasses the MMU.
kseg1 0xA0000000-0xBFFFFFFF
Uncached, accessible only in kernel mode, bypasses the MMU.
kseg2 0xC0000000-0xDFFFFFFF
Accessible only in kernel and supervisor modes, mapped via the MMU.
kseg3 0xE0000000-0xFFFFFFFF
Accessible in only kernel mode, mapped via the MMU.
useg 0x00000000-0x7FFFFFFF
Accessible in user, kernel, and supervisor mode, mapped via the MMU.

Linux uses the MIPS virtual address segments as follows:

kseg0 0x80000000-0x8FFFFFFF
Kernel code and data, first 256MB of physical RAM.
kseg1 0xA0000000-0xBFFFFFFF
Uncached access to peripherals and the first 256MB of physical RAM.
kseg2 0xC0000000-0xDFFFFFFF
Kernel virtual memory, allocated and freed via vmalloc()/vfree().
Kernel modules are loaded in this segment.
kseg3 0xE0000000-0xFFFFFFFF
Not used by the Linux kernel (??).
useg 0x00000000-0x7FFFFFFF
User mode code, data, and memory mapped regions..

For user mode processes, addresses within **useg** are typically assigned as follows:

Address Range	Typical Usage
`0x00400000-0x0FFFFFFF`	User mode applications.
`0x10000000-0x1xxxxxxx`	Heap memory (allocated and freed via sbrk()).
`0x20000000-0x6FFFFFFF`	Shared library code/data, heap memory (allocated and freed via mmap() and munmap()), other mmap() regions.
`0x7xxxxxxx-0x7FFFFFFF`	User process stack starts at the top of this range and grows downward.

B.6 Further Reading

For a good introduction and in-depth coverage of the MIPS architecture, *See MIPS Run* [Swe99] is hard to beat (an updated version appears to be coming, you may want to wait before rushing to your favorite bookstore to get this one). The authoritative references to the MIPS architecture are published by MIPS Inc [MT01a, MT01b, MT01c]. For implementation-dependent features, refer to CPU vendor documentation.

Bibliography

[MT01a] Inc. MIPS Technologies. *MIPS64 Architecture for Programmers Volume I: Introduction to the MIPS64 Architecture*. MIPS Technologies, Inc., 2001.

[MT01b] Inc. MIPS Technologies. *MIPS64 Architecture for Programmers Volume II: The MIPS64 Instruction Set*. MIPS Technologies, Inc., 2001.

[MT01c] Inc. MIPS Technologies. *MIPS64 Architecture for Programmers Volume III: The MIPS64 Privileged Resource Architecture*. MIPS Technologies, Inc., 2001.

[Swe99] Dominic Sweetman. *See MIPS Run*. Morgan Kaufmann Publishers, Inc., 1999.

Appendix C

Jerry/Debug Commands

C.1 aaa Debug Commands

debug aaa configure failsafe-account deny all permanently

jerry [**policy-client** | **policy-client-get**] <*option*> { <*polname*> }

C.2 acl Debug Commands

debug acl clear hal-pending-flag { **policy** <*polName*> }

debug acl show dynamic pending-list

debug show access-list dynamic

debug show access-list dynamic policy <*PolicyName*> { **detail** }

debug show access-list refresh-counter

jerry acl [**cfgacl** <*value1*> <*value2*> | **adddelloop** <*value1*> <*value2*> <*value3*>]

jerry clear clear-flow <*aclname*> [**any** | **ports** <*port*> | **vlan** <*vlanname*> { **ports** <*portlist*> }] { **rule** <*rulename*> }

jerry clear-flow [[**debuglevel** | **counterlimit** | **testlib**] <*value*> | **mallocstats** { <*value*> } | **counterstats** | **syscnt**] { <*value*> }

jerry configure access-list <*aclname*> [**any** | **vlan** <*vlanname*> | **ports** <*portlist*>] { **precedence-normal** | **precedence-high** | **precedence-low** }

jerry show access-list

jerry show access-list [<*aclname*> { **rule** <*rulename*> } | **any** | **port** <*port*> | **vlan** <*vlanname*>] { **ingress** | **egress** }

jerry unconfigure access-list { <*aclname*> | **any** | **vlan** <*vlanname*> | **ports** <*portlist*> } { **precedence-normal** | **precedence-high** | **precedence-low** | **ingress** }

C.3 bfd Debug Commands

debug bfd configure [**maxCtrlSession** <*int1*> | **maxEchoSession** <*int2*> | **minCtrlTxIntval** <*int3*> | **minCtrlRxIntval** <*int4*> | **minEchoRxIntval** <*int5*>]

debug bfd create sessType <*int1*> **protoFamily** <*int2*> **ttl** <*int3*> **vrId** <*int4*> **destIP** [<*dipaddr*> | <*dipv6addr*>] { **srcIP** [<*sipaddr*> | <*sipv6addr*>] } { **vlan** <*vlan_name*> } { **client** <*client_name*> }

debug bfd delete vrId <*int1*> **destIP** [<*dipaddr*> | <*dipv6addr*>] { **srcIP** [<*sipaddr*> | <*sipv6addr*>] } { **vlan** <*vlan_name*> } { **client** <*client_name*> } { **protoFamily** <*int2*> }

debug bfd show

debug bfd show [**session** { <*ipaddress*> } | **client** | **interface** | **vlanIfMap** | **snmpTraps**]

C.4 bgp Debug Commands

debug bgp configure [**add** | **del**] **ipv4 route** [<*ipNetmask*> | <*ipaddress*> <*netmask*>] <*nhop*> { **LNH** } { **multicast** } { **l3vpn-label** <*label*> } { **vr** <*vrld*> } [**ebgp** | **ibgp**]

debug bgp configure [**add** | **del**] **ipv6 route** <*ipv6Netmask*> <*nhop6*> { **LNH** } { **vr** <*vrld*> } [**ebgp** | **ibgp**]

debug bgp configure internal-diagnostics [**enable** | **disable**] **memory verification**

debug bgp configure internal-diagnostics [**enable** | **disable**] **trace developer-logs**

debug bgp configure internal-diagnostics [**enable** | **disable**] **trace event**

debug bgp configure internal-diagnostics [**enable** | **disable**] **trace internal**

debug bgp configure internal-diagnostics [**enable** | **disable**] **trace ips**

debug bgp configure internal-diagnostics [**enable** | **disable**] **trace perf**

debug bgp configure internal-diagnostics assert-method [**assert** | **exit** | **trap** | **continue**]

debug bgp configure internal-diagnostics dump dmp-trace <*product*> { <*product2*> { <*product3*> { <*product4*> { <*product5*> } } } }

debug bgp configure internal-diagnostics dump ips-trace

debug bgp configure internal-diagnostics dump memory [**usage** | **tuning**] { **text** <*string*> }

debug bgp configure internal-diagnostics memory verification frequency <*frequency*>

debug bgp configure internal-diagnostics trace [**stop** | **start**]

debug bgp configure internal-diagnostics trace internal filter [**in** | **out**] { **func** } [**clear** | <*string*>]

debug bgp configure internal-diagnostics trace internal level <*level*>

debug bgp configure internal-diagnostics trace ips buffer-size <*size*>

debug bgp configure internal-diagnostics trace ips filter [**type** | **proc**] [**in** | **out**] [**clear** | <*string*>]

debug bgp configure internal-diagnostics trace pd filter [**in** | **out**] [**clear** | <*string*>]

debug bgp configure internal-diagnostics trace pd level [**default** | **debugging** | **explicit** [**none** | **problem** | **exception** | **audit**]]

debug bgp configure memstate <*state*>

debug bgp show internal-diagnostics { **detail** }

debug bgp show routes { { **address-family** } <*address_family*> } **summary** [**as-path** <*path-expression*> | **community** [**no-advertise** | **no-export** | **no-export-subconfed** | **number** <*community-number*> | <*autonomous-system-id*> : <*bgp-community*>] | **network** [**any** / <*netMaskLen*> | <*networkPrefixFilter*>] { **exact** }]

debug bgp show tech message { **pools** | **statistics** }

C.5 brm Debug Commands

debug brm show bandwidth clients

debug brm show bandwidth clients history { **clear** } { **keep** <*numhist*> } { **roll** | **no-roll** }

C.6 ces Debug Commands

debug ces-mpls debugMsg [**on** | **off**]

debug ces-mpls show clientLib [**ces** | **pw**] { <*ces_name*> }

debug show ces getCesPwInst [<*ces_name*> | **all**]

debug show ces mpls

debug show ces { <*ces_name*> }

C.7 cli Debug Commands

debug cli echo <*varname*>

debug cli show result

C.8 devmgr Debug Commands

debug devmgr [**disable** | **enable**] [**slot** [**temperature-check** | **power-off-on-failure**]]

debug devmgr clear non-volatile-memory slot [<*slot*> | **all-io-fm**]

debug devmgr configure power supply <*ps_num*> [**auto** | **on**]

debug devmgr configure reboot-loop-protection threshold <*time-interval*> <*count*>

debug devmgr set dmlib [[<*process*> | **all**] **trace-level** <*trace_level*> | **printf-level** <*printf_level*>]

debug devmgr show [**slot configuration** | **device** <*dev_name*>]

debug devmgr show dmdev { <*device*> }

debug devmgr show dmlib <*process*>

debug devmgr show non-volatile-memory results { **slot** [<*slot*> | **all-io-fm**] }

C.9 dirser Debug Commands

debug ds show directory-services { <*entry*> }

jerry ds debug { **trace** <*trace-level*> } { **print** <*printf-level*> }

C.10 dot1ag Debug Commands

debug configure cfm domain <*domain_name*> **association** <*association_name*> { **ports** <*port_list*> **end-point** [**up** | **down**] } **transmit-interval** [**20** | **100** | **1000** | **10000** | **60000** | **600000**]

debug configure cfm segment <*segment_name*> { **frame-delay** | **frame-loss** } **transmit-interval** [**msec** <*msec_interval*> | **mins** <*mins_interval*>]

debug dot1ag [**create** | **delete**] **switch-mac-acl**

debug dot1ag [**enable** | **disable**] **hwsupport**

debug dot1ag [**enable** | **disable**] **trace**

debug dot1ag clear counters

debug dot1ag configure [**max_upmep** <*int1*> | **max_downmep** <*int2*> | **max_mip** <*int3*> | **max_segment** <*int4*> | **min_loss** <*min_loss*> **max_loss** <*max_loss*> | **max_swmep** <*swmep*> | **lmm_hal_timeout** <*lmm_hal_timeout*>]

debug dot1ag show [**vlanIf** | **pseudowire-db**]

debug dot1ag show counters

debug dot1ag show domains { **associations** { **ports** } }

debug dot1ag show max_limits

debug dot1ag show mep-group

debug dot1ag show segment <*segment_name*> { **mep** <*mep_id*> }

debug dot1ag show vpifs { **details** } { **rmeps** } { **lmm** }

debug show cfm { <*domain_name*> { <*association_name*> { **ports** <*port_list*> { **intermediate-point** | **end-point** [**up** | **down**] } | **vpls** } } }

C.11 eaps Debug Commands

debug eaps [**enable** | **disable**] [**pdu-checkpointing** | **priority-domains**]

debug eaps [**enable** | **disable**] **sd-shc-track**

debug eaps [**enable** | **disable**] **segment-shc-track**

debug eaps [**enable** | **disable**] **shc-debug**

debug eaps check config

debug eaps configure preforwarding <*val*>

debug eaps configure segup_max <*count*>

debug eaps configure unacktime <*seconds*>

debug eaps generate-v1-pdu <*eapsDomain*> [**primary** | **secondary**] { **port** } [**health** | **ringupflush** | **ringdownflush** | **linkdown** | **flushfdb** | **querylinkstatus** | **linkup**]

debug eaps pdu-routine shared-port <*port*> [**segmenthealthcheck** | **pathdetect** | **flushnotify** | **segnotify** | **segquery**]

debug eaps server [**enable** | **disable**] **notification-messages**

debug eaps server generate-notify-msg [**vlan-conn-status** | **flush-fdb**]

debug eaps server show clients

debug eaps show [**detail** | **summary** | **misc** | **domain-groups** | **checkpoint-stats** | **neighbor-info**]

C.12 elrp Debug Commands

debug elrp config port-disable-wait { <*seconds*> }

debug elrp dump [**disabled-ports** | **service-objects** | **global-info**]

C.13 elsm Debug Commands

jerry elsm ports <*portlist*> **sendevent** <*simEvent*>

jerry elsm ports <*portlist*> **startHelloSim**

jerry elsm ports <*portlist*> **startTxHello**

jerry elsm ports <*portlist*> **stopHelloSim**

jerry elsm ports <*portlist*> **stopTxHello**

jerry show elsm ports [<*portlist*> | **all**]

C.14 ems Debug Commands

debug ems [**enable** | **disable**] **trace** [<*process*> { **slot** <*slot*> } | **all**] [<*buffer*> | **all**]

debug ems clear trace [<*process*> { **slot** <*slot*> } | **all**] [<*buffer*> | **all**]

debug ems show clients

debug ems show components { <*index*> } { **family** } { **checkpoint** }

debug ems show library [<*process*> | **all**]

debug ems show trace [<*process*> { **slot** <*slot*> } | **all**] [**info** | [<*buffer*> | **all**] { **match** <*match*> } { **starting** <*start-sequence*> } { **until** <*end-sequence*> } { **logto** [**cli** | **printf** | **log** | **file** { <*file-name*> }] }]

C.15 epm Debug Commands

debug epm [**enable heapdbg process** <*name*> | **disable heapdbg process** [<*name*> | **all**]]

debug epm [**enable heaptrace process** <*name*> | **disable heaptrace process** [<*name*> | **all**]]

debug epm [**enable** | **disable**] **heartbeat process** [<*name*> | **all**]

debug epm [**enable** | **disable**] **memory-monitoring**

debug epm clear trial-license

debug epm configure process <*name*> <*module*> **trace** <*trace-level*>

debug epm configure trial-license <*secs*>

debug epm configure upgrade { **verbose** | **quiet** }

debug epm enable trial-license

debug epm reboot [**self** | **slots** <*slot_list*> { **force** }]

debug epm show process license

debug epm show heapdbg

debug epm show heaptrace

debug epm show nvram

debug epm show peers

debug epm show process <*name*> <*module*> { <*id*> }

debug msgsrv configure msgqueues ktrace <*level*>

debug msgsrv show msgqueues { **detail** }

debug performance [[**stop** | **terminate** | **show**] **profiling** | [**start** | **restart**] **profiling** { **buffer-size** <*size*> }]

debug performance upload profile <*ipaddress*> { <*prefix*> }

jerry [**show** | **start**] **remote-debug process** <*name*>

jerry clear image partition

jerry disable license service

jerry force_synchronize

jerry upload mtrace <*ipaddress*> { <*name*> }

jerry xx_force_synchronize

jerry xx_synchronize { **slot** <*slot-number*> }

jerry xx_synchronize2

C.16 erps Debug Commands

debug erps [**all** | **control-path** { **exos** | **cfm** } | **data-path** | **none**]

debug erps show <*RingName*>

C.17 esrp Debug Commands

jerry set esrp debugPktDrop <*drop*>

jerry show esrp ports

jerry show esrp ports <*esrpDomain*>

jerry show esrp ports <*ports*>

jerry show esrp secondary-mac-table

jerry show esrp vlan <*vlan_name*>

jerry show esrp { <*esrp_name*> } **pseudowire-data**

C.18 ethoam Debug Commands

debug ethernet oam [**enable** | **disable**] **link-fault-management**

debug ethernet oam show debug-info { **ports** <*port_list*> { **port-data** | **statistics** | **support** | **hallink-data** | **all** } } { **global-data** }

debug ethernet oam transmit_pdu port <*port_num*> { **count** <*times*> { **interval** <*milli-seconds*> } }

C.19 etmon Debug Commands

jerry configure rmon alarm <*index*> { **interval** <*interval*> } { **variable** <*variable*> } { **sample-type** [**absoluteValue** | **deltaValue**] } { **startup-alarm** [**risingAlarm** | **fallingAlarm** | **risingOrFallingAlarm**] } { **rising-threshold** <*rising_threshold*> } { **falling-threshold** <*falling_threshold*> } { **rising-event** <*rising_event*> } { **falling-event** <*falling_event*> } { **owner** <*owner*> } { **status** [**valid** | **createrequest** | **undercreation** | **invalid**] }

jerry configure rmon event <*index*> { **description** <*description*> } { **type** [**none** | **log** | **snmp-trap** | **log-and-trap**] } { **community** <*community*> } { **owner** <*owner*> } { **status** [**valid** | **createrequest** | **undercreation** | **invalid**] }

jerry configure rmon history <*index*> { **datasource** <*oid*> } { **owner** <*owner*> } { **status** [**valid** | **createrequest** | **undercreation** | **invalid**] } { **interval** <*interval*> } { **bucket** <*bucket*> }

jerry configure rmon probe download { **file** <*file-name*> } { **server** <*ip*> } { **action** [**no** | **ram** | **pram**] }

jerry configure rmon probe reset-control [**running** | **warm** | **cold**]

jerry configure rmon probe time <*time*>

jerry configure rmon statistics <*index*> { **datasource** <*oid*> } { **owner** <*owner*> } { **status** [**valid** | **createrequest** | **undercreation** | **invalid**] }

jerry rmon <*func*> { <*param1*> } { <*param2*> }

jerry sflow [**clear** | **show**] **stats**

jerry sflow [**enable** | **disable**] **stats message**

jerry show rmon alarm { <*index*> }

jerry show rmon event { <*index*> }

jerry show rmon extreme-statistics { <*index*> }

jerry show rmon history-control { <*index*> }

jerry show rmon history-data { <*index*> { <*sample-index*> } }

jerry show rmon log { <*index*> { <*log-index*> } }

jerry show rmon probe-config

jerry show rmon statistics { <*index*> }

C.20 exsshd Debug Commands

debug ssh2 show access-profile

jerry ssh2 <*debug_level*>

C.21 fcoe Debug Commands

debug fip snooping delete acl <*acl_name*> **port** <*port*>

debug fip snooping set level <*num*>

debug fip snooping show [**vlan** | **fcf**]

debug fip snooping show trace

C.22 fdb Debug Commands

debug fdb [**enable** | **disable**] **vsm peer-sync**

debug fdb [**enable** | **disable**] **vsm trace**

debug fdb [**set** | **clear**] [**ld** | **rd** | **both**] **fdbentry** [**all** | <*mac_addr*> <*vlan_name*>]

debug fdb clear neighbor-discovery statistics { { **vr** } <*vr_name*> }

debug fdb configure aging-thread max-processing-count <*iterations*>

debug fdb configure aging-thread { **sleep-more** | **sleep-done** } <*msec*>

debug fdb configure server-debug-level <*level*>

debug fdb configure vrrp-mlag [**static** | **dynamic**]

debug fdb create fdbentry <*mac_addr*> { **vlan** } <*vlan_name*> **ports** <*port_list*> { **msrp** }

debug fdb modify fdbentry <*mac_addr*> { **vlan** } <*vlan_name*> [**add** | **delete**] **ports** <*port_list*> { **msrp** }

debug fdb run vsm <*debug_no*>

debug fdb show aging-thread

debug fdb show base filters { **mac** <*mac_addr*> **vlan-id** <*vlan_id*> }

debug fdb show fdbentry [**all** { **isc** [<*isc_index*> | **peer** <*peer_name*>] } | <*mac_addr*> <*vlan_name*>]

debug fdb show fdbentry port <*port_num*> { **vlan** } <*vlan_name*>

debug fdb show filters

debug fdb show globals

debug fdb show iparp statistics { { **vr** } <*vr_name*> }

debug fdb show isc <*isc_port*>

debug fdb show mac-tracking

debug fdb show mlag <*mlag_port*>

debug fdb show neighbor-discovery statistics { { **vr** } <*vr_name*> }

debug fdb show vsm { **vlan** <*vlan_name*> | **isc** [<*isc_index*> | **peer** <*peer_name*>] }

debug fdb sync isc [<*isc_index*> | **peer** <*peer_name*>]

debug fdb unconfigure aging-thread

jerry clear counter fdb

jerry clear fdb debug stats

jerry configure cpu fdbentry <*filterId*> [**add** | **delete**] [**ingress** | **egress**] **ports** <*portList*>

jerry create cpu fdbentry <*macAddr*> **vlanId** <*vlanId*> **ports** <*portList*>

jerry delete cpu fdbentry <*filterId*>

jerry printk [**enable** | **disable**]

jerry show counter fdb

jerry show cpu fdbentry

jerry show debug fdb { <*debug_no*> }

jerry show fdb debug stats

C.23 flowvsr Debug Commands

jerry flow-vsr test <*testnum*> { **port** <*port*> } { **ip** <*ipa*> } { <*param1*> } { <*param2*> }

C.24 gptp Debug Commands

debug gptp send <*src_port*> { <*dst_mac_addr*> }

debug gptp set [**clock-class** <*clock_class*> | **clock-accuracy** <*clock_accuracy*> | **clock-identity** <*clock_id_mac_addr*> | **offset-scaled-log-variance** <*offset_scaled_log_variance*>]

debug gptp set debug-latency <*debug_level*>

debug gptp set ports [<*port_list*> | **all**] [**ingressLatency** <*latency_ns*> | **egressLatency** <*latency_ns*> | **latency** <*latency_ns*> [**bcmUnit**] <*bcmUnit_val*> [**bcmPort**] <*bcmPort_val*>]

debug gptp show ports [<*port_list*> | **all**] **latency**

C.25 hal Debug Commands

debug hal [**disable** | **enable**] **card-error-recovery**

debug hal [**disable** | **enable**] **slot-led-control**

debug hal [**enable** | **disable**] **vsm redirect**

debug hal [**run** [**platform** | **sys-health-check**] <*cmd*> { <*parm1*> { <*parm2*> { <*parm3*> { <*parm4*> } } } } | **show platform** <*cmd2*> { <*string1*> }]

debug hal clear [**ipv4Mc** | **sys-health-check**] **counters**

debug hal configure [**function** | **cardPresentMask** | **fanPresentMask** | **pwrSupplyPresentMask** | **deviceState** | **deviceCommand** | **debugTraceMask** | **cardError** | **systemError** | **ioKeepAlive** | **ioNolinkTimeout** | **controlPingTimeout** | **lpm** | **msgStatsClear** | **cardPollPeriod** | **hpcc** | **ipg** | **backplane-port-ls** | **forceCardTemperature** | **forcePsTemperature** | **forceFanPercent** | **sysrq** | **packet-processor**] { [<*parm1*> | <*string*>] { <*parm2*> { <*parm3*> { <*parm4*> } } } }

debug hal configure counter-reduction slot [<*slot*> | **all**] [**on** | **off**]

debug hal configure debug-value <*value*>

debug hal configure emcm [**control-path-interlink** | **slow-path-interlink**] **port** [**1** | **2**] [**enable** | **disable**]

debug hal configure emcm [**control-path-port** | **slow-path-port**] **slot** [<*slot*> | **all**] [**default** | **alternate**]

debug hal configure emcm [**control-path** | **slow-path**] **maxbuffer** [<*buffer_percent*> | **default**]

debug hal configure emcm cpu-queue <*queue_no*> **weight** [<*weight_val*> | **default**]

debug hal configure emcm threshold port [<*port*> | **all**] { **queue** <*queue_no*> } **shared-packet-buffer** [<*buffer_percent*> | **default**]

debug hal configure forceBackplaneMode [**normal** | **4x20Gb** | **8x16Gb**] { **slot** <*slotNo*> }

debug hal configure gbic_i2c <*port*> <*i2c_addr*> <*reg*> <*value*>

debug hal configure gbic_i2c8 <*port*> <*i2c_addr*> <*reg*> <*value*>

debug hal configure hwa-oam debug-level <*debug-level*>

debug hal configure ipv6 mc-control <*number*>

debug hal configure ports <*port_list*> [**link** [**flap** { <*msec*> } | **up** | **down** | **restore**]]

debug hal configure ports <*port_list*> [**shutdown** | **dumplink** | **dumpstats** | **enable-mid-filter** | **disable-mid-filter** | **update-rom** | **load-ram** | **corrupt-rom**]

debug hal configure ports <*port_list*> [**transceiver** [**rescan** | **remove** | **restore**]]

debug hal configure ports <*port_list*> **link-status** [**hardware** | **software**]

debug hal configure psu <*command*> <*psuNum*>

debug hal configure ptp slot <*slot_no*> **priority1** <*priority_val*>

debug hal configure ptp slot <*slot_no*> **priority2** <*priority_val*>

debug hal configure ptp slot <*slot_no*> [**add-port** | **delete-port**] <*ptp_port_number*> <*port_vlan*> <*ipv4_addr*> <*multicast*>

debug hal configure ptp slot <*slot_no*> [**topinit** | **topready** | **phy-sync**]

debug hal configure ptp slot <*slot_no*> **acceptable-master** [**enable** | **disable**] <*ptp_port_number*>

debug hal configure ptp slot <*slot_no*> **acceptable-master add** <*ipv4_addr*> <*priority1*> <*ptp_port*>

debug hal configure ptp slot <*slot_no*> **acceptable-master remove** <*ipv4_addr*> <*ptp_port*>

debug hal configure ptp slot <*slot_no*> **announce interval** <*ptp_port_number*> <*log_announce_interval*>

debug hal configure ptp slot <*slot_no*> **announce timeout** <*ptp_port_number*> <*log_announce_timeout*>

debug hal configure ptp slot <*slot_no*> **bc-create** <*num_ports*>

debug hal configure ptp slot <*slot_no*> **domain** <*domain_val*>

debug hal configure ptp slot <*slot_no*> **dpll-reg-write** <*reg_addr*> <*reg_val*>

debug hal configure ptp slot <*slot_no*> **e2etc-create**

debug hal configure ptp slot <*slot_no*> **min-delay-request-interval** <*ptp_port_number*> <*log_min_delay_req_interval*>

debug hal configure ptp slot <*slot_no*> **sync-interval** <*ptp_port_number*> <*log_sync_interval*>

debug hal configure ptp slot <*slot_no*> **unicast master add** <*port_vlan*> <*mac-address*> <*ipv4_addr*> <*log_sync_interval*> <*log_min_delay_request_interval*>

debug hal configure ptp slot <*slot_no*> **unicast master remove** <*ipv4_addr*>

debug hal configure ptp slot <*slot_no*> **unicast negotiation** [**enable** | **disable**] <*ptp_port_number*>

debug hal configure ptp slot <*slot_no*> **unicast slave add** <*ptp_port_number*> <*vlan_id*> <*ipv4_addr*> <*log_announce_interval*> <*log_sync_interval*> <*log_delay_request_interval*> <*remote_port_number*>

debug hal configure ptp slot <*slot_no*> **unicast slave remove** <*ptp_port_number*> <*ipv4_addr*>

debug hal configure pwm [**help** | <*command*> { <*string1*> { <*string2*> { <*string3*> } } } { <*parm1*> { <*parm2*> { <*parm3*> { <*parm4*> { <*parm5*> { <*parm6*> { <*parm7*> } } } } } }]

debug hal configure stacking pdu-trace [**mask** <*hex-mask*> | **capture** <*filename*>]

debug hal configure stacking port <*port-number*> [**enable** | **disable** { **data-only** } | **flap** <*flap-period-ms*>]

debug hal configure switch-manager trace { **run** | **stop** | **none** } { **filter-mask** <*mask*> }

debug hal configure temp-flooding [**off** | **on**]

debug hal configure temp_alarm <*degrees*>

debug hal configure temp_alarm_low <*degrees*>

debug hal disable ptp ports <*port_list*> **e2etc**

debug hal enable ptp ports <*port_list*> **e2etc**

debug hal manufact_image <*string*>

debug hal run external-table calibration [**slot** <*slot*> | **system**]

debug hal run manufacturing-init

debug hal run pib_acl <*command*>

debug hal run platform int-stat slot <*slot*> **unit** <*unit*>

debug hal run platform port_dir_en slot <*slot_num*> **unit** <*unit_num*> **port** <*port_num*> [**disable** | **enable**] [**rx** | **tx**] [**physical** | **data-only**]

debug hal run platform pwe3menu { <*menuopts*> }

debug hal run simulate-hardware-error [**slot** <*slot*> | **system**] [**on** | **off**]

debug hal run vsm [**block** | **unblock**] **ingress-port** <*ingress_port*> **block-port** <*block_port*>

debug hal run vsm redirect [**enable** | **disable**] **egress-port** <*egress_port*> **redirect-port** <*redirect_port*>

debug hal show [[[**ipv4Fib** | **ipv4Host** | **ipv6Fib**] { **dest** <*dest*> } | **ipv4Adj** { **nhop** <*nhop*> } { **routes** | **hosts** | **hw-info** | **all** } | **ipv6Adj** { **nhop6** <*nhop6*> } { **routes** | **hw-info** | **all** }] { **vrld** <*vrld*> } | **ipv4Sharing** | **ipv6Sharing**] **slot** <*slot*>

debug hal show [**fdb** { **vlanId** <*vlanId*> } { **pti** <*pti*> } | **ipv4Intf** { **vlanId** <*vlanId*> } { **pti** <*pti*> } { **vrld** <*vrld*> } | [**ipv4Fib** | **ipv6Fib** | **ipv4Host**] { **dest** <*dest*> } { **vrld** <*vrld*> } | [**ipv4Mc** { **flood** } | **ipv6Mc**] { **gaddr** <*gaddr*> } { **pti** <*pti*> } { **vrld** <*vrld*> } { **slot** <*slot*> } | **ipv4Adj** { **nhop** <*nhop*> } { **routes** | **hosts** | **hw-info** | **all** } { **vrld** <*vrld*> } | **mplsIlm** | **ipv6Adj** { **nhop6** <*nhop6*> } { **routes** | **hw-info** | **all** } { **vrld** <*vrld*> } | **tunnel** { **all** } | **forwarding distributions** [**slot** <*slot*> | **system**] | **rxLosStruct** { **slot** <*slot*> } | **fefiDump** { **slot** <*slot*> } { **port** <*port*> } | **sfpPhyDump** { **slot** <*slot*> } { **port** <*port*> } | **dwdmChannel slot** <*slot*> **port** <*port*>]

debug hal show [**optic-info** { **eeprom** | **ddmi** } { **port** <*portno*> }] [**NULL** | **port** <*portno*>]

debug hal show [**optic-info** { **eeprom** | **ddmi** }] [**slot** <*slotno*>] [**NULL** | **port** <*portno*>]

debug hal show [**sys-health-check** { **detail** } | **sharing** { **interfaces** | **lbn** { **slot** <*slotno*> } } | **conduits** | **upgrade-compatibility**]

debug hal show [**version** | **crossbar-stats** { **tx** | **rx** | **txerr** | **rxerr** | **clear** }] [**NULL** | **slot** <*slotno*> | **msm** [**a** | **b**] | **mm** [**a** | **b**] | **port** <*portno*>]

debug hal show boot-codes mm [**a** | **b**]

debug hal show compact-flash

debug hal show congestion { **slot** <*slot-number*> | **msm-a** | **msm-b** }

debug hal show cpu-queue { **slot** [<*slot*> | **all**] }

debug hal show device 10GBaseT link-errors [[**LDPC** | **CRC8**] **ports** [<*portlist*> | **all** { **slot** <*slotno*> }] | **Noise ports** [<*portlist*> | **all** { **slot** <*slotno*> }] { **minutes** <*minutes*> } | **ports** [<*portlist*> | **all** { **slot** <*slotno*> }]]

debug hal show device [[**fdb** | **ipv4-routing** | **ipv4-multicast** | **port-info** | **acl-rule** | **acl-mask** | **acl-slice** | **acl-range** | **load-sharing**] [**slot** <*slotno*> | **system**] **unit** <*unit*> | **vlan-info** [**slot** <*slotno*> | **system**] **unit** <*unit*> { **vlanId** <*vlanId*> }]

debug hal show device [**ge-mac** | **xge-mac** | **portstat** | **spi4** | **eap** | **slowpath** | **fap** | **psi** | **prfl**] <*cmd*> { <*parm1*> { <*parm2*> { <*parm3*> { <*parm4*> { <*parm5*> } { <*parm6*> } } } } } { **string1** <*string1*> } { **string2** <*string2*> }

debug hal show device [**switch-fabric** | **packet-processor** | **10G-mac**] **regs** { <*par1*> { <*par2*> { <*par3*> } } }

debug hal show device [**ucpipe**] <*cmd*> { <*parm1*> { <*parm2*> { <*parm3*> { <*parm4*> { <*parm5*> { <*parm6*> } } } } } } { **string1** <*string1*> } { **string2** <*string2*> }

debug hal show device registers [[**slot** <*slotno*> | **system**] **unit** <*unitno*> | **phy ports** [<*portlist*> | **all** { **slot** <*slotno*> }]]

debug hal show eeprom [**1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **msm** | **chassis** | **msm_other** | **msm_board**] <*address*> <*length*>

debug hal show emcm [**l2table** | **cpu-queue-stats** | **cpu-queue-weights** | **service-pool**] **slot** <*slot*>

debug hal show emcm events

debug hal show fabric-manager [**uc** { <*modid*> } | **vlan** { <*vlanid*> } | **ipmc** { <*group-index*> } | **info** | **l2mc** { <*l2-group-index*> }]

debug hal show forwarding distributions

debug hal show gbic_i2c <*port*> <*i2c_addr*> <*reg*>

debug hal show gbic_i2c8 <*port*> <*i2c_addr*> <*reg*>

debug hal show platform [**platformInfo** | **cardMisc** | **fdb** | **linkInfo** | **cardInfo** | **cardCapability** | **msgStats** | **deviceInfo** | **debugTraceMask** | **IPv4Fib** | **IPv6Fib** | **IpAdj** | **Ipv6Adj** | **IPv4Mc** | **IPv6Mc** | **portStats** | **version** | **vlan**] { [<*parm1*> | <*string*>] { <*parm2*> { <*parm3*> { <*parm4*> } } } }

debug hal show platform acl [**AclTree** | **DynRuleTree** | **AclInstTree** | **DynCounterTree** | **Memory** | **MeterTable** | **PbrTable** | **FlowRedirectTable**]

debug hal show platform connectInfo { **path** [**fast-path** | **slow-path** | **control-path** | **all-paths**] } { **channel** <*chan-num*> } { **instance** <*inst-num*> } { **entries** [**valid** | **invalid** | **all**] } { **slot** <*ioslotno*> } { **downstatonly** }

debug hal show platform hwa-oam tables

debug hal show platform learning { **vlan** { <*vlanId*> } | **ports** { <*portList*> } | **all** }

debug hal show platform mirroring

debug hal show platform mpls { **neighbor** { <*ipaddr*> } | **nhlfe** { <*instance*> | **txmi** <*txmi*> } | **ilm** { <*instance*> | **label** <*label*> } | **pw** { <*pw_instance*> | **label** <*label*> | **pti** <*pti*> } | **vpls** { <*vlan_name*> } | **label-usage** }

debug hal show platform pstag { <*vlan_name*> }

debug hal show platform trill { **adjacency** | **fib** | **dtree** }

debug hal show platform vlan stats

debug hal show ports { <*port_list*> } **qosmonitor** { **congestion** } { **no-refresh** }

debug hal show ptp slot <*slot_no*> **default-data-set**

debug hal show ptp slot <*slot_no*> **acceptable-master-enabled** <*ptp_port_number*>

debug hal show ptp slot <*slot_no*> **acceptable-master-table**

debug hal show ptp slot <*slot_no*> **announce interval** <*ptp_port_number*>

debug hal show ptp slot <*slot_no*> **announce timeout** <*ptp_port_number*>

debug hal show ptp slot <*slot_no*> **current-data-set**

debug hal show ptp slot <*slot_no*> **domain**

debug hal show ptp slot <*slot_no*> **dpll-reg-read** <*reg_addr*>

debug hal show ptp slot <*slot_no*> **min-delay-request-interval** <*ptp_port_number*>

debug hal show ptp slot <*slot_no*> **parent-data-set**

debug hal show ptp slot <*slot_no*> **port-data-set** <*ptp_port_number*>

debug hal show ptp slot <*slot_no*> **port-info** <*ptp_port_number*>

debug hal show ptp slot <*slot_no*> **priority1**

debug hal show ptp slot <*slot_no*> **priority2**

debug hal show ptp slot <*slot_no*> **servo performance**

debug hal show ptp slot <*slot_no*> **servo status**

debug hal show ptp slot <*slot_no*> **sync-interval** <*ptp_port_number*>

debug hal show ptp slot <*slot_no*> **time-properties-data-set**

debug hal show ptp slot <*slot_no*> **unicast-master-table**

debug hal show ptp slot <*slot_no*> **unicast-slave-list**

debug hal show reach slot <*slot*> **port** <*port*>

debug hal show sflow { **slot** <*slot*> }

debug hal show stacking [[**active** | **stack**] { **suspended** | **viewable** } | **ocpd-info** | **ports**]

debug hal show switch-manager [**info** | **uc** { <*modid*> } | **vlan** { <*vlanid*> } | **ipmc** { <*group-index*> | **all** } | **l2mc** { <*l2-group-index*> | **all** } | **trace** { [**next** | **full** | **first** | **last**] { <*num-events*> } }]

debug hal show sys-health-check [**slot** <*slot*> { **detail** } | **detail**]

debug hal show vsm

debug hal show wkinfo [**card** <*slot*> | **chassis**] <*id*>

debug packet capture { **ports** <*port_list*> } [**on** { **vr-id** <*vr_id*> } { **interface** <*iface_name*> | **direction** [**rx** | **tx** | **both**] } { **mirror-to-cpu** } { **vlan** <*vlan-name*> } { **filter** <*filter_type*> } { **cmd-args** <*cmd_str*> } { **prefix** <*name_prefix*> | **file-name** <*filename*> | **print-to-console** } { **count** <*count*> } | **off** { <*ipaddress*> } { <*remote-dir*> } | **upload** { <*ipaddress*> } { <*remote-dir*> } | **to-file** <*filename*> { **cmd-args** <*cmd_str*> } | **help**]

jerry [**enable** | **disable**] **bootrom-debug**

jerry [**enable** | **disable**] **pacman**

jerry hal configure adm slot <*slot*> **list-id** <*list_id*>

jerry hal configure forcePs220v [**off** | **on**]

jerry hal eeprom write [**1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **msm** | **chassis** | **msm_other** | **msm_board**] <*address*> [**string** <*string*> | **byte** <*value*>]

jerry hal odometer [**reset** [**slot** <*slot*> | **chassis**] | **set** [**slot** <*slot*> | **chassis**] **value** <*value*> { **power-on-date** <*date*> }]

jerry hal platform bcm-cmd slot <*slot*> **unit** <*unit*> <*cmd-str*>

jerry hal run [**hot-alarm** | **cold-alarm** | **normal-alarm**]

jerry hal set_serial_num <*string*>

jerry hal show adm slot [<*slot*> | **all**]

jerry hal swidswap oem <*oemtype*> [**slot** <*slotno*> | **chassis**]

jerry hal swidswap { **slot** <*slotno*> }

jerry hal wkinfo write card <*slot*> <*id*> [**string** <*string*> | **hex** <*value*>]

C.26 hclag Debug Commands

debug hclag member-port <*port*> [**detach-from-lag** | **attach-to-lag**]

debug hclag member-port <*port*> **track-tcp** [**down** | **up**]

debug hclag show summary

C.27 idMgr Debug Commands

debug idmgr [**enable** | **disable**] **consoleprint**

debug idmgr [**enable** | **disable**] **full-domain-match**

debug idmgr configure cleanup-interval <*seconds*>

debug idmgr configure kerberos trace-ip <*ipaddress*>

debug idmgr flush

debug idmgr resolve ldap-server <*server-index*> [**host-name** | **connection-status**]

debug idmgr show [**gVariables** | **clients**]

debug idmgr show [**ldap-config** | **role-config** { <*role_name*> } | **policy-config** { <*policy_name*> } | **ldap-attr** | **user-attr** { <*user_name*> } | **configured-groups**]

debug idmgr show ksnoop asn1 last-error

debug idmgr show ksnoop gVariables

debug idmgr show memory-details

debug idmgr trigger fdb MacMove <*prevPort*> <*curPort*> <*vlan*> <*mac*>

debug idmgr trigger fdb [**MacAdd** | **MacAgeout**] <*port*> <*vlan*> <*mac*>

debug idmgr trigger fdb [**addIPv4Binding** | **delIPv4Binding**] <*port*> <*vlan*> <*mac*> <*ipaddress*>

debug idmgr trigger ip-security [**addIPv4Binding** | **delIPv4Binding**] <*port*> <*vlan*> <*mac*> <*ipaddress*>

debug idmgr trigger kerberos [**detect** <*cname*> <*realm*> <*port*> <*mac*> <*vlan*> <*ipaddress*> { <*netbios_host*> } | **ageout** <*cname*> <*realm*> <*port*> <*mac*>]

debug idmgr trigger lldp [**detect** | **undetect**] <*port*> <*mac*> <*capabilities*> <*model-name*> <*manufacturer-name*> <*system-description*>

debug idmgr trigger netlogin [**auth** [<*user*> <*port*> <*mac*> { <*vl1*> { <*vl2*> { <*vl3*> { <*vl4*> } } } } [**web** | **mac** | **dot1x**]] | **unauth** [<*user*> <*port*> <*mac*> [**web** | **mac** | **dot1x**]] | **reauth** [<*user*> <*port*> <*mac*> { <*vl1*> { <*vl2*> { <*vl3*> { <*vl4*> } } } } [**web** | **mac** | **dot1x**]] | **authfailure** [<*user*> <*port*> <*mac*> [**web** | **mac** | **dot1x**]]]

C.28 isis Debug Commands

debug isis [**enable** | **disable**] **debug-flags** [**events** | **ifsm** | **lsp** | **nfsm** | **nsm** | **pdu** | **spf** | **all**]

debug isis show configuration vlan <*vlan-name*>

debug isis show debug-flags

debug isis show ifdb

debug isis show is-neighbors { **area** <*area-name*> } { **vlan** <*vlan-name*> } { **detail** }

debug isis show lanids { **all** }

debug isis show routes { **area** <*area-name*> } { **ipv6** }

debug isis show vlan <*vlan-name*>

jerry isis <*func*>

jerry isis [**enable** | **disable**] <*func*> { <*val*> }

jerry isis vlan <*vlan-name*> <*func*>

C.29 lacp Debug Commands

debug lacp [**enable** | **disable**] **vsm trace**

debug lacp send-marker-pdu member-port <*port*>

debug lacp show isc [<*isc_index*> | **peer** <*peer_name*>]

debug lacp show mlag [**all** | <*mlag_index*>]

jerry lacp <*port*> **sendtrap** [**add** | **del**]

jerry lacp [**enable** | **disable**]

jerry lacp [**enable** | **disable**] **lag** <*group-id*>

jerry lacp [**enable** | **disable**] **member-port** <*port*>

jerry lacp show debug all

C.30 lldp Debug Commands

debug dcbx configure level <*value*>

C.31 mcmgr Debug Commands

debug [**mtrace** | **mrinfo**] [**enable** | **disable**] **pktDump**

debug [**mtrace** | **mrinfo**] **show** [**sess** | **stats**]

debug configure igmp snooping cache <*length*> <*cacheNum*>

debug mcmgr [**enable** | **disable**] **log checkpoint**

debug mcmgr [**igmp** | **mld**] **show ssm-map** { **vr** <*vrname*> }

debug mcmgr clear [<*grpaddress*> <*sourceIP*> | <*grpaddress6*> <*sourceIP6*>] <*vlanname*>

debug mcmgr configure [**kern-snoop-log** | **kern-mc-log**] [[**ipv4** | **ipv6**] { **cache-oper** } { **egress-pkt** } { **ingress-pkt** } | **warning** | **all** | **none**]

debug mcmgr configure ipv6 solicit-multicast { **install-in-hw** | **donot-install** | **forwarding-mode** [**group-vlan** | **source-group-vlan**] }

debug mcmgr show group [<*v4Group*> | <*v6Group*>] <*vlan_name*>

debug mcmgr show ipv6 stats { { **vlan** } <*name*> }

debug mcmgr show mlag { **vlan** } <*vlanname*> { **ports** <*portlist*> }

debug mcmgr show software-cache [<*grpaddress*> <*sourceIP*> | <*grpaddress6*> <*sourceIP6*>] <*vlanname*>

debug mcmgr show vlan <*vlanname*> [**ipv4** | **ipv6**]

jerry configure igmp { **hal** | **pim** } **smart-msg-batching** <*threshold*>

jerry mcm clear stats { { **vlan** } <*name*> }

jerry mcm set command <*num*>

jerry mcm show client

jerry mcm show igmp cache config

jerry mcm show stats { { **vlan** } <*name*> }

C.32 mpls Debug Commands

debug mpls [**create** | **delete**] **fdbentry** <*mac_addr*> { **vlan** } <*vlan_name*> { <*port_instance*> }

debug mpls [**create** | **delete** | **replace**] **ipv4** [<*ipPrefix*> | <*ipDest*> <*ipMask*>] <*nhlfeInstance*> { **vr** <*vrName*> }

debug mpls [**create** | **replace**] **ilm** <*ilmInstance*> <*rxLabel*> **egress** [**mpls** <*nhlfeInstance*> | **eth** <*egressVlanName*> { **spoke** } { **tagged** } | [**ipv4** | **ipv6**] { <*egressVrname*> }] { **count** }

debug mpls [**create** | **replace**] **nhlfe** <*nhlfeInstance*> { **vr** <*vrName*> } <*vlanName*> [**ipv4** <*ipv4Addr*> | **ipv6** <*ipv6Addr*> | **eth** <*macAddr*>] **label** <*label1*> { <*label2*> { <*label3*> { <*label4*> } } } { **meter** <*meterInstance*> } { **count** } { **tunNhlfe** <*tunNhlfeInstance*> }

debug mpls [**create** | **delete**] **ces** <*ces-name*>

debug mpls [**enable** | **disable**] **ces** <*ces_name*>

debug mpls clear dataplane { **hardware** | **software** }

debug mpls configure ces <*ces-name*> [**add** | **delete**] **peer** <*ces-pw-ip*> **pwId** <*pw-id*>

debug mpls configure debug-key-values <*key*> [<*value*> | **hex** <*hexvalue*>]

debug mpls configure internal-diagnostics [**enable** | **disable**] **memory verification**

debug mpls configure internal-diagnostics [**enable** | **disable**] **trace** [**developer-logs** | **verbose-logs**]

debug mpls configure internal-diagnostics [**enable** | **disable**] **trace event**

debug mpls configure internal-diagnostics [**enable** | **disable**] **trace internal**

debug mpls configure internal-diagnostics [**enable** | **disable**] **trace ips**

debug mpls configure internal-diagnostics assert-method [**assert** | **exit** | **trap** | **continue**]

debug mpls configure internal-diagnostics dump dmp-trace <*product*> { <*product2*> { <*product3*> { <*product4*> { <*product5*> } } } }

debug mpls configure internal-diagnostics dump ips-trace

debug mpls configure internal-diagnostics dump memory [**usage** | **tuning**] { **text** <*string*> }

debug mpls configure internal-diagnostics memory verification frequency <*frequency*>

debug mpls configure internal-diagnostics trace [**stop** | **start**]

debug mpls configure internal-diagnostics trace internal filter [**in** | **out**] { **func** } [**clear** | <*string*>]

debug mpls configure internal-diagnostics trace internal level <*level*>

debug mpls configure internal-diagnostics trace ips buffer-size <*size*>

debug mpls configure internal-diagnostics trace ips filter [**type** | **proc**] [**in** | **out**] [**clear** | <*string*>]

debug mpls configure internal-diagnostics trace pd filter [**in** | **out**] [**clear** | <*string*>]

debug mpls configure internal-diagnostics trace pd level [**default** | **debugging** | **explicit** [**none** | **problem** | **exception** | **audit** | **dev** | **devextra**]]

debug mpls configure lspmgr lspdown [**all** | **rsvp-te** | **ldp**]

debug mpls configure nhrquery [**default** | **cspf** | **rtmgr**]

debug mpls configure rsvp-te lsp [<*lsp_name*> | **all**] **reinstall**

debug mpls configure vpls <*vpls_name*> **saii** <*saii_value*>

debug mpls configure vpls <*vpls_name*> **taii** <*taii_value*>

debug mpls configure vpls max-num-peers <*max_peers*>

debug mpls configure vpls restrictions [**strict** | **loose**]

debug mpls create pw <*pwInstance*> { **vlan** } <*vlanName*> { **spoke** } { **tagged** } { **vtype** <*inVtype*> } { **vid** <*inVid*> } **txlabel** <*vclabel*> **nhlfe** <*nhlfeInstance1*> { <*nhlfeInstance2*> { <*nhlfeInstance3*> } }

debug mpls delete ilm <*ilmInstance*> { **egress** [**mpls** <*nhlfeInstance*> | **eth** <*egressVlanName*> | [**ipv4** | **ipv6**] { <*egressVrname*> }] }

debug mpls delete nhlfe <*nhlfeInstance*> { **vr** <*vrName*> }

debug mpls delete pw <*pwInstance*> { **nhlfe** <*nhlfeInstance1*> { <*nhlfeInstance2*> { <*nhlfeInstance3*> } } }

debug mpls disable rsvp-te hello-message [{ **vlan** } <*vlan_name*> | **vlan all**]

debug mpls disable rsvp-te local-repair

debug mpls enable rsvp-te hello-message [{ **vlan** } <*vlan_name*> | **vlan all**]

debug mpls enable rsvp-te local-repair

debug mpls send-eaps-req { **register** | **unregister** | **vlan-status** }

debug mpls send-mac-addr-wdrwl vpls <*vpls_name*> **peer** <*ipaddress*>

debug mpls show ces pwTree

debug mpls show ces { **list** | **tree** | <*ces_name*> }

debug mpls show cspf { { **database** } { <*db_max*> } | **cache** { <*cach_max*> } | **stats** } { **detail** }

debug mpls show debug-key-values

debug mpls show dpmdb hist [**lsp** <*lsp_id*> | **fec** <*ipmask*>]

debug mpls show dpmdb interface { <*if_index*> } { **detail** }

debug mpls show dpmdb { **lsp_xc_index** <*index*> { **detail** } | **in_seg_index** { <*index*> | **label** <*label_number*> } | **out_seg_index** { <*index*> | **label** <*label_number*> } | **xc** | **detail** }

debug mpls show fec

debug mpls show hal { **all** | **nhlfe** { **label** <*tx_label*> | **instance** <*nhlfe_instance*> { **all-refs** } } | **ilm** { **label** <*rx_label*> | **instance** <*ilm_instance*> } | **pw** { **label** <*pw_rx_label*> | **instance** <*pw_instance*> } | **summary** }

debug mpls show hello

debug mpls show internal-diagnostics { **detail** }

debug mpls show l2vpn global-data

debug mpls show l2vpn { **vpls** { <*vpls_name*> { **peer** <*ipaddress*> } } | **vpws** { <*vpws_name*> { **peer** <*ipaddress*> } } }

debug mpls show ldp ipr { **prefix** <*ipNetmask*> | **pseudo-wire** <*pwid*> }

debug mpls show lspdb [**all** | **fec** { <*ipmask*> } | **dpm** { <*dpm_handle*> } | **name** { <*lsp_name*> } | **lsplist** | **path** | **profile** | **tcb** | **pcb** | **rcb**] { **rsvp-te** | **ldp** | **static** }

debug mpls show lspdb static [**all** | **xcs** | **labels**]

debug mpls show lsr

debug mpls show mi-callback queue { **rtmgr** | **l2vpn** | **smsfromxdt** | **smstoxdt** | **dpmlsifromxdt** | **dpmlsitoxdt** } { **detail** }

debug mpls show mpls-capabilities

debug mpls show nhrquery

debug mpls show pw [**tpm-stats** | **tpm-fsm** { <*name*> { **peer** <*ipaddress*> } }]

debug mpls show rsvp-te hello-message { { **vlan** } <*vlan_name*> }

debug mpls show rsvp-te local-repair

debug mpls show tech [**processes** | **queues** | **cspf** | **rsm** | **rtmgr** | **ospf** | **fdb** | **l2vpn** | **rsvp-te** | **te-mib** | **te-mib-tunnel** | **te-mib-tunnel-error** | **te-mib-hop** | **te-mib-resource** | **te-mib-chop** | **te-mib-arhop** | **lspmgr** | **rlm** | **ldp-pm** | **ldp-sc** | **cspf-mj** | **cspf-ent** | **rsm-prod** | **fsm** | **malloc-stats** | **fixed-memory-stats** | **ldp-entity** | **ldp-peer** | **ldp-session** | **ldp-hello-adjacency** | **interfaces** | **label-stack** | **in-segment** | **in-segment-map** | **out-segment** | **xc** | **label-space** { <*label*> } | **pw-table** | **pw-mpls-table** | **pw-mpls-outbound-table**]

debug mpls show tech message { **pools** | **statistics** }

debug mpls show tunnels

debug mpls show vpls [**ldp** [**adjacency** | **callback-params**] | **vlan-tree**]

debug mpls show vpls lsp

debug mpls show vpls restrictions

C.33 mrp Debug Commands

debug msrp clear timers

debug msrp create streams { **static** | **dynamic** } **port** <*port_num*> <*num_streams*> { **accumulated-latency** <*accumulated_latency*> } { **base-stream-dest-mac** <*dest_mac*> } { **base-stream-id** <*stream_id*> } { **base-stream-src-mac** <*src_mac*> } { **data-frame-prio** <*data_frame_prio*> } { **max-frame-size** <*max_frame_size*> } { **max-interval-frames** <*max_interval_frames*> } { **mode** [**sequential** | **non-sequential**] } { **rank** <*rank_number*> } { **stream-type** [**advertise** | **fail**] } { **tag** <*vlan_tag*> }

debug msrp delete debug-streams

debug msrp set auto-listener [**on** | **off**] { **ports** [<*port_list*> | **all**] }

debug msrp set debug-level <*debug_level*>

debug msrp set ports [<*port_list*> | **all**] **boundary** [**auto** | **false**]

debug msrp set ports [<*port_list*> | **all**] **eav-mode** [**on** { **qp1** } { **qp2** } { **qp3** } { **qp4** } { **qp5** } { **qp6** } { **qp7** } { **qp8** } | **off**]

debug msrp show data

debug msrp show ports

debug msrp show streams

debug msrp show timers

debug msrp test latency <*port_num*>

debug msrp test { <*test_no*> }

debug mvrp rxpkt ports <*port_list*> **num_val** <*num_val*> **leave_all_val** <*leave_all_val*> { **first_val** <*first_val*> **event_val** <*event_val*> }

debug mvrp set [**debug-level** | **state-debug-level** | **parse-debug-level** | **build-debug-level**] <*debug_level*>

debug mvrp show client-usage

debug mvrp show data

debug mvrp show debug-level

debug mvrp show vlan

C.34 msdp Debug Commands

jerry [**create** | **delete**] **msdp local-sa** <*src*> <*grp*> <*rp*> { **vr** <*vrname*> }

jerry show msdp <*opcode*> { **vr** <*vrname*> }

C.35 netLogin Debug Commands

debug netlogin clear dynamic-vlan [**all** | <*vlan_name*>]

debug netlogin clear mac-cache

debug netlogin configure dot1x trace-level [**trace-buf** | **console**]

debug netlogin create client <*vlan*> <*port*> <*mac*>

debug netlogin create dot1x-client <*vlan*> <*port*> <*mac*> [**eapol-start** | **eap-identity** | **eap-ttls**]

debug netlogin show client <*port*> <*mac*>

debug netlogin show dot1x-clients { **vlan** <*vlan*> } { **port** <*port*> }

debug netlogin show mac-cache

debug netlogin show pif { <*port*> } { **clients** { **details** } }

debug netlogin show vpif { **vlan** <*vlan*> } { **port** <*port*> } { **clients** { **details** } }

debug netlogin show vr { **detailed** } { **clients** }

C.36 netTools Debug Commands

debug dhcpv6 [**enable** | **disable**] **consoleprint**

debug nettools create tcp-proxy mgmt-port <*mgmt_port*> **control-ip** <*control_ip*> **control-port** <*control_port*>

debug nettools delete tcp-proxy mgmt-port <*mgmt_port*> **control-ip** <*control_ip*> **control-port** <*control_port*>

debug nettools show bootprelay { **vlan** } <*vlan_name*> **statistics**

debug nettools show tcp-proxy

debug packet [**run** { **packet** } <*packet*> **ports** <*ports*> { **count** <*count*> } { **rate** <*rate*> | **delay** <*delay*> } | [**stop**] { **packet** } <*packet*>]

debug packet configure <*packet*> **type** [**eth etype** <*etype*> { **length** <*length*> } | **arp** | **ipv4** { **length** <*length*> } | **icmp** { **length** <*length*> } | **igmp** | **udp** { **length** <*length*> } | **tcp** { **length** <*length*> } | **wol**] { **dmac** <*dmac*> { <*dmac_step*> <*dmac_count*> } } { **smac** <*smac*> { <*smac_step*> <*smac_count*> } }

debug packet configure <*packet*> **type** [**eth** | **arp** | **ipv4** | **icmp** | **igmp** | **udp** | **tcp** | **wol**] **svid** <*vid*> { <*vid_step*> <*vid_count*> } { **svcfi** <*cfi*> } { **svpri** <*pri*> } { **svtpid** <*tpid*> }

debug packet configure <*packet*> **type** [**eth** | **arp** | **ipv4** | **icmp** | **igmp** | **udp** | **tcp** | **wol**] **vid** <*vid*> { <*vid_step*> <*vid_count*> } { **vcfi** <*cfi*> } { **vpri** <*pri*> } { **vtpid** <*tpid*> }

debug packet configure <*packet*> **type** [**ipv4 proto** <*proto*> | **icmp** | **igmp** | **udp** | **tcp**] **sip** <*sip*> { <*sip_step*> <*sip_count*> } **dip** <*dip*> { <*dip_step*> <*dip_count*> } { **tos** <*tos*> } { **frg** <*frg*> } { **ttl** <*ttl*> }

debug packet configure <*packet*> **type arp** <*arp-type*> { **sha** <*sha*> { <*sha_step*> <*sha_count*> } } { **spa** <*spa*> { <*spa_step*> <*spa_count*> } } { **tha** <*tha*> { <*tha_step*> <*tha_count*> } } { **tpa** <*tpa*> { <*tpa_step*> <*tpa_count*> } }

debug packet configure <*packet*> **type icmp** <*icmp-type*> { **code** <*code*> } { **info** <*info-value*> }

debug packet configure <*packet*> **type igmp** <*igmp-type*> { **code** <*code*> } { **group** <*group*> { <*step*> <*count*> } }

debug packet configure <*packet*> **type tcp sp** <*sp*> **dp** <*dp*> { **seq** <*seq*> } { **ack** <*ack*> } { **win** <*win*> } { **urg** <*urg*> } { **doff-flg** <*doff-flg*> }

debug packet configure <*packet*> **type udp sp** <*sp*> **dp** <*dp*>

debug packet configure <*packet*> **type wol wolmac** <*wol-dmac*> { **passwordMac** <*pwdMac*> } { **passwordIp** <*pwdIpv4*> }

debug packet create <*name*>

debug packet delete <*packet*>

debug packet read <*packet*> { **pcap** } { **raw** }

debug packet save <*packet*> { **pcap** }

debug packet show <*packet*>

debug packet show packets

C.37 ntp Debug Commands

debug ntp run [**internal** | **commands** <*command_name*> { <*args1*> } { <*args2*> } { <*args3*> } { <*args4*> }]

debug ntp show kern-info

debug ntp show loop-info

debug ntp start ntpd { <*debug-level*> }

debug ntp terminate ntpd

C.38 openflow Debug Commands

debug openflow run <*cmd*> { <*string*> | <*parm1*> } { <*parm2*> } { <*parm3*> } { <*parm4*> }

debug openflow show [**ofproto-exos** | **tables** | **vendor-table**]

debug openflow { **on** | **off** | { **on** } { **verbosity** <*verbosity*> { **output** <*output_file*> } | **output** <*output_file*> { **verbosity** <*verbosity*> } } }

C.39 ospf Debug Commands

debug ospf show epm

debug show ospf route { **origin** [**ospf-intra** | **ospf-inter** | **ospf-extern1** | **ospf-extern2**] | <*ipaddress*> <*netmask*> | <*ipNetmask*> }

jerry configure ospf add host <*ipaddress*> <*cost*> <*area-identifier*>

jerry configure ospf delete host <*ipaddress*>

jerry configure ospf lsa-timer-multiple <*seconds-in-minute*>

jerry ospf <*func*> { <*param1*> } { <*param2*> }

jerry ospfdebug <*func*> { <*param1*> } { <*param2*> }

jerry show ospf clients { **summary** | **detail** | **verbose** }

jerry show ospf host

jerry show ospf lsa-timer-multiple

jerry show ospf lsp-info

C.40 ospfv3 Debug Commands

debug ospfv3 [**enable** | **disable**] { **domain** <*domainName*> } **capability te**

debug ospfv3 [**enable** | **disable**] { **domain** <*domainName*> } **use-ip-router-alert**

debug ospfv3 configure { **domain** <*domainName*> } [{ **vlan** } <*vlan-name*> | { **tunnel** } <*tunnel-name*>] **neighbor** [**add** | **delete**] <*ip-address*>

debug ospfv3 configure { **domain** <*domainName*> } **ase-limit** <*number*> { **timeout** <*seconds*> }

debug ospfv3 configure { **domain** <*domainName*> } **ase-summary add** <*ipAddress*> / <*maskLen*> **cost** <*cost*> { **tag** <*number*> }

debug ospfv3 configure { **domain** <*domainName*> } **ase-summary delete** <*ipAddress*> / <*maskLen*>

debug ospfv3 configure { **domain** <*domainName*> } **lsa-batch-interval** <*seconds*>

debug ospfv3 create domain <*domainName*>

debug ospfv3 delete domain <*domainName*>

debug ospfv3 disable { **domain** <*domainName*> } **originate-default**

debug ospfv3 enable { **domain** <*domainName*> } **originate-default** { **always** } **cost** <*cost*> **type** [**ase-type-1** | **ase-type-2**] { **tag** <*number*> }

debug ospfv3 show { **domain** <*domainName*> } <*opcode*>

debug ospfv3 show { **domain** <*domainName*> } **ase-summary**

C.41 pim Debug Commands

debug pim [**show** | **send-bulk**] **msdp-local-sa-table**

debug pim [**show** | **send-bulk**] **msdp-remote-group-table**

debug pim show msdp-remote-source-route-table

debug pim show { **ipv4** | **ipv6** } **route destination** <*dest_addr*> **source** <*src_addr*> { **group** <*grp_addr*> }

debug pim { **ipv4** | **ipv6** } **configure state-refresh inter-packet-delay** <*value*>

debug pim { **ipv4** | **ipv6** } **configure state-refresh max-cache-entries-per-second** <*value*>

debug pim { **ipv4** | **ipv6** } **configure state-refresh timer origination-timer** [**start** | **stop**]

debug pim { **ipv4** | **ipv6** } **configure** { **vlan** } <*vlan_name*> **propagation_delay** <*delay*> **override_interval** <*interval*>

debug pim { **ipv4** | **ipv6** } **join** { **group** } <*grp_addr*> { **vlan** } <*vlan_name*>

debug pim { **ipv4** | **ipv6** } **leave** { **group** } <*grp_addr*>

debug pim { **ipv4** | **ipv6** } **register-ageout** { **group** } <*grp_addr*> { **source** } <*src_addr*>

debug pim { **ipv4** | **ipv6** } **register-pdu** { **group** } <*grp_addr*> { **source** } <*src_addr*>

debug pim { **ipv4** | **ipv6** } **show** [**nbr-sec-addr** <*ip_address*> | **kill-entry**] { **vlan** } <*vlan_name*>

debug pim { **ipv4** | **ipv6** } **show mlag** [**chkpt-database** | **vlan-mlag-info** <*vlan_name*>]

debug pim { **ipv4** | **ipv6** } **show state-refresh** { <*group_addr*> { <*source_addr*> } | **elapsed-time** [**gt** | **lt**] <*threshold*> | **src-active-time** }

debug pim { **ipv4** | **ipv6** } **show** { **vlan** } <*vlan_name*> **join_prune_override_interval**

debug pim { **ipv4** | **ipv6** } **show** { **vlan** } <*vlan_name*> **secondary-ipaddress**

jerry pim set memstate <*state*>

jerry pim show msdp-remote-source-route-table

jerry pim { **ipv4** | **ipv6** } [**clear** | **dump**] **stats** { { **vlan** } <*name*> }

jerry pim { **ipv4** | **ipv6** } [**register** | **unregister**] **ipprefix** <*ipaddress*>

jerry pim { **ipv4** | **ipv6** } **create cache** { <*number*> }

jerry pim { **ipv4** | **ipv6** } **set command** <*num*>

jerry pim { **ipv4** | **ipv6** } **show** [**route** | **igmp**]

jerry pim { **ipv4** | **ipv6** } **show cache** <*grp_addr*> <*src_addr*>

C.42 polMgr Debug Commands

jerry disable policy debug <*client*>

jerry enable policy debug <*client*>

jerry save [**policy** <*name*> | **policy-client** <*client*> **policy** <*name*>] { <*dname*> }

jerry show policy debug <*client*>

C.43 ptpV2 Debug Commands

debug ptp [**enable** | **disable**] **network-clock boundary acceptable-master** { **vlan** } <*vlan_name*>

debug ptp configure network-clock boundary add acceptable-master <*ipv4_address*> { **priority1** [<*priority*> | **unused**] }

debug ptp configure network-clock boundary delete acceptable-master <*ipv4_address*>

debug ptp show network-clock boundary acceptable-master

C.44 pwmib Debug Commands

debug configure pw tdm <*pw-name*> **index** <*pw-index*> **id** <*pwid*>

debug configure pw tdm <*pw-name*> **index** <*pw-index*> **psn** <*psnType*>

debug create pw tdm <*pw-name*>

debug delete pw tdm <*pw-name*> **index** <*pw-index*>

debug pwmib show [**globalData** | **pwIndexMap** | **pwTable** { **all** | **pwIndex** <*pwmibIndex*> | **pwID** <*pwmibID*> } | **client** | **memory**]

C.45 rip Debug Commands

jerry rip <*func*> { <*param1*> } { <*param2*> }

jerry rip [**memZone** <*zoneNo*> | **allocMem** <*pagesNumber*> [**get** | **free**]]

jerry show rip <*opcode*>

jerry snmp get <*object_identifier*>

C.46 rtmgr Debug Commands

debug rtmgr [**enable** | **disable**] **iproute show direct-host-rte**

debug rtmgr configure iproute replace [<*ipaddress*> <*netmask*> | <*ipNetmask*>] { <*gateway1*> } { <*gateway2*> } { <*gateway3*> } { <*gateway4*> } { **vr** <*vrname*> }

debug rtmgr configure memoryTracking size <*size*>

debug rtmgr configure rpf { **ipv4** } [**add** | **delete**] **table** [**unicast** | **multicast**] { **vr** <*vrname*> }

debug rtmgr configure rpf { **ipv4** } **table** [**unicast** | **multicast**] **priority** <*priority*> { **vr** <*vrname*> }

debug rtmgr show gateway [**ipv4** | **ipv6**] { **detail** } **vr** <*vrname*>

debug rtmgr show iproute sharing chunking

debug rtmgr show iproute sharing vr <*vrname*>

debug rtmgr show memoryTracking

debug rtmgr show rpf { **ipv4** } **table** { **vr** <*vrname*> }

debug rtmgr show rpf { **ipv4** | **ipv6** } **registered-prefix** { <*ipaddress*> <*netmask*> | <*ipNetmask*> | **resolved** | **unresolved** | **client** [**pim** | **msdp** | **mbgp**] } { **vr** <*vrname*> }

debug rtmgr unconfigure memoryTracking size

jerry [**enable** | **disable**] **rtmgr forceNotifQueue**

jerry configure debug rtmgr <*level*>

jerry configure iproute [**add** | **del**] <*ipaddress*> <*masklen*> <*gateway*> <*number*> <*origin*> { **vr** <*vrname*> }

jerry configure tunnel [**add** | **del**] [**encap** <*ipv6Netmask*> <*remoteTunnelAddr*> | **decap** <*localTunnelAddr*> <*ipv6Destination*>]

jerry create lsp <*lspname*> [**rsvp-te** <*endpointAddr*> | **static** | **ldp** <*ipv4Netmask*>] **label** <*label*> **next-Hop** <*nhipaddress*>

jerry delete lsp <*lspname*>

jerry iproute { **ipv4** | **ipv6** } **compression** [**correction** | **validation**] { **vr** <*vrname*> }

jerry iproute { **ipv4** } **compression** [**compress** | **uncompress**] [<*ipaddress*> <*netmask*> | <*ipNetmask*>] { **flagOnly** } { **vr** <*vrname*> }

jerry iproute { **ipv4** } **compression** [**imm_children** | **parent** | **self**] [<*ipaddress*> <*netmask*> | <*ipNetmask*>] { **vr** <*vrname*> }

jerry iproute { **ipv6** } **compression** [**imm_children** | **parent** | **self**] [<*ipv6Netmask*>] { **vr** <*vrname*> }

jerry iproute { **ipv6** } **compression** [**compress** | **uncompress**] [<*ipv6Netmask*>] { **flagOnly** } { **vr** <*vrname*> }

jerry rtmgr [**register** | **unregister**] [**prefix** | **prefixTrack**] [<*ipaddress*> <*netmask*> | <*ipNetmask*>] [**unicast** | **unimulti** | **multicast** | **rpf** | **lsp_and_uni** | **lsp_only** | **lgw_rpf**] { **vr** <*vrname*> }

jerry rtmgr vlan <*name*> [**linkup** | **linkdown**]

jerry show counter rtmgr { **vr** <*vrname*> }

jerry show ipmroute [**mcast-source** | **proto-list**] { **vr** <*vrname*> }

jerry show iproute { **ipv4** | **ipv6** } **compression** { **vr** <*vrname*> }

jerry show rtmgr [**prefix** | **prefixTrack**] { **ipv4** | **ipv6** } [**unicast** | **unimulti** | **multicast** | **lsp_only** | **lsp_and_uni** | **lgw_rpf**] [**resolved** | **unresolved**] { <*ipaddress*> <*netmask*> | <*ipNetmask*> } { **vr** <*vrname*> }

jerry show rtmgr client { <*clientId*> }

jerry show rtmgr halNotifQueue

jerry show rtmgr halNotifQueue count

jerry show rtmgr lsp

jerry show rtmgr protoattr { **vr** <*vrname*> }

jerry show rtmgr rtnode <*ipaddress*> <*masklen*> { <*vrname*> }

jerry show tunnel [**encap** | **decap**]

C.47 snmpMaster Debug Commands

debug snmp show access-profile

C.48 snmpMaster Debug Commands

debug snmp show access-profile

C.49 stp Debug Commands

debug stp [**enable** | **disable**] **stp** [**pktrx** | **pkttx** | **pktdecode** | **timers** | **mstp-trace** | **mstp-pkt-dump** | **rstp-trace** | **rstp-pkt-dump** | **rstp-state**]

debug stp show internal-state

C.50 techSupport Debug Commands

debug tech-support configure max-collectors <*max_collectors*>

C.51 telnetd Debug Commands

debug telnetd configure debug-level { <*level*> | **off** }

debug telnetd show access-profile

C.52 tftpd Debug Commands

debug tftpd configure debug-level { <*level*> | **off** }

jerry configure tftp [**read_only** | **read_write**]

jerry configure tftp port [<*portno*> | **default**]

jerry disable tftp

jerry enable tftp

C.53 thttpd Debug Commands

debug thttpd configure debug-level { <*level*> | **off** }

debug thttpd configure port [<*portno*> | **default**]

debug thttpd configure vr [<*vr_name*> | **all** | **default**]

debug thttpd show access-profile

C.54 trill Debug Commands

debug trill [**enable** | **disable**] **stackDebug** [**pdu** | **lsp** | **spf** | **event** | **ifsm** | **info** | **warn** | **all**]

debug trill [**enable** | **disable**] [**af** | **appointed-forwarder** | **designated**] <*port_list*> <*vlan_name*>

debug trill [**enable** | **disable**] [**m4** | **m6**] <*port_list*> <*vlan_name*>

debug trill configure [**add** | **delete**] **pif** <*nickname_id*>

debug trill configure hal [**add** | **delete**] **neighbor** <*port*> <*neighbor_nickname_id*> **mac-address** <*mac_address*>

debug trill configure hal access-disable-learning-time <*num*>

debug trill configure hal dynamic-dtree-extra-hop-count <*num*>

debug trill configure hal dynamic-unicast-extra-hop-count <*num*>

debug trill configure hal next-hop <*egress_nickname_id*> [**add** | **delete**] **port** <*port*> **neighbor** <*neighbor_nickname_id*>

debug trill configure hal tree-node <*tree_node_nickname_id*> [**add** | **delete**] **port** <*port*> **neighbor** <*neighbor_nickname_id*>

debug trill configure stack dtree-in-use <*dtree-nickname*>

debug trill configure stack dtree-nickname-to-compute <*dtree-nickname*>

debug trill configure stack force spf

debug trill configure stack hello-delay sec <*sec*> **usec** <*usec*>

debug trill configure stack number-of-dtrees-to-compute <*num*>

debug trill configure stack number-of-dtrees-to-use <*num*>

debug trill create hal trill nickname <*nickname_id*>

debug trill hal flush

debug trill show db [**adjacencies** | **neighbors** | **routes** | **dtrees** { **rpf** | **pruning** [**vlan** | **multicast**] }]

debug trill show stack dump [**summary** | **detailed**]

debug trill show stack dump vid-to-vlanInst-map

debug trill show stack fdb [**unicast** | **multicast** | **detail**]

debug trill show stack interface <*port_list*>

debug trill show stack lsp { **detailed** }

debug trill show stack neighbor

debug trill show stack nicknamedatabase

debug trill show stack pruning [**vlan** | **multicast**] { **dtree** <*name*> }

debug trill show stack rpf

debug trill show stack topology

debug trill show { **ports** [<*port_list*> | **all**] }

C.55 upm Debug Commands

debug upm show internal

debug upm show profile { **profileName** <*profile-name*> } { **eventType** <*event-type*> } { **timerName** <*timer-name*> }

jerry [**create** | **delete**] **upm file** <*profile-name*>

C.56 vlan Debug Commands

debug configure { **vr** } <*vr-name*> **l3vpn-label** <*label*>

debug ip dad show ifa-info

debug ip dad show { { { **vr** } <*vr_name*> | **vr all** | { **vlan** } <*vlan_name*> } { **tentative** | **valid** | **duplicate** } | { { **vr** } <*vr_name*> } <*ipaddress*> } { **detail** }

debug ipv6 dad show { { { **vr** } <*vr_name*> | **vr all** | { **vlan** } <*vlan_name*> | { **tunnel** } <*tunnel_name*> } { **tentative** | **valid** | **duplicate** } | { { **vr** } <*vr_name*> } <*ipaddress*> } { **detail** }

debug unconfigure { **vr** } <*vr-name*> **l3vpn-label**

debug vlan [**reset** | **stop**] **snapshot-counters**

debug vlan configure ports <*port_list*> **drive-strength** <*drive_strength*>

debug vlan restart protocol ipv6 vlan [**all** { <*num*> } | <*vlan1*> { <*vlan2*> }]

debug vlan show ports { <*port_list*> } **drive-strength**

debug vlan show private-vlan <*pvlan-name*>

debug vlan show { **lib** <*clientName*> | **kernel** } [**string** <*str*> | **ports** { **rbridge** | **tdm** } | **vlans** { <*vlan_name*> } | **vrs** | **rtifs** | **protocolVlans** | **snmpInfo** | **vpifs** { <*vlan_name*> { <*port*> | <*rbridge_nickname*> } } | **globalData** | **RegdProtocols** | **vmac** | **memory** { **vlanIf** { **snapShot** } } | **isid** | **mirroring** | **attachment-circuit** | **sharing** | **tunnels** { <*tunnel_name*> }]

debug wred [[**enable** [**global** | **port**] | **disable**] **dump** | **configure dyn-buf** <*dyn_buf_percent*>]

jerry configure vlan [**enable_untaggedPorts_on_10G6X** | **disable_untaggedPorts_on_10G6X**]

jerry show statistics vlan

jerry show super-vlans { <*super_vlan_name*> <*sub_vlan_name*> }

C.57 vmt Debug Commands

debug vm-tracking show client vlans

debug vm-tracking show dynamic-vlan int-mac { **ports** } <*portlist*>

debug vm-tracking show dynamic-vlan { **ports** } <*portlist*>

debug vm-tracking show vpp-to-vm

C.58 vrrp Debug Commands

jerry set vrrpdebug [**send-vmac-flush** { **chkpnt** | **api** } | <*debuglevelval*>]

jerry show vrrpdebug

jerry vrrp [**enable** | **disable**] **checkpointing**

C.59 vsm Debug Commands

debug vsm show peer { <*peer_name*> }

debug vsm show ports { **id** <*value*> | **ports** <*portlist*> | **peer** <*peer_name*> }

C.60 xmlc Debug Commands

debug xmlc [**enable** | **disable** | **start** | **terminate**] **memory-allocations**

debug xmlc [**start** | **clear**] **performance-analysis**

debug xmlc check <*target*> **message** <*message*>

debug xmlc configure [**all** | <*target*>] { **queue-size** <*queue-size*> } { **group-count** <*group-count*> } { **connect-timeout** <*connect-timeout*> } { **send-timeout** <*send-timeout*> } { **recv-timeout** <*recv-timeout*> } { **http-response** [**enable** | **disable**] }

debug xmlc configure performance-analysis module [**all** | <*module*>] **event-count** <*num-events*> { **event-delay** <*event-delay*> }

debug xmlc configure performance-analysis target [**all** | <*target*>] **queue-size** <*queue-size*> { **group-count** <*group-count*> }

debug xmlc dump { **target** <*target*> | **module** <*module*> }

debug xmlc show configuration { <*target*> }

debug xmlc show performance-analysis configuration

debug xmlc show performance-analysis results

Appendix D

Kernel /proc Filesystem

This chapter describes a subset of standard and EXOS-specific /proc filesystem entries that may be useful for debugging.

For additional information about standard /proc filesytem entries, see `linux26/Documentation/filesystems/proc.txt`.

D.1 /proc/cmdline

/proc/cmdline contains the kernel boot parameters given to the kernel by the bootloader at boot time.

```
/proc # cat cmdline
root=/dev/ram0 ro init=/linuxrc quiet dhash_max=5 ihash_max=5 mhash_max=4
                              bhash_max=7 pacman=1 ramdisk_blocksize=4096
/proc #
```

Figure D.1: Example /proc/cmdline Contents

D.2 /proc/core_dump_info

/proc/core_dump_info contains kernel oops and process crash information stored in NVRAM.

The stored crash information can be logically cleared by writing to the proc entry, e.g.:

```
echo clear > /proc/core_dump_info
```

When the stored crash information is logically cleared, it can still be retrieved (as long as it hasn't been overwritten by a subsequent crash) through the /proc/core_dump_info_last entry (see below). Note that this information can *only* be overwritten after it has been cleared (logically or permanently.)

The stored crash information can be erased by writing "clear_old" to the /proc entry, e.g.:

```
echo clear_old > /proc/core_dump_info
```

For more information, see chapter 6 on page 37.

D.3 /proc/core_dump_info_last

The /proc/core_dump_info_last entry allows kernel oops and process crash information stored in NVRAM to be retrieved after it has been logically cleared.

D.4 /proc/cpu_util_info

/proc/cpu_util_info contains cumulative CPU and memory utilization information for each process. An example of its contents is shown in Figure D.2.

The format of each line consists of:

```
/proc # cat cpu_util_info
init: 1 0.190000 0.320000 154 1060
keventd: 2 0.000000 0.000000 0 0
ksoftirqd_CPU0: 3 0.000000 0.000000 0 0
kswapd: 4 0.000000 0.000000 0 0
bdflush: 5 0.000000 0.000000 0 0
kupdated: 6 0.000000 0.050000 0 0
mtdblockd: 7 0.000000 0.000000 0 0
ftld: 8 0.000000 0.000000 0 0
watchdog thread: 30 0.000000 0.000000 0 0
jffs2_gcd_mtd1: 95 0.000000 0.000000 0 0
jffs2_gcd_mtd2: 99 0.000000 0.000000 0 0
ahd: 174 0.000000 0.010000 0 0
rpciod: 201 0.000000 1.030000 0 0
syslogd: 221 0.000000 0.020000 158 1057
cardmgr: 227 0.000000 0.050000 175 504
exos: 228 0.030000 0.020000 173 1067
epm: 229 0.830000 0.660000 773 3751
bcm-shell: 293 0.000000 0.200000 0 0
bcmDPC: 294 0.000000 0.000000 0 0
bcm.user.proxy: 296 0.000000 0.000000 110 566
tbcm_msm_tx0: 314 0.000000 0.000000 0 0
tbcm_msm_tx1: 315 0.000000 0.000000 0 0
epm: 328 0.000000 0.010000 773 3751
epm: 329 0.000000 0.040000 773 3751
epm: 330 0.000000 0.000000 773 3751
ds: 414 0.180000 0.260000 475 1732
...
```

Figure D.2: Example /proc/cpu_util_info Contents (Partial)

- Process name (with colon appended)
- Process ID
- Cumulative user time
- Cumulative system call time
- Memory RSS (in 4K pages)
- Memory VM size (in 4K pages)

D.5 /proc/cpuinfo

/proc/cpuinfo provides information about the CPU(s) in the system. As shown in Figure D.3, this includes a “system type" string that identifies the generic platform type, followed by the following information for each CPU:

```
/proc # cat cpuinfo
system type             : Extreme Networks SiByte
processor               : 0
cpu model               : SiByte SB1 V0.3  FPU V0.3
BogoMIPS                : 399.76
wait instruction        : no
microsecond timers      : yes
tlb_entries             : 64
extra interrupt vector  : yes
hardware watchpoint     : yes
VCED exceptions         : not available
VCEI exceptions         : not available
```

Figure D.3: Example /proc/cpuinfo Contents

processor	CPU index.
cpu model	CPU manufacturers information, including revision number.
BogoMIPS	Relative performance indicator.
wait instruction	Whether this CPU supports the “wait" instruction.
microsecond timers	Whether this CPU provides a hardware timer

	with microsecond resolution.
tlb_entries	Number of TLB entries.
extra interrupt vector	Whether this CPU provides a dedicated interrupt vector.
hardware watchpoint	Whether this CPU provides hardware watchpoint register(s).
VCED exceptions	Whether this CPU uses Virtual Coherence Exceptions for the data cache.
VCEI exceptions	Whether this CPU uses Virtual Coherence Exceptions for the instruction cache.

D.6 /proc/crashtest

This /proc filesystem entry exists for the purpose of unit testing various debugging facilities. Writing specific strings to this file will cause various kernel crashes, including:

- `crashtest_oom` - Deplete kernel memory by repeatedly calling kmalloc. This should result in a system reboot with OOM information in NVRAM.
- `crashtest_oops` - Force a kernel crash by dereferencing an invalid pointer. This should result in a system reboot with kernel oops information in NVRAM.
- `crashtest_irq` - Force a watchdog expiration by looping with interrupts disabled.
- `crashtest_panic` - Assert a kernel panic. This should result in a sytem reboot with kernel panic information in NVRAM.
- `crashtest_process` - Crash the current process with SIGABRT. This should not result in a system reboot, however process crash information should be written to NVRAM.
- `crashtest_watchdog` - Force watchdog expirations by looping in the kernel with interrupts enabled.

D.7 /proc/devices

D.8 /proc/diskstats

D.9 /proc/driver

D.10 /proc/eprofile

/proc/eprofile is the userland interface to the EXOS system profiler.

Reading this /proc entry returns binary data, so attempting to view its contents via "cat" or "more" is not recommended.

The following parameters can be written to this proc entry to control profiling:

- **buffer=**<*buffer_kbytes*> - Create a profiling buffer and start profiling.
- **status** - Show current profiling status.
- **clear** - Clear profiling buffer and restart profiling.

See Section 9.2 on page 73 for more information.

D.11 /proc/execdomains

D.12 /proc/interrupts

The /proc/interrupts file lists all hardware interrupts for which interrupt handlers have been registered. The first column contains the hardware interrupt number. The second column show the interrupt count for each interrupt for the first CPU. For SMP systems there will be additional columns for the remaining CPUs. Following the per-CPU interrupt count columns is a label identifying the hardware interrupt controller, and the final column contains the name(s) of the device(s) and/or kernel module(s) using each interrupt.

```
/proc # cat interrupts
           CPU0
  0:          0      SB1250-IMR  watchdog
  1:          0      SB1250-IMR  watchdog
  6:       3162      SB1250-IMR  i2c_sibyte
  7:     118218      SB1250-IMR  i2c_sibyte
  8:     311963      SB1250-IMR  uart0
 12:          0      SB1250-IMR  pcmcia
 19:     104329      SB1250-IMR  eth0
 20:      68254      SB1250-IMR  eth1
 33:      92539      SB1250-IMR  ide0
 35:        237      SB1250-IMR  ahd_pfc
 36:          0      SB1250-IMR  ahd_pfc
 41:        499      SB1250-IMR  ide1
 56:       4840      SB1250-IMR  linux-kernel-bde
 57:       9506      SB1250-IMR  linux-kernel-bde
 58:       4834      SB1250-IMR  linux-kernel-bde

ERR:          0
```

Figure D.4: Example /proc/interrupts Contents

D.13 /proc/kallsyms

The /proc/kallsyms file contains all text (code) symbols for the kernel. This table is used within the kernel for symbol lookups related to kernel oopses, and is symlinked to /System.map for userland applications.

```
/proc # head -n 16 kallsyms
80100400 t _stext
80100400 t ejtag_debug_handler
80100400 t stext
801005b4 t ejtag_return
801005c0 t nmi_handler
80100760 t rest_init
80100798 t do_linuxrc
8010088c t prepare_namespace
80100a2c t init
80100bc0 t __compute_return_epc
80100ed0 t cpu_idle
80100f38 t start_thread
80100fb4 t exit_thread
80100fbc t flush_thread
80100fc4 t copy_thread
8010112c t dump_task_fpu
...
```

Figure D.5: Example /proc/kallsyms Contents

D.14 /proc/meminfo

The `/proc/meminfo` file contains statistics related to memory allocation within the system. As example of the contents of this file are shown in figure D.6 on the facing page,

A brief description of each the contained values is as follows:

- **MemTotal** - The total amount of usable physical memory in the system, not including the memory used to load the kernel and initial ramdisk.
- **MemFree** - The total amount of memory in the system that is not currently in use for any purpose.
- **Buffers** - The amount of physical memory in use for file buffers.
- **Cached** - The amount of physical memory in use for the page cache.
- **SwapCached** - The amount of swap storage in use for the page cache.
- **Active** - The amount of allocated memory that has been used recently.
- **Inactive** - The amount of allocated memory that has not been used recently.
- **HighTotal** - The total amount of system "high" memory. Most commonly in MIPS systems, "high memory" is all physical RAM beyond 256 MB.

```
/proc # cat meminfo
MemTotal:         255164 kB
MemFree:            3144 kB
Buffers:           36144 kB
Cached:            80628 kB
SwapCached:            0 kB
Active:            79076 kB
Inactive:         108576 kB
Active(anon):      35364 kB
Inactive(anon):    35536 kB
Active(file):      43712 kB
Inactive(file):    73040 kB
HighTotal:             0 kB
HighFree:              0 kB
LowTotal:         255164 kB
LowFree:            3144 kB
SwapTotal:             0 kB
SwapFree:              0 kB
Dirty:                 0 kB
Writeback:             0 kB
AnonPages:         70908 kB
Mapped:            17152 kB
Slab:              29184 kB
SReclaimable:       7904 kB
SUnreclaim:        21280 kB
PageTables:         1472 kB
NFS_Unstable:          0 kB
Bounce:                0 kB
WritebackTmp:          0 kB
CommitLimit:      127580 kB
Committed_AS:     830880 kB
VmallocTotal:    1015800 kB
VmallocUsed:       38344 kB
VmallocChunk:     956944 kB
```

Figure D.6: Example /proc/meminfo Contents

- **HighFree** - The total amount of system "high" memory that is not currently in use.
- **LowTotal** - The total amount of system "low" memory. Most mmonly in MIPS systems, "low memory" is physical RAM in the first 256 MB of the physical address space. Note that only "low memory" is directly usable within the kernel for normal memory allocations (e.g. via kmalloc()).
- **LowFree** - The total amount of system "low" memory that is not currently allocated.
- **SwapTotal** - The total amount of swap space.
- **SwapFree** - The total amount of unused swap space.
- **Dirty** - The total amount of memory containing pages that are waiting to be written back to disk.
- **Writeback** - The total amount of memory containing pages that are being written to disk.
- **AnonPages** - The total amount of memory containing pages that are mapped anonymously (i.e. not backed by a file on disk) in user-space.
- **Mapped** - The total amount of memory containing pages that are mapped to files (e.g. for executables and shared libraries.)
- **Slab** - The total amount of memory in use for the kernel slab cache.
- **SReclaimable** - The total amount of memory in use for the kernel slab cache that can be freed for other uses if necessary.
- **SUnreclaim** - The total amount of memory in use for the kernel slab cache that cannot be freed for other uses if necessary.
- **PageTables** - The total amount of memory in use for page tables.
- **NFS_Unstable** - The total amount of memory in use for pages that have been written to an NFS server, but have not yet been committed to storage.
- **Bounce** - The total amount of memory in use for bounce buffers.
- **WritebackTmp** - The total amount of memory in use for for temporary filesystem writeback buffers.
- **CommitLimit** - The amount of memory that could be allocated at this time if strict overcommit accounting is enabled.
- **Committed_AS** - This value represents the size of physical memory that would be required, based on current allocations, to guarantee that the system will not run out of memory. (Note that Linux allows pages to be allocated without reserving physical memory for those pages - e.g. a process can malloc() a 1GB chunk of memory, but the kernel will not allocate a physical page until that page is accessed.)

- **VmallocTotal** - The total size of the vmalloc address space.
- **VmallocUsed** - The total total amount of memory currently in use for vmalloc allocations. See Section D.24 on page 144 for details of current vmalloc allocations.
- **VmallocChunk** - The size of the largest contiguous region of the vmalloc address space.

D.15 /proc/meminfo_better

The /proc/meminfo_better file contains a superset of the information in `/proc/meminfo`, with the following two additional values:

- **LowMemFreeAct** - The "actual" amount of free low memory, taking into account any low memory that could be freed by shrinking slab caches, evicting clean pages from the page cache, or other means.
- **LowMemFreeAct** - The "actual" total amount of free memory, including both low and high memory, and taking into account any low memory that could be freed by shrinking slab caches, evicting clean pages from the page cache, or other means.

D.16 /proc/modules

The /proc/modules file contains information about kernel modules currently loaded in the system. The format of the information is as follows:

- Column 1: The name of the kernel module.
- Column 2: The amount of memory allocated for the kernel module at load time (in bytes).
- Column 3: The reference count for the module.
- Column 4: The list of other kernel modules referencing this module.

```
/proc # cat /proc/modules
exfib                   3912   0 (unused)
exsflow                10096   1
exacl                  37992   2
exsnoop                22248   1
exosmc                 29484   0 (unused)
exdos                   9316   2
bcmpkt                 27484   0 (unused)
exvlan                265228   2 [exsflow exacl exsnoop exosmc bcmpkt]
nettx                  69736  11 [exsflow exacl exsnoop exdos bcmpkt exvlan]
exosq                  35496  40
bcmhelper              14508   3
linux-uk-extr           7268   0 [bcmhelper]
linux-bcm-diag-full  6157080   0 [bcmpkt bcmhelper linux-uk-extr]
linux-uk-proxy          5880   0 [linux-bcm-diag-full]
linux-kernel-bde       13752   0 [linux-bcm-diag-full]
aspenpmap              13020   2 [bcmpkt bcmhelper linux-bcm-diag-full]
ahd                    49184   0 [bcmpkt linux-bcm-diag-full aspenpmap]
extrpfc                 4608   2
extrfpga                3648  11
pca9555                 4144  15
eeprom16                4408  17
adm1066                 5512  13
lm75                    3824  14
pca9548                 5632   1
blkmtd                  9440   4
mtdram                  1840   4
watchdog               22896   1
exosnvram               4640   0
```

Figure D.7: Example /proc/modules Contents

D.17 /proc/pci

The /proc/pci file contains information about PCI devices in the system.

```
/proc # cat /proc/pci
PCI devices found:
  Bus  0, device   0, function  0:
    Class 0600: PCI device 166d:0001 (rev 3).
      Prefetchable 32 bit memory at 0x60000000 [0x7fffffff].
      Prefetchable 32 bit memory at 0x70000000 [0x7fffffff].
      Prefetchable 32 bit memory at 0x71000000 [0x71ffffff].
      Prefetchable 32 bit memory at 0x0 [0xffffffff].
      Prefetchable 32 bit memory at 0x80000000 [0xffffffff].
  Bus  0, device   5, function  0:
    Class 0280: PCI device 14e4:5675 (rev 1).
      IRQ 56.
      Master Capable.  Latency=252.
      Non-prefetchable 64 bit memory at 0x4ffd0000 [0x4ffdffff].
  Bus  0, device   6, function  0:
    Class 0280: PCI device 14e4:5675 (rev 1).
      IRQ 57.
      Master Capable.  Latency=252.
      Non-prefetchable 64 bit memory at 0x4ffe0000 [0x4ffeffff].
  Bus  0, device   7, function  0:
    Class 0280: PCI device 14e4:5675 (rev 1).
      IRQ 58.
      Master Capable.  Latency=252.
      Non-prefetchable 64 bit memory at 0x4fff0000 [0x4fffffff].
```

Figure D.8: Example /proc/pci Contents

D.18 /proc/prom_envp

The /proc/prom_envp file contains information passed to the kernel by the boot loader.

```
/proc # cat /proc/prom_envp
memsize=512
cpuclock=600000000
busclock=100000000
systype=Aspen_MSM
brkcmd=l -r @cpc 1
datasize=-b
dlecho=off
dlproto=none
bootp=no
hostport=tty0
inalpha=hex
inbase=16
moresz=10
prompt=BootRom >
regstyle=sw
regsize=32
rptcmd=trace
trabort=^K
ulcr=cr
uleof=%
heaptop=80100000
showsym=yes
fpfmt=both
fpdis=yes
activeBootRom=default
extreme_board_id=0x00030002
eth0macaddr=00:04:96:1d:05:20
selfslotno=5
alt_boot_vers=1.0.0.14
def_boot_vers=1.0.1.7
warm_restart=0
eth1macaddr=02:00:00:11:05:01
bootmode=00000000
```

Figure D.9: Example /proc/prom_envp Contents

D.19 /proc/slabinfo

/proc/slabinfo contains memory allocation statistics for each "slab cache" in use by the kernel. See section 7.7 on page 57 for more information regarding the contents of this file.

```
/exos/bin # cat /proc/slabinfo
slabinfo - version: 2.1
# name            <active_objs> <num_objs> <objsize> <objperslab> <pagesperslab> : tunables <limit> <batchcount> <sharedfactor> : slabdata <active_sla
ndisc_cache2           1     20    192   20    1 : tunables  120   60    0 : slabdata      1      1      0
arp_cache2             0      0    192   20    1 : tunables  120   60    0 : slabdata      0      0      0
FldActHead           980   1015     16  203    1 : tunables  120   60    0 : slabdata      5      5      0
FldActAct           1176   1183     20  169    1 : tunables  120   60    0 : slabdata      7      7      0
FldPatIp6              0      0     32  113    1 : tunables  120   60    0 : slabdata      0      0      0
...
```

Figure D.10: Example /proc/slabinfo Contents

D.20 /proc/extr/nvram/cmdline

The /proc/extr/nvram/cmdline file is the interface for viewing or modifying kernel parameters passed to the kernel at boot time by the bootrom.

```
/proc # cat extr/nvram/cmdline
pacman=1
```

Figure D.11: Example /proc/extr/nvram/cmdline Contents

D.21 /proc/net/dev

The

```
/proc # cat net/dev
                | Receive                                                | Transmit                                                |
   Interface |bytes    packets errs drop fifo frame compressed multicast|bytes    packets errs drop fifo colls carrier compressed|refcnt
          lo:    2940      36    0    0    0     0          0         0     2940      36    0    0    0     0       0          0     10
        eth0:116698624 110336    0    0    0     0          0         0  7983920   53755    0    0    0     0       0          0     25
        eth1:15597311  136198    0    0    0     0          0         0 22460519  109792    0    0    0     0       0          0      3
        sit0:       0       0    0    0    0     0          0         0        0       0    0    0    0     0       0          0      2
       netTx:72535611  206832    0    0    0     0          0         0        0       0    0    0    0     0       0          0    108
         lo1: 5603644   38914    0    0    0     0          0         0  5603644   38914    0    0    0     0       0          0     13
intVla_100000:13145747  136198    0    0    0     0          0         0 22459598  109792    0    0    0     0       0          0     11
         lo2:       0       0    0    0    0     0          0         0        0       0    0    0    0     0       0          0      8
     pimreg_2:       0       0    0    0    0     0          0         0        0       0    0    0    0     0       0          0      3
  Defau_F4244:       0       0    0    0    0     0          0         0        0       0    0    0    0     0       0          0      1
  test1_F4247:       0       0    0    0    0     0          0         0        0       0    0    0    0     0       0          0      3
  test2_F4249:       0       0    0    0    0     0          0         0        0       0    0    0    0     0       0          0      3
  test3_F424E:       0       0    0    0    0     0          0         0        0       0    0    0    0     0       0          0      3
```

Figure D.12: Example /proc/net/dev Contents

D.22 /proc/extr/watchdog

The /proc/extr/watchdog file is the interface to the system watchdog kernel module. The current status and configuration of the watchdog subsystem can be queried via, and sample output is shown in figure D.13:

cat /proc/extr/watchdog

```
/exos/bin # cat /proc/extr/watchdog
watchdog - Watchdog Enabled
intrNum: -1 jiffies: 961695 bootedFlag 0x1 intrCount: 0
Hardware Configuration:
   Timeout: 120000(msec) mode: 1

Application Configuration:
      Instance: 0 Name:  timeout: 0(msec) mode: 0
      inUse: 0 MyJiffies: 961384 delta: 311

      Instance: 1 Name: unknown timeout: 120000(msec) mode: 0
      inUse: 0 MyJiffies: 8854 delta: 952841
```

Figure D.13: Example /proc/extr/watchdog Contents

Requests to the watchdog kernel module are made via e.g.:

echo *request-string* > **/proc/extr/watchdog**

Supported requests include:

- "**readReg** <*regAddr*>" - Read a SiByte register; e.g. "echo 'readReg 0x10020008' > /proc/sys/watchdog" will print the value of the SiByte System Configuration Register.
- "**writeReg** <*regAddr*> <*value*>" - Write a SiByte register.
- "**enableSibyteWdg**" - Enables SiByte watchdog timer.
- "**disableSibyteWdg**" - Disables SiByte watchdog timer.
- "**testSibyteWdg** <*value*>" - Test SiByte watchdog timer. Useful values include:
  - 0,1 - Normal operation.
  - 2 - Don't reset watchdog timer, otherwise process as normal (tests software watchdog reset function.)
  - 3 - Don't reset watchdog timer, don't reset system via software (tests reset function of hardware watchdog.)
- "**enableuCWdg**" - Enables microcontroller watchdog timer.
- "**disableuCWdg**" - Disables microcontroller watchdog timer.

- "**testuCWdg** <*value*>" - Test microcontroller watchdog timer. Useful values are:
  - 0 - Normal operation.
  - 1 - Don't reset watchdog timer, otherwise process as normal (tests microcontroller watchdog reset function.)
- "**disableSystemReset**" - Disables software system reset (allows debugging).
- "**dumpHwWdg**" - Dumps watchdog state.
- "**dumpBusError**" - Dumps internal SiByte error status useful for debugging hardware bus errors. [1]
- "**clearBusError**" - Clears internal SiByte error status.
- "**cleanUpTimer** <*application*>" - Disables application watchdog for the specified application; e.g. "echo 'cleanUpTimer epm' > /proc/sys/watchdog" will disable the application watchdog timer for epm (useful when debugging.)

D.23 /proc/sys/kernel/printk

The /proc/sys/kernel/printk file contains four integer values, interpreted as follows:

1. The first is referenced in the kernel as "console_loglevel", and controls whether a particular printk message will be output to the system console based on the printk log level; any printk with a log level equal to or less than "console_loglevel" will be output to the serial console.
2. The second value is referenced in the kernel as "default_message_loglevel", and contains the default log level to be used for messages that do not explicitly specify a log level.
3. The third value is referenced in the kernel as "minimum_console_loglevel", and contains the minimum value "console_loglevel" is allowed to be set to via the syslog() system call.
4. The fourth value is referenced in the kernel as "default_console_loglevel", and contains the value used to initialize "console_loglevel" when console output is enabled by the syslog() system call.

D.24 /proc/vmallocinfo

Figure D.14 on the facing page contains an example of the contents of `/proc/vmallocinfo`. The fields presented in each line include:

- The kernel address range for the allocation.
- The size in bytes of the allocation.
- The function name and instruction offset of the allocator.
- For vmalloc allocations, the number of pages of memory used.
- For ioremap allocations, the physical address mapped.
- The type of the allocation, (ioremap, vmalloc, vmap, user, vpages).

[1] Ironically, on the 1125 this command **causes** a bus error.

```
/exos/bin # cat /proc/vmallocinfo
0xc0000000-0xc1001000 16781312 __ioremap+0x14c/0x394 ioremap
0xc1002000-0xc1045000  274432 jffs2_zlib_init+0x24/0xcc pages=66 vmalloc
0xc1046000-0xc1052000   49152 jffs2_zlib_init+0x5c/0xcc pages=11 vmalloc
0xc1053000-0xc1454000 4198400 init_mtdram+0x40/0xc4 pages=1024 vmalloc
0xc145a000-0xc145d000   12288 module_alloc_update_bounds+0x14/0x60 pages=2 vmalloc
0xc1491000-0xc14b1000  131072 module_alloc_update_bounds+0x14/0x60 pages=31 vmalloc
0xc14c0000-0xc14c9000   36864 module_alloc_update_bounds+0x14/0x60 pages=8 vmalloc
0xc14dc000-0xc14df000   12288 module_alloc_update_bounds+0x14/0x60 pages=2 vmalloc
0xc14f0000-0xc14f9000   36864 module_alloc_update_bounds+0x14/0x60 pages=8 vmalloc
0xc1579000-0xc15f2000  495616 module_alloc_update_bounds+0x14/0x60 pages=120 vmalloc
0xc15ff000-0xc1607000   32768 module_alloc_update_bounds+0x14/0x60 pages=7 vmalloc
0xc1611000-0xc1614000   12288 module_alloc_update_bounds+0x14/0x60 pages=2 vmalloc
0xc1620000-0xc1631000   69632 __ioremap+0x14c/0x394 ioremap
0xc1632000-0xc1637000   20480 module_alloc_update_bounds+0x14/0x60 pages=4 vmalloc
0xc1640000-0xc1651000   69632 __ioremap+0x14c/0x394 ioremap
0xc27be000-0xc36fe000 15990784 module_alloc_update_bounds+0x14/0x60 pages=3903 vmalloc vpages
0xc36ff000-0xc3703000   16384 module_alloc_update_bounds+0x14/0x60 pages=3 vmalloc
0xc3710000-0xc3719000   36864 module_alloc_update_bounds+0x14/0x60 pages=8 vmalloc
0xc373a000-0xc374e000   81920 module_alloc_update_bounds+0x14/0x60 pages=19 vmalloc
0xc3766000-0xc3776000   65536 module_alloc_update_bounds+0x14/0x60 pages=15 vmalloc
0xc3784000-0xc378d000   36864 module_alloc_update_bounds+0x14/0x60 pages=8 vmalloc
0xc3792000-0xc3794000    8192 module_alloc_update_bounds+0x14/0x60 pages=1 vmalloc
0xc37c2000-0xc37df000  118784 module_alloc_update_bounds+0x14/0x60 pages=28 vmalloc
0xc37f2000-0xc37fd000   45056 module_alloc_update_bounds+0x14/0x60 pages=10 vmalloc
0xc385d000-0xc38b8000  372736 module_alloc_update_bounds+0x14/0x60 pages=90 vmalloc
0xc38c2000-0xc38c5000   12288 module_alloc_update_bounds+0x14/0x60 pages=2 vmalloc
0xc38d9000-0xc38e4000   45056 module_alloc_update_bounds+0x14/0x60 pages=10 vmalloc
0xc38f3000-0xc38fa000   28672 module_alloc_update_bounds+0x14/0x60 pages=6 vmalloc
0xc3904000-0xc3908000   16384 module_alloc_update_bounds+0x14/0x60 pages=3 vmalloc
0xc391b000-0xc3925000   40960 module_alloc_update_bounds+0x14/0x60 pages=9 vmalloc
0xc3932000-0xc393a000   32768 module_alloc_update_bounds+0x14/0x60 pages=7 vmalloc
0xc3950000-0xc395c000   49152 module_alloc_update_bounds+0x14/0x60 pages=11 vmalloc
0xc3966000-0xc396a000   16384 module_alloc_update_bounds+0x14/0x60 pages=3 vmalloc
0xc3972000-0xc3974000    8192 module_alloc_update_bounds+0x14/0x60 pages=1 vmalloc
0xc3978000-0xc397a000    8192 dumpAcl+0x14e0/0x1aa8 [exacl] pages=1 vmalloc
```

Figure D.14: Example /proc/vmallocinfo Contents

Appendix E

Kernel /debug Filesystem

E.1 /debug/mips/fpuemustats

The `/debug/mips/fpuemustats` directory reports statistics for MIPS floating-point instruction emulation, which is used for CPUs such as the RMI/Netlogic XLR and XLS series which lack support for floating-point operations in hardware.

Floating-point emulation statistics entries include:

- `/debug/mips/fpuemustats/cpu1ops` - The number of emulated coprocessor 1 instructions.
- `/debug/mips/fpuemustats/cpu1xops` - The number of emulated indexed coprocessor 1 instructions.
- `/debug/mips/fpuemustats/emulated` - Total number of emulated instructions.
- `/debug/mips/fpuemustats/errors` - Total number of FPU emulation errors.
- `/debug/mips/fpuemustats/loads` - Total number of emulated FPU load instructions.
- `/debug/mips/fpuemustats/stores` - Total number of emulated FPU store instructions.

Excessive rates of floating point instruction emulation can impair performance.

E.2 /debug/mips/unaligned_instructions

The `/debug/mips/unaligned_instructions` file reports the total number of emulated load/store instructions resulting from unaligned memory accesses.

Excessive rates of unaligned load/store instruction emulation can impair performance.

E.3 /debug/mips/unaligned_action

The `/debug/mips/unaligned_action` file controls the action taken by the kernel when an unaligned access exception occurs. This can be useful in isolating specific code that is performing unaligned accesses.

The unaligned action can be set by writing a numeric character to this file. Settings include:

- "0" - Emulate quietly (UNALIGNED_ACTION_QUIET). This is the default action.
- "1" - Send SIGBUS to process performing the unaligned access. (UNALIGNED_ACTION_SIGNAL).
- "2" - Emulate normally, but output register and stack information to kernel log. (UNALIGNED_ACTION_SHOW).

E.4 /debug/tracing/

The `/debug/tracing` directory contains controls for various kernel tracing features, including scheduler tracing.

Entries in this directory include:

- `/debug/tracing/README` - Brief usage instructions and examples.
- `/debug/tracing/available_filter_functions` - Names of functions for function tracing.

- `/debug/tracing/available_tracers` - Names of available tracers (currently “function”, “sched_switch”, and “nop”).
- `/debug/tracing/current_tracer` - Name of currently active tracer. Defaults to “nop”, may be modified by writing name of desired tracer to this file.
- `/debug/tracing/dyn_ftrace_total_info` -
- `/debug/tracing/failures` -
- `/debug/tracing/iter_ctrl` - Miscellaneous controls for tracing and formatting (print-parent, sym-offset, sym-addr, verbose, raw, hex, bin, block, stacktrace, sched-tree, ftrace_printk), each control can be prefixed with "no" to disable.
- `/debug/tracing/latency_trace` -
- `/debug/tracing/set_ftrace_filter` -
- `/debug/tracing/set_ftrace_trace` -
- `/debug/tracing/trace` - Trace buffer contents.
- `/debug/tracing/trace_entries` - Capacity of trace buffer.
- `/debug/tracing/trace_marker` -
- `/debug/tracing/trace_pipe` - Pipe interface to user-space trace reader.
- `/debug/tracing/tracing_cpumask` - Controls which CPUs tracing is allowed on.
- `/debug/tracing/tracing_enabled` - Master enable/disable for tracing.
- `/debug/tracing/tracing_max_latency` -
- `/debug/tracing/tracing_on` -
- `/debug/tracing/tracing_thresh` -

Index

/debug Filesystem, 147
- /debug/mips/fpuemustats, 147
- /debug/mips/unaligned_action, 147
- /debug/mips/unaligned_instructions, 147
- /debug/tracing/, 147
- /debug/tracing/README, 147
- /debug/tracing/available_filter_functions, 147
- /debug/tracing/available_tracers, 148
- /debug/tracing/current_tracer, 148
- /debug/tracing/dyn_ftrace_total_info, 148
- /debug/tracing/failures, 148
- /debug/tracing/iter_ctrl, 148
- /debug/tracing/latency_trace, 148
- /debug/tracing/set_ftrace_filter, 148
- /debug/tracing/set_ftrace_trace, 148
- /debug/tracing/trace, 148
- /debug/tracing/trace_entries, 148
- /debug/tracing/trace_marker, 148
- /debug/tracing/trace_pipe, 148
- /debug/tracing/tracing_cpumask, 148
- /debug/tracing/tracing_enabled, 148
- /debug/tracing/tracing_max_latency, 148
- /debug/tracing/tracing_on, 148
- /debug/tracing/tracing_thresh, 148

/proc Filesystem, 135
- /proc/cmdline, 135
- /proc/core_dump_info, 135
- /proc/core_dump_info_last, 135
- /proc/cpu_util_info, 135
- /proc/cpuinfo, 136
- /proc/crashtest, 137
- /proc/devices, 137
- /proc/diskstats, 137
- /proc/driver, 137
- /proc/eprofile, 137
- /proc/extr/nvram/cmdline, 142
- /proc/extr/watchdog, 143
- /proc/interrupts, 137
- /proc/kallsyms, 138
- /proc/meminfo, 138
- /proc/meminfo_better, 140
- /proc/modules, 140
- /proc/net/dev, 143
- /proc/pci, 140
- /proc/prom_envp, 141
- /proc/slabinfo, 142
- /proc/sys/kernel/printk, 144
- /proc/vmallocinfo, 144

addr2line, 12

bcm.shell, 93
- dump Command, 95
- getreg Command, 94
- Help Command, 94
- setreg Command, 94
- show counters Command, 95
- Unit Selection, 93

Broadcom Platform Debugging, 93
- Quick Start, 93

Buffer Overrun Debugging
- Using Electric Fence, 56

Code Coverage, 97

Core Dump
- Quick Start, 25
- Using corePrep.pl, 25

Core Dumps, 21
- Causes, 22
- Controlling via /proc, 28
- Enabling from EXOS CLI, 24
- Forcing a core dump, 91
- Mechanics, 21
- Snapshot, 24
- Transferring, 25
- using gdb with, 13

corePrep.pl, 25
- Autologging, 28
- Automation, 26
- Customization, 28
- DDD GUI, 28
- Options, 27

Debug XMOD, 12

Debugging Memory Issues, 51

efence, 56

Electric Fence, 56

eprofile, 73

eprofmt, 76

exmmon, 52

EXOS Heap Tracing, 52

gdb, 12
- commands, 13
- Core Dumps, 13
- Kernel Debugging, 64

GDB Automation, 31

GDB Macros

Directory Server, 34
EMS Trace Buffer, 34
Example, 31
IPML, 34
Predefined, 33

ip, 18

ldd, 14
lsof, 16

Magic SysRq, 69
Memory Leak Debugging
enabling via PACMAN, 83
Heap Fragmentation, 51
Kernel kmalloc() tracing, 57
User vs. Kernel, 51
Using /proc/meminfo_better, 56
Using /proc/slabinfo, 57
Using EXOS Heap Tracing (exmmon), 52
mips-extr473-linux-gnu-addr2line, 12
mips-extr473-linux-gnu-gdb, 12
mips-extr473-linux-gnu-nm, 15
mips-extr473-linux-gnu-objdump, 15
mips-extr473-linux-gnu-readelf, 16
mpls command, 19

nm, 15
NVRAM Crash Dumps, 37
core.nvram Files, 38
Erasing from EXOS CLI, 38
Kernel Oops Details, 38, 48
Process Crash Details, 42
Showing, 37
Showing from EXOS CLI, 37
Showing from Linux Shell, 37
System Memory Depletion Details, 48
NVRAM Kernel Crash Dumps
Quick Start, 41
NVRAM Memory Depletion Debugging
Quick Start, 50
NVRAM Process Crash Dumps
Quick Start, 45
Using the crashfmt Script, 46

objdump, 15
Obtaining a Core Dump
Quick Start, 21

PACMAN, 79
allow /var log file, 83
back to original settings, 84
boot_broadcast, 82
boot_cf_alt_fsck, 80
boot_cf_config_fsck, 80
boot_debug, 82
boot_gateway, 82
boot_ipaddr msm a, 82
boot_ipaddr msm b, 82
boot_netmask, 82
continue with boot process, 84
debug mode, 83
enable heap tracing, 83
enable perf hooks, 80
Enabling from EXOS CLI, 80
Enabling from Linux Shell, 80
Enabling from PMON, 79
environment vars, 84
Invoking Linux Shell From, 80
load settings from..., 84
mount exos from, 83
nfs exos mount, 83
nfs var mount, 82
node failure restart, 83
nvram core dump info, 83
pacman autostart, 84
pacman autostart delay, 84
read settings from PASTE, 84
reboot, 84
save settings to..., 84
show settings for COPY, 84
start exos at boot, 82
var partition, 82
Performance Analysis
eprofmt output, 76
Profiling, 73
PMON, 87
boot, 87
cmdline, 88
config, 88
configip, 88
debug-mode, 88
pboot, 89
reboot, 87
Rescue/Recovery, 89, 90
printk, 69
Profiling
Enabling from EXOS CLI, 75

Quick Start Guides
Broadcom Shell, 93
corePrep.pl, 25
Eprofile, 73
Eprofile on I/O Modules, 74
NVRAM Kernel Crash Dumps, 41
NVRAM Process Crash Dumps, 45
NVRAM System Memory Depletion, 50
Obtaining a Core Dump, 21
Remote Debugging, 29
Using Code Coverage, 97

readelf, 16
Remote Debugging, 29
Operation, 30
Quick Start, 29
rgdb, 29

Scheduler Tracing, 70
Signals

SIGABRT, 23
SIGBUS, 23
SIGEMT, 24
SIGFPE, 23
SIGILL, 22
SIGQUIT, 22
SIGSEGV, 23
SIGSYS, 24
SIGTRAP, 23
SIGXCPU, 23
SIGXFSZ, 23
strace, 17

tcpdump, 17

Using Code Coverage
Quick Start, 97

watchdog, 92, 143
Disabling, 92
Forcing watchdog expiration (BD8800), 92
Forcing watchdog expiration (non-BD8800), 92